

$P = NP$ by Exclusion of the SAT Separator Endpoint

An AASC Endpoint-Rigidity Proof for Arbitrary Finite CNF-SAT

Amos Jay Maley

June 2026

Abstract

This manuscript proves $P = NP$ by excluding the SAT separator endpoint on the unrestricted finite CNF-SAT carrier. The branch-exclusion argument is AASC-internal: AASC supplies the endpoint-rigidity argument that excludes the bare negative branch, while Cook–Levin/Karp supplies only the standard endpoint correspondence from polynomial-time CNF-SAT to $P = NP$. The positive raw branch is $\text{Pos}_{\text{raw}} := \text{CnfSATInPolyTime}$, asserting deterministic polynomial-time decidability of arbitrary finite CNF-SAT. The bare negative branch Sep_{bare} is the raw complement of Pos_{raw} on the same carrier.

The proof is kernel-first. Theorem-bearing endpoint use is treated as governed construction, and governed construction is possible only under the kernel of admissibility, standing, reference, and irreversibility. The downstream AASC machinery is therefore not an external classifier or a source of theorem standing; it is the fixed-domain consequence layer forced by kernel necessity. Ordinary theoremhood, positive or negative, is kernel-internal when it is genuine theoremhood rather than raw trace.

The decisive step is the SAT-local endpoint asymmetry. The positive branch directly occupies the constructive endpoint carrier. The raw negative branch, by contrast, first transports by the SAT lower-bound/no-positive equivalence into standard lower-bound normal form and candidate endpoint-image exclusion. That support-level lower-bound content is not automatically forbidden. It becomes separator endpoint-image occupation only when the negative branch is used as the official endpoint-resolving branch on the same fixed carrier. Under that endpoint use, it induces an independent same-domain endpoint-status discriminator, and kernel-forced structural rigidity excludes such discriminators. Hence the SAT separator route is unavailable.

It follows that Sep_{bare} is excluded as the official endpoint-resolving negative branch. In the official endpoint evaluation, raw complementarity, $\text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}}$, then forces the positive branch. Therefore arbitrary finite CNF-SAT is decidable in deterministic polynomial time. The final passage from CnfSATInPolyTime to $P = NP$ is the standard Cook–Levin/Karp endpoint correspondence. The proof of branch exclusion is AASC-internal; external complexity theory is used only to identify the official endpoint consequence.

Mathematical-Status Lock and Lean Support Boundary

Mathematical-status lock. AASC is used here as a mathematical constraint formalism, not as philosophical commentary, metaphor, optional narration, or an interpretive overlay. The proof is the theorem chain stated in this manuscript. The AASC primitives used below—reference, standing, admissibility, irreversibility, bivalence, AMetric no-selector discipline, unique admissible interior, report preservation, and no-independent-discriminator closure—are theorem-bearing fixed-domain constraints. A same-mode objection must therefore identify a failure in a definition, theorem link, dependency claim, kernel-forcing argument, SAT bridge, endpoint-governance inference, or final

endpoint correspondence. It is not a refutation merely to classify the vocabulary as unfamiliar or to demand a circuit-lower-bound, diagonalization, natural-proofs, algebrization, proof-complexity, or algorithm-construction route.

Lean support boundary. The manuscript proof is the mathematical theorem chain below. The Lean4 material in the appendix is audit support for the local SAT-operator bridge, the kernel consequence interface, and the proof-spine routing. It is not a substitute for the manuscript proof and does not claim to formalize the Cook–Levin/Karp endpoint correspondence from foundations. A challenge to the Lean layer must identify a concrete mismatch between the cited Lean-facing support surface and the manuscript theorem chain. It is not enough to observe that Lean is not itself the proof, because the manuscript does not claim that it is.

Read This First: Endpoint Proof Spine

The live proof spine is intentionally short. In this display, O_R^- abbreviates official negative endpoint-resolution use. That use is supplied only inside official SAT endpoint evaluation, not by raw lower-bound content alone:

$$\begin{aligned}
\text{NonDegenerate}_{\text{SAT}}(R, model) &\Rightarrow K_{\text{SAT}} \Rightarrow A_{\text{SAT}}^+, \\
\text{Sep}_{\text{bare}} &\Rightarrow \text{StdLB}(model) \Rightarrow \text{CandImageExcl}(model), \\
\text{CandImageExcl}(model) \wedge O_R^- &\Rightarrow \text{Sep}_{\text{img}}^{\text{occ}}, \\
\text{Sep}_{\text{img}}^{\text{occ}} &\Rightarrow \text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}), \\
\text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}) \wedge \text{Sep}_{\text{img}}^{\text{occ}} &\Rightarrow \mathcal{D}_{\text{sep}}(model), \\
\text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}) &\Rightarrow \mathcal{N}_{\text{sep}}(model) \Rightarrow \neg \mathcal{D}_{\text{sep}}(model), \\
\mathcal{D}_{\text{sep}}(model) \wedge \neg \mathcal{D}_{\text{sep}}(model) &\Rightarrow \perp.
\end{aligned}$$

Thus $\text{Sep}_{\text{img}}^{\text{occ}}$ is impossible. In the official endpoint evaluation,

$$\neg(\text{Sep}_{\text{bare}} \wedge O_R^-), \quad \text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}},$$

so the positive branch follows, and $\text{Pos}_{\text{raw}} \Rightarrow P = NP$. The proof does not retype raw non-polynomiality in one step. The raw negative branch first becomes standard SAT lower-bound normal form on the context-bound encoded candidate image. That normal form becomes candidate endpoint-image exclusion. Candidate endpoint-image exclusion is still support-level lower-bound content unless it is used as the official endpoint-resolving negative branch. Only under that endpoint-resolution use does it become separator endpoint-image occupation, and only then does it become endpoint-status governance. This is the SAT analogue of the negative-branch bridge audit used throughout the AASC endpoint closure papers.

Dependency-Inversion Exclusion

The dependency order is not

$$\text{AASC framework} \Rightarrow \text{SAT constraints}.$$

The dependency order is

$$\text{non-degenerate fixed-carrier SAT endpoint regime} \Rightarrow \text{kernel governance already active}.$$

The official P versus NP target fixes the carrier of evaluation in complexity-theoretic form: the unrestricted finite CNF-SAT carrier, the positive polynomial-time endpoint slot, and the raw negative complement branch. It does not assert the positive endpoint and does not supply truth of the theorem. Once a proof, counterproof, endpoint resolution, or theorem-bearing act is evaluated as a determinate same-carrier endpoint act at all, it already requires target determinacy, step evaluability, act-time finality, and same-regime fidelity. Those kernel-neutral requirements force the roles later named reference, standing, irreversibility, and admissibility. AASC names and audits the consequence layer of those forced roles; it does not impose an optional proof standard from outside complexity theory.

Thus a reading that treats the kernel as an optional framework premise is a dependency inversion. It changes the argument from a necessity proof into a framework-application proof. A same-mode denial must identify a specific failure in the passage from target adequacy to the kernel roles, exhibit a faithful weaker same-carrier endpoint regime, or break one of the SAT-native bridge links. A refusal of vocabulary alone does not reach the proof.

Contents

Mathematical-Status Lock and Lean Support Boundary	1
Read This First: Endpoint Proof Spine	2
Dependency-Inversion Exclusion	2
1 Proof Class, Endpoint Validity, and Kernel-First Discipline	5
1.1 Proof-class lock	5
1.2 Necessity, not optional framework	5
1.3 Clay-valid as official mathematical endpoint	5
1.4 Endpoint and method	5
1.5 Kernel-first ordering	5
1.6 The SAT context object is neutral	6
1.7 Endpoint layers	6
2 Official Target and SAT Endpoint Carrier	7
2.1 The official P versus NP target	7
2.2 CNF satisfiability	7
2.3 Encoded candidate model adequacy	8
2.4 Raw bivalence is bare-level bivalence	10
3 Self-Contained Kernel Proof for Non-Degenerate Construction	11
3.1 Fixed-Domain Construction and Minimal Setting	11
3.2 Presupposition Structure of Derivation	20
3.3 Non-Derivability of the Kernel	22
3.4 Mutual Closure and Minimality	26
3.5 Fixed-Domain Closure Results	28
3.6 Consequences for Admissible Structure	31
3.7 Closure Against Additional Foundational Conditions	32
3.8 Mechanization Boundary and Proof-Theoretic Scope	33
3.9 Endpoint-facing kernel wrappers	34

4	A+ as Kernel-Forced Consequence Layer	35
5	Local SAT Kernel Packet and Weakening Resistance	36
5.1	Local SAT kernel packet	36
5.2	Weakening-resistance of the local SAT packet	37
6	Endpoint-Status Discriminator Discipline	40
7	SAT Branch Asymmetry and Separator Occupation	41
7.1	SAT-native lower-bound normal form and endpoint-status governance	44
7.2	SAT Negative Occupation Exhaustion and Endpoint-Status Bivalence	48
8	Route-Locus Exhaustion for SAT Separator Use	53
9	Endpoint-Image Resolution	55
10	Positive Report Classification	56
11	AASC Closure and Endpoint Correspondence	57
12	Denial Burden and Exhaustion of Remaining Routes	57
13	Conclusion	60
A	Lean4 SAT Operator Bridge Appendix	61
A.1	Release, environment, and audit runner	61
A.2	Declaration locations	62
A.3	Manuscript-to-Lean correspondence	63
A.4	Selected source excerpts	64
A.5	Focused axiom-output table	68
A.6	Audit scope and boundary	69
B	Standard CNF-SAT to P=NP Correspondence	70
C	Reference Integrity Appendix	70
D	Theorem Ladder and Anti-Circularity Audit	70
E	Notation Ledger	72

1 Proof Class, Endpoint Validity, and Kernel-First Discipline

1.1 Proof-class lock

The proof class is

kernel-forced endpoint-image discriminator closure.

It is not a circuit lower bound, diagonalization argument, relativization argument, natural-proofs argument, algebrization argument, proof-complexity lower bound, exhaustive-search lower bound, or empirical solver-hardness claim. It is a main-text mathematical proof about which theorem-bearing endpoint statuses can survive on the fixed SAT endpoint image once the kernel of non-degenerate construction is in force.

1.2 Necessity, not optional framework

The argument does not introduce AASC as an optional foundational system layered on top of the P versus NP problem. The kernel of non-degenerate construction, and the downstream consequences used in the proof, are derived as necessities forced by the minimal evaluability conditions required for any raw sequence to count as construction of a determinate same-domain object. These conditions are stated in kernel-neutral form before governed construction is defined. Once they are fixed, the governance roles of admissibility, standing, reference, and irreversibility follow as forced roles; the fixed-domain consequences of those roles include the structural rigidity principles that exclude independent same-domain endpoint-status discriminators. AASC is the systematic development of those necessity constraints. The positive endpoint is therefore not obtained by adopting a proprietary framework, but by applying the consequences of non-degenerate construction to the official unrestricted finite CNF-SAT carrier.

1.3 Clay-valid as official mathematical endpoint

Definition 1.1 (Clay-valid mathematical endpoint). In this manuscript, “Clay-valid” is used only in the mathematical endpoint sense. A Clay-valid endpoint targets the official P versus NP problem, preserves the standard definitions of P, NP, polynomial time, SAT, CNF-SAT, polynomial-time reductions, and NP-completeness, and reaches one of the official mathematical branches. The term is used only for this mathematical endpoint property.

1.4 Endpoint and method

The branch-exclusion step is supplied by the necessity constraints already forced by kernel-neutral target adequacy on the official carrier. AASC records those constraints and their fixed-domain consequences; it does not supply an optional foundational layer. The proof is not a diagonalization argument, circuit lower-bound argument, relativization argument, natural-proofs argument, proof-complexity argument, or constructive algorithm search. Classical complexity theory enters only at the endpoint-correspondence layer: unrestricted finite CNF-SAT is the standard NP-complete carrier, and deterministic polynomial-time decidability of that carrier entails $P = NP$ by the Cook–Levin/Karp correspondence. The proof of $\neg\text{Sep}_{\text{bare}}$ is AASC-internal.

1.5 Kernel-first ordering

The central dependency is

$$\text{EndpointUse}_R(B) \Rightarrow \text{governed construction} \Rightarrow K_R \Rightarrow A_R^+(B).$$

The reverse dependency is not used. AASC does not create standing, and reportability does not supply standing. Standing is one member of the kernel already presupposed by genuine governed construction. A proof of a proposition, when it is a proof rather than raw syntax, is therefore already kernel-internal.

Proposition 1.2 (Downstream status of AASC machinery). *In this manuscript, AASC, UEAP, ATS, AMetricity, no-selector discipline, no-third-status discipline, unique-interior discipline, standing conservation, and report-preservation discipline are downstream consequence machinery forced by the kernel under fixed-domain theorem-bearing endpoint use. None of them is a primitive source of theorem standing.*

Proof. The kernel proof in Section 3 begins with raw step-sequences and kernel-neutral target adequacy, then proves that target determinacy, step evaluability, act-time finality, and same-regime fidelity force reference, standing, irreversibility, and admissibility. The later AASC machinery is the endpoint interface of those roles: it preserves target, carrier, admissibility boundary, theorem-status reuse, redescription stability, and report integrity. Therefore it records forced consequences of the kernel rather than adding an independent source of standing. $\square$

1.6 The SAT context object is neutral

Definition 1.3 (Fixed SAT endpoint context). The context object

$$\mathcal{C}_{\text{SAT}}(R, m)$$

records only the official SAT/P-vs-NP endpoint setting: the unrestricted finite CNF-SAT carrier, the raw positive branch Pos_{raw} , the raw negative branch Sep_{bare} , candidate endpoint-image exclusion $\text{CandImageExcl}(\text{model})$, the separator endpoint-image occupation slot $\text{Sep}_{\text{img}}^{\text{occ}}$, and the fixed model/-carrier on which the raw branches are evaluated. It is a carrier-and-slot object. It is not an A+ certificate, not an AMetric premise, not a no-selector axiom, not a separator-classification premise, and not a positive endpoint premise.

Theorem 1.4 (Context non-smuggling). *The context object $\mathcal{C}_{\text{SAT}}(R, m)$ does not assert any of*

$$A_R^+(B), \quad \text{AMetricBoundary}(R), \quad A_{\text{sep}}, \quad \neg \text{Sep}_{\text{bare}}, \quad \neg \text{Sep}_{\text{img}}^{\text{occ}}, \quad \text{Pos}_{\text{raw}}.$$

It fixes the official SAT carrier and branch slots only.

Proof. By Definition 1.3, the clauses of $\mathcal{C}_{\text{SAT}}(R, m)$ are carrier and slot clauses. They say which endpoint image is under evaluation and which raw branches are being evaluated on that image. They do not classify any branch, exclude any branch, or assert the positive branch. All endpoint-status consequences are derived later from endpoint use plus the kernel. $\square$

1.7 Endpoint layers

Definition 1.5 (Bare proposition layer). The raw positive proposition is

$$\text{Pos}_{\text{raw}} := \text{CnfSATInPolyTime}.$$

The bare negative proposition Sep_{bare} is the raw complement branch for the same unrestricted finite CNF-SAT carrier.

Definition 1.6 (Candidate-image exclusion and separator occupation). The support-level SAT-native lower-bound image is $\text{CandImageExcl}(model)$: candidate endpoint-image exclusion on the exact context-bound encoded polynomial-time CNF-SAT candidate image. It is lower-bound content and may remain proof support, local obstruction, or ordinary theorem content.

The separator endpoint-image occupation $\text{Sep}_{\text{img}}^{\text{occ}}$ denotes candidate-image exclusion after it is used as the official endpoint-resolving negative branch on the fixed SAT carrier. Its formal Lean carrier is the object $\text{CnfSATImageSeparatorBranch}(R, model)$, but the manuscript keeps the support-level content $\text{CandImageExcl}(model)$ distinct from endpoint occupation $\text{Sep}_{\text{img}}^{\text{occ}}$. Thus the proof does not infer endpoint use from lower-bound content alone.

Definition 1.7 (Endpoint classification layer). The symbol A_{sep} denotes the role-specific separator endpoint-status factor, if such a factor is asserted for endpoint-image separator occupation $\text{Sep}_{\text{img}}^{\text{occ}}$. The Clay-valid separator endpoint is

$$\text{Sep}_{\text{Clay}} := \text{Sep}_{\text{img}}^{\text{occ}} \wedge A_{\text{sep}}.$$

The positive report classification is A_{pos} , and the Clay-valid positive endpoint is

$$\text{Pos}_{\text{Clay}} := \text{Pos}_{\text{raw}} \wedge A_{\text{pos}}.$$

These classification symbols are downstream status factors; they do not create kernel standing.

2 Official Target and SAT Endpoint Carrier

2.1 The official P versus NP target

The official P versus NP problem asks whether every language accepted by a nondeterministic polynomial-time algorithm is also accepted by a deterministic polynomial-time algorithm. The problem statement is

$$P \stackrel{?}{=} NP.$$

The proof works through encoded CNF satisfiability as the SAT endpoint carrier.

2.2 CNF satisfiability

For a CNF formula φ , an assignment a assigns Boolean values to its variables, and

$$\text{SAT}(\varphi) \iff \exists a \text{ Eval}(\varphi, a) = 1.$$

The formal carrier uses finite encodings of CNF formulas, the Boolean characteristic function satChar , and a Turing-machine polynomial-time certificate for the positive endpoint.

Definition 2.1 (Unrestricted finite CNF carrier; bounded-CNF clarification). Throughout this manuscript, “bounded-CNF” means the ordinary finitely encoded CNF-SAT carrier: arbitrary finite CNF formulas over a finite Boolean alphabet, with input length equal to the size of the encoded formula. It does not mean fixed-width k -CNF, fixed clause count, fixed variable count, bounded instance size, bounded search depth, promise-restricted CNF, parameterized tractable fragments, or any other restricted SAT fragment. Thus

$$\text{CnfSATInPolyTime}$$

is the assertion that arbitrary finite CNF-SAT is decidable by a deterministic polynomial-time machine.

2.3 Encoded candidate model adequacy

This subsection fixes the ordinary coding burden for the unrestricted finite CNF-SAT carrier. The encoded candidate model is not an AASC source of standing, not a separator, and not a hidden proof device. It is the standard effective presentation of deterministic polynomial-time CNF-SAT candidates used to state the positive endpoint and its raw complement. Its role is to make explicit that the later branch proof quantifies over the full official candidate image.

Definition 2.2 (Context-bound encoded CNF model).

$$\text{Model}_{\text{CNF}}(\text{model}, R)$$

means that *model* is the encoded CNF-SAT endpoint model associated with the fixed SAT context *R*: the carrier is arbitrary finite CNF over finite Boolean alphabets, the input measure is formula length, admissible encodings preserve the same endpoint-image carrier, and the encoded candidate image satisfies the adequacy packet below. The corresponding Lean structure is `CnfEncodedCandidateModel`; the adequacy packet is `CnfEncodedCandidateModelAdequate`.

Definition 2.3 (Encoded candidate model adequacy). For a context-bound encoded CNF model, $\text{Adeq}_{\text{CNF}}(\text{model})$ is the conjunction of four ordinary coding conditions:

$$\begin{aligned} \text{Adeq}_{\text{CNF}}(\text{model}) := & \text{Coverage}_{\text{CNF}}(\text{model}) \wedge \text{Sound}_{\text{CNF}}(\text{model}) \\ & \wedge \text{FailAdeq}_{\text{CNF}}(\text{model}) \wedge \text{ImageExact}_{\text{CNF}}(\text{model}). \end{aligned}$$

Here:

- (i) $\text{Coverage}_{\text{CNF}}(\text{model})$ says every deterministic polynomial-time CNF-SAT candidate procedure has a code in the model;
- (ii) $\text{Sound}_{\text{CNF}}(\text{model})$ says every code marked polynomial-time decodes to a deterministic polynomial-time CNF-SAT candidate procedure;
- (iii) $\text{FailAdeq}_{\text{CNF}}(\text{model})$ says SAT-failure status is invariant under equal decoded procedures, so no encoding artifact changes endpoint status;
- (iv) $\text{ImageExact}_{\text{CNF}}(\text{model})$ says the positive endpoint `CnfSATInPolyTime` is equivalent to the existence of an encoded polynomial-time candidate whose decoded procedure has no SAT failure.

These clauses are formalized by `CnfSATCandidateModelCoverage`, `CnfSATCandidateModelSoundness`, `CnfSATCandidateFailureAdequacy`, `CnfSATCandidateImageExactness`, and `CnfEncodedCandidateModelAdequate` in `AnticheckerReduction.lean`.

Definition 2.4 (Standard encoded polynomial-time SAT model). The standard encoded polynomial-time CNF-SAT model is the ordinary uniform machine model whose codes are pairs

$$(e, k),$$

where *e* is an effective code for a deterministic Boolean procedure on finite CNF inputs and *k* ∈ ℕ is a declared global polynomial-time bound. A code is marked polynomial-time exactly when the decoded procedure is licensed as a uniform deterministic candidate running within its declared polynomial bound on every finite CNF input. The same decoded procedure handles all finite CNF inputs; no advice strings, promise restrictions, fixed-size test sets, nonuniform circuit families, or per-length classifiers are part of this model.

Proposition 2.5 (Standard model instantiation). *The standard encoded polynomial-time CNF-SAT model of Definition 2.4 instantiates the adequacy packet of Definition 2.3. In particular, it satisfies coverage, soundness, failure adequacy, and candidate-image exactness for the unrestricted finite CNF-SAT carrier.*

Proof. Deterministic procedures over finite binary strings, and hence deterministic procedures over finite CNF encodings, admit ordinary effective descriptions. Pairing such a description with a declared global polynomial bound gives the standard candidate code. Coverage follows because every uniform deterministic polynomial-time CNF-SAT decider has some effective description and some polynomial exponent or bound. Soundness follows because a code marked polynomial-time is admitted only when the decoded procedure runs within the declared global polynomial bound on every finite CNF input. Failure adequacy follows because SAT failure is a property of the decoded procedure on lawful finite CNF inputs, not a property of the presentation of its code. Candidate-image exactness follows because `CnfSATInPolyTime` is precisely the existence of a uniform deterministic polynomial-time procedure deciding arbitrary finite CNF-SAT correctly on all finite CNF inputs. Therefore the standard encoded model supplies the same candidate image as the ordinary definition of P-time CNF-SAT decidability. $\square$

Definition 2.6 (SAT-failure predicate and uniformity convention). For an encoded polynomial-time candidate, $\text{FailsSAT}_{\text{model}}(\text{decode}_{\text{model}}(c))$ means that there exists a lawful finite CNF formula φ such that the decoded uniform deterministic procedure either returns the wrong Boolean value relative to $\text{SAT}(\varphi)$, fails to return a lawful Boolean answer on the encoded input, or fails to meet the declared global polynomial-time bound on that input. The predicate is evaluated over arbitrary finite CNF instances, not over a promise subset, a fixed-width fragment, a bounded-size table, a finite benchmark set, or a nonuniform family.

Remark 2.7 (Duplicate encodings and extensional harmlessness). The model does not require a unique code for each extensional procedure. Multiple presentations may decode to the same candidate, or to extensionally equivalent candidates on all finite CNF inputs. Duplicate encodings do no endpoint work: endpoint status is read from the decoded procedure and the SAT-failure predicate. The formal Lean adequacy clause records invariance under equal decoded procedures; strengthening this to full extensional equivalence would leave the branch route unchanged, because the proof uses coverage, soundness, and candidate-image exactness rather than uniqueness of code representatives.

Theorem 2.8 (Context-bound model adequacy). *For any context-bound encoded CNF-SAT endpoint model,*

$$\text{Model}_{\text{CNF}}(\text{model}, R) \Rightarrow \text{Adeq}_{\text{CNF}}(\text{model}).$$

Proof. By Definition 2.2, $\text{Model}_{\text{CNF}}(\text{model}, R)$ fixes the ordinary effective presentation of deterministic polynomial-time CNF-SAT candidate procedures on the unrestricted finite CNF carrier. That presentation includes coverage, sound decoding, failure adequacy, and exactness of the encoded positive endpoint image. In Lean these clauses are assembled by `cnfEncodedCandidateModelAdequate` from the model fields and the endpoint exactness theorem. $\square$

Theorem 2.9 (Encoded positive endpoint exactness). *On a context-bound adequate encoded CNF-SAT model,*

$$\text{Model}_{\text{CNF}}(\text{model}, R) \Rightarrow \left(\text{CnfSATInPolyTime} \leftrightarrow \exists c (\text{PolyCand}_{\text{model}}(c) \wedge \neg \text{FailsSAT}_{\text{model}}(\text{decode}_{\text{model}}(c))) \right).$$

The Lean anchor is `cnfSATInPolyTime_iff_exists_encoded_nonfailing_candidate`.

Proof. The forward direction is the coverage clause applied to a deterministic polynomial-time CNF-SAT decider for the unrestricted carrier: such a candidate has a code marked polynomial-time, and because it decides SAT correctly it has no SAT failure. The reverse direction is soundness plus failure adequacy: if an encoded polynomial-time candidate has no SAT failure, the decoded candidate is a deterministic polynomial-time procedure deciding arbitrary finite CNF-SAT. This is exactly `CnfSATInPolyTime`. The Lean theorem records this equivalence over `CnfEncodedCandidateModel`. $\square$

Theorem 2.10 (Encoded negative endpoint exactness). *On a context-bound adequate encoded CNF-SAT model,*

$$\text{Model}_{\text{CNF}}(\text{model}, R) \Rightarrow \left(\neg \text{CnfSATInPolyTime} \leftrightarrow \forall c (\text{PolyCand}_{\text{model}}(c) \Rightarrow \text{FailsSAT}_{\text{model}}(\text{decode}_{\text{model}}(c))) \right).$$

The Lean anchor is `cnfSATNotInPolyTime_iff_all_encoded_candidates_fail`.

Proof. Negating Theorem 2.9 gives the standard lower-bound normal form on the exact encoded candidate image: if there is no deterministic polynomial-time CNF-SAT decider, then every encoded polynomial-time candidate fails SAT; conversely, if every encoded polynomial-time candidate fails, no encoded non-failing polynomial-time candidate exists, and by coverage no unencoded deterministic polynomial-time decider has been missed. This is the formal equivalence proved in `cnfSATNotInPolyTime_iff_all_encoded_candidates_fail`. $\square$

Proposition 2.11 (Standing burden for model-adequacy objections). *A model-adequacy objection has standing only if it identifies a failure of standard-model instantiation, coverage, soundness, failure adequacy, or image exactness. Equivalently, the objector must exhibit a deterministic polynomial-time CNF-SAT candidate lost by the encoded image, an encoded candidate admitted despite not being a lawful uniform polynomial-time candidate, an encoding artifact that changes SAT-failure status, a mismatch between `FailsSAT` and ordinary failure to decide arbitrary finite CNF-SAT, or a mismatch between `CnfSATInPolyTime` and the existence of an encoded non-failing polynomial-time candidate. Merely objecting that the proof uses an encoded candidate model does not meet the burden.*

Proof. Every standard formulation of $\mathbf{P}$ quantifies over effective descriptions of uniform deterministic procedures together with polynomial-time bounds. The standard encoded model of Definition 2.4 makes that ordinary candidate image explicit. If standard-model instantiation, coverage, soundness, failure adequacy, and image exactness are preserved, the encoded image is neither smaller nor larger than the deterministic polynomial-time CNF-SAT candidate space relevant to `CnfSATInPolyTime`. Therefore a standing objection must identify a concrete defect in one of those clauses rather than treat encoding itself as a defect. $\square$

2.4 Raw bivalence is bare-level bivalence

Raw bivalence is stated at the bare layer:

$$\text{Pos}_{\text{raw}} \vee \text{Sep}_{\text{bare}}.$$

The final proof does not infer reportability from the raw disjunction. It uses the context-bound model adequacy packet of Subsection 2.3 and the SAT-local lower-bound/no-positive bridge

$$\text{Sep}_{\text{bare}} \Rightarrow \text{StdLB}(\text{model}) \Rightarrow \text{CandImageExcl}(\text{model})$$

to move the raw negative branch first into candidate endpoint-image exclusion. No reportability theorem is used in the bare-branch exclusion. Report acts reappear only after the positive endpoint has been derived.

3 Self-Contained Kernel Proof for Non-Degenerate Construction

This section internalizes the kernel proof in the manuscript. The kernel is not cited as an external AASC preference and is not treated as an appendix assumption. The definitions below begin with raw step-sequences and kernel-neutral target adequacy, prove that the roles of reference, standing, irreversibility, and admissibility are forced by determinate same-domain construction, prove non-derivability and no-faithful-lower-generator results, and derive the fixed-domain consequence package used later by the SAT endpoint argument.

3.1 Fixed-Domain Construction and Minimal Setting

This section records the minimal structural conditions under which a raw step-sequence is assessable as construction of a determinate object on a fixed domain. They are not introduced as optional definitions for a preferred formalism. They are fixed here because without them object identity, admissible stephood, and the distinction between continuation and repair are not stably assessable at all.

Definition 3.1 (Raw step-sequence). A raw step-sequence on a domain D is an ordered sequence of inscriptions, rewrites, transitions, or operations on D . As such it carries no automatic claim to determinate target identity, admissible stephood, or preservation of object identity across the sequence.

Definition 3.2 (Kernel-neutral target adequacy). A raw-sequence regime is target-adequate for the present inquiry iff some of its raw sequences are assessable under the following four requirements, stated without yet using the kernel as a definition of construction:

- (A1) *Target determinacy*: the sequence has a repeatable object of evaluation rather than merely a presentation-dependent trace.
- (A2) *Step evaluability*: its transitions are assessable as advancing, failing to advance, or departing from that same target rather than merely occurring after one another.
- (A3) *Act-time finality*: a later intervention that changes the status of an earlier performance yields a distinct act or distinct sequence rather than retroactively making the same performance admissible.
- (A4) *Same-regime fidelity*: identity-preserving redescription, encoding, or translation preserves target, step-status, and act-time classification; otherwise the presentation has changed regime or changed the object of evaluation.

These adequacy requirements are neutral with respect to the later vocabulary. They specify what must be preserved if a raw trace is to be assessed as construction of a determinate same-domain object at all. They are not proprietary terms of the present framework. They are the minimal commitments already incurred by any opponent who claims to have a counterexample to the paper's target: there must be something fixed, something done toward it, a stable distinction between the original act and later correction, and a preservation condition under redescription. A regime that denies any of these does not refute the target; it declines to instantiate it. Any regime that denies one or more of these conditions while still claiming to evaluate constructions of determinate

same-domain objects has changed the target of inquiry rather than supplied a counterexample within it.

Lemma 3.3 (Target adequacy forces the kernel roles). *Any target-adequate raw-sequence regime already requires the governance roles later named reference, standing, irreversibility, and admissibility.*

Proof. Target determinacy supplies the role later called reference: without a fixed target, there is no object whose construction could be assessed. Step evaluability supplies the role later called standing: without licit or illicit step-status, a transition is only a transition, not a construction step of the same target. Act-time finality supplies the role later called irreversibility: if later normalization can preserve the same act while changing its status, the difference between continuation and repair is not fixed at the time of performance. Finally, same-regime fidelity requires a boundary that holds those classifications together under identity-preserving redescription; that boundary is the role later called admissibility. Thus the kernel terms are introduced as names for forced roles, not as stipulative entry conditions. $\square$

Remark 3.4 (Kernel terms as forced role names). The kernel terms are introduced here only as names for roles whose necessity has already been fixed by the adequacy conditions. The downstream AASC machinery records consequences of this necessity rather than adding independent structure to it.

Lemma 3.5 (Kernel concession lemma). *Any assessment that grants determinate target identity, evaluable stephood, act-time distinction between continuation and repair, and same-regime fidelity has already granted the kernel roles, whether or not it uses the vocabulary of AASC. Conversely, any assessment that denies one of those requirements no longer assesses a theorem-bearing construction of the same determinate endpoint. Therefore there is no third position from which one may reject the kernel while preserving ordinary theorem-bearing endpoint use for the same target.*

Proof. By Lemma 3.3, the four target-adequacy requirements force the roles later named reference, standing, irreversibility, and admissibility. Granting those requirements therefore grants the corresponding governance roles, independently of the terminology used to name them. If an assessment denies one of the four requirements, then by Definition 3.2 it no longer preserves the conditions under which a raw trace is assessable as construction of a determinate same-domain object. Such an assessment changes or exits the target class rather than preserving theorem-bearing endpoint use for the same target. The two cases are exhaustive: the requirements are either granted or denied. $\square$

Definition 3.6 (Act identity). Two performances count as the same act only if they agree on the admissibility-relevant invariant bundle, fix the same target on the same fixed domain, and have the same admissibility status at act-time. Any alteration that changes target, repairs inadmissibility, or reclassifies the admissibility status of the original performance yields a numerically distinct act or a distinct construction sequence.

Definition 3.7 (Governed construction and non-degenerate construction). Let X be a target object on a fixed domain D . A governed construction of X , written $\text{Constr}(X)$, is a raw step-sequence on D that preserves, at each step, determinate reference to X , standing of the step as an admissible construction step, and irreversibility of inadmissible steps. A governed construction is *non-degenerate* iff those three conditions hold together on the same fixed domain. A regime is *degenerate relative to the present target* iff it may still compute, transform, symbolize, or produce outputs but fails to preserve target identity, admissible stephood, act-time finality, or redescription fidelity as conditions of determinate same-domain construction.

This definition is classificatory rather than generative: it does not assume that all raw step-sequences already count as governed constructions, but distinguishes exactly those sequences for which admissibility-assessable construction is well-defined on the fixed domain.

Admissibility is not a fifth ingredient placed beside reference, standing, and irreversibility; it names the governance boundary under which those three conditions are jointly enforceable on the same sequence.

Meaningful regime. A regime is meaningful for the present target iff it is target-adequate in the sense of Definition 3.2 and some of its raw step-sequences are assessable as governed constructions of determinate objects rather than as bare symbol manipulations, encodings, or bookkeeping traces. This is not a stipulative filter. It is the least class in which object identity, admissible stephood, and act-time distinction between continuation and repair are available for assessment at all.

Fixed domain. A fixed domain is the domain on which target identity, identity-preserving transformation, and admissibility-relevant status are all evaluated. Operationally, two constructions remain on the same fixed domain iff there exists a translation between them that preserves (i) object identity under the admissibility-relevant invariant bundle, (ii) the admissibility classification invariant distinguishing admissible continuation from prohibited repair, and (iii) the class of governed constructions up to governance equivalence. A purported continuation remains on the same fixed domain only when these preservation conditions are met.

Why this target class is forced. The target class is not selected because it already contains the conclusion. It is the minimal class in which outputs are evaluable as determinate objects rather than presentation-dependent artifacts. By Lemma 3.3, the roles later named reference, standing, irreversibility, and admissibility are forced by target adequacy before governed construction is defined. If identity-preserving transformation is weakened, object identity becomes redescription-relative; if standing is weakened, there is no licit stephood rather than mere inscription; if irreversibility is weakened, the distinction between admissible continuation and retrospective rescue is unstable at act-time. A regime lacking these conditions may still support transformation, search, or computation, but it does not instantiate a weaker version of the same target phenomenon. The class is therefore fixed by evaluability conditions, not by convenience or stipulation.

Adequacy note. The question is not which extra norms we prefer to impose on an already functioning calculus. The question is what the weakest conditions are under which a raw step-sequence is assessable as construction of a determinate object at all. The next definitions and lemmas argue that stable reference, standing, and irreversibility are forced by that adequacy demand rather than borrowed from a thicker prior theory.

Scope restriction. All universal claims below are restricted to meaningful regimes in this adequacy sense. Systems lacking the adequacy conditions uncovered below are not counterexamples to the present thesis; they are different kinds of formal process because they lack the resources required for governed construction of a determinate same-domain object.

Definition 3.8 (Derivation). When the target object X is a proposition or proof conclusion, the corresponding proposition-targeted special case of governed construction is called a derivation and is written $\text{Der}(X)$.

Definition 3.9 (Governance equivalence). Two governance conditions P and Q are governance-equivalent, written $P \simeq_{\text{gov}} Q$, iff they perform the same governance work for identity-preserving construction on the same fixed domain; that is, replacing P by Q leaves the class of governed constructions unchanged.

Remark 3.10 (Equivalence as identity-level invariance). For the present target, governance-equivalence is not a loose behavioral similarity relation. Once admissibility-assessable, identity-preserving construction is fixed, admissibility classification is bivalent on acts and standing-bearing continuation is fixed by the same admissibility-relevant invariant bundle. Accordingly, any purported alternative governance structure that differs on any act, or on the admissibility-preserving continuation of any act, is not governance-equivalent on the fixed domain. Conversely, where no such divergence exists, the purported alternative contributes no distinct governance content for the target phenomenon.

The companion equivalence and conservation closures sharpen this point further: admissibility and standing do not form two merely coextensive tracks but collapse to the same standing-bearing classification, and conservation is not an auxiliary constraint but the invariant form of that classification. In that sense, equivalence collapses to identity at the level that matters here: any same-domain invariant that agrees with admissibility and standing on all acts and admissibility-preserving continuations is not merely extensionally similar but identical in governance content for admissibility-assessable construction.

Definition 3.11 (Admissibility-relevant invariant bundle). An admissibility-relevant invariant bundle is any family of invariants sufficient to determine when two construction acts on a fixed domain preserve the same target, the same standing-bearing status, and the same admissibility boundary. Two acts or presentations count as belonging to the same fixed domain only when they agree on this invariant bundle up to governance equivalence.

Independent governance work. A condition performs independent governance work on a fixed domain iff its addition changes the set of governed constructions on that domain.

Same fixed domain. References below to “the same meaningful regime” are governed by preservation, not by mere reuse of vocabulary. Two presentations, encodings, or meta-systems count as the same fixed domain for the present target only when translation between them preserves determinate target, standing-bearing step status, the admissibility boundary between admissible continuation and prohibited repair, and the class of governed constructions up to governance equivalence. Failure of any one of these preservation conditions constitutes a change of domain rather than a redescription of the same one.

Definition 3.12 (Derivable from below). A governance condition P is derivable from below iff there exists a governed construction of P whose admissibility can be certified using only governance conditions strictly independent of P , does not already presuppose P or any governance-equivalent condition, and does not rely on any condition whose own governance validity already requires P . In particular, approximation, convergence, limit, or emergent characterization of P counts as derivation from below only if the admissibility of the approximating, convergent, or emergent process itself does not presuppose P or any governance-equivalent condition.

The phrase “governed construction” in this definition is not meant to smuggle in the target conclusion. It marks the minimal condition under which the alleged lower derivation counts as a derivation rather than an unlicensed trace. A purported lower derivation may begin from any formal substrate it likes, but it must eventually explain how its own sequence obtains constructional license without using the role it claims to generate.

Definition 3.13 (Governance generation versus definability). A condition is *defined* in a system when the system can write down, encode, or characterize it. A condition is *generated* in the present sense only when the system can produce that condition as new governance work from a lower basis. Definability without new governance work is not generation from below.

Definition 3.14 (Externally proposed construction or derivation system). Let Σ be any purported construction system, derivation system, meta-theory, logical framework, encoding, or external presentation under which a statement P is said to be derivable. We say that Σ is *kernel-respecting with respect to P* iff the admissibility conditions under which sequences in Σ count as governed constructions already enforce P , some governance-equivalent condition, or some condition whose own admissibility already requires P . This definition classifies the admissibility conditions of Σ ; whether a non-kernel-respecting system can still count as a system of non-degenerate construction for the present target is a theorem proved below, not a stipulation built into the definition.

Definition 3.15 (Kernel of non-degenerate construction). The kernel of non-degenerate construction is the governance-role set

$$K := \{\text{Adm}, \text{St}, \text{Ref}, \text{Irr}\},$$

where *Adm* abbreviates admissibility, *St* standing, *Ref* reference, and *Irr* irreversibility. The minimality and uniqueness claims below concern governance roles and governance work on the fixed domain, not uniqueness of one four-part bookkeeping decomposition.

Remark 3.16. The point of the kernel definition is not to present four independent axioms, nor to shield four merely stipulated starting points from scrutiny. It is to name the lowest invariant cluster that any genuine governed construction already presupposes. Theorems below show that the members of K are neither optional nor independently eliminable.

Why standing is not redundant with admissibility. Admissibility marks gate status: whether an act may enter licensed construction. Standing marks act-carried eligibility for later licensed use, continuation, or commitment. The term “standing” is used because the relevant property is not merely that an act has passed a gate, but that it can stand as a source for subsequent licensed construction. The two coincide extensionally only after fixed-domain closure is proved in Theorem 3.66; they are not introduced as the same role. Before that closure theorem, admissibility answers whether the act is admitted, while standing answers whether the admitted act can function as a standing-bearing source for subsequent construction. The later collapse is therefore a result, not a definition.

Definition 3.17 (Same-regime faithful counterexample). A same-regime faithful counterexample to the kernel is a target-adequate raw-sequence regime together with a class of candidate constructions such that all of the following are preserved on the same fixed domain: determinate target identity, step evaluability, act-time finality, identity-preserving redescription, and the class of standing-bearing continuations. The counterexample must nevertheless omit at least one member of K , and not merely replace it by a governance-equivalent role. Any example that loses one of the preservation conditions is a different formal process; any example that restores the missing work by an auxiliary selector, bridge, ranking, tolerance, or repair channel imports fresh governance rather than omitting the role.

Proposition 3.18 (Burden of a successful objection). *To refute the kernel result for the present target, one must do at least one of two things: derive some $P \in K$ from below in the sense of Definition 3.12, or exhibit a same-regime faithful counterexample in the sense of Definition 3.17. Pointing to a different formal practice, a reversible search regime, a bookkeeping encoding, or a system with weaker target identity does not meet this burden.*

Proof. The target is fixed by target adequacy and same-domain fidelity. If an objection claims that a kernel role is generated rather than presupposed, then it must satisfy the strict lower-generation condition; otherwise the alleged derivation already uses the role it claims to produce. If the objection

instead claims that the role is unnecessary, then it must preserve the same target phenomenon while omitting that role; otherwise it has changed the object of evaluation. Within the present target, a substantive challenge therefore alleges either lower generation or same-regime dispensability. The former is governed by derivability from below; the latter is governed by same-regime faithful counterexample. Anything else changes subject, imports structure, or supplies only notation. $\square$

This burden is not an artificial raising of the evidential standard. It follows from target preservation. A counterexample to a necessity claim about determinate same-domain construction must preserve determinate same-domain construction. A regime that abandons target identity, step evaluability, act-time classification, or redescription fidelity is not a counterexample to the necessity claim; it is a different target.

Lemma 3.19 (Raw sequences are not yet constructions). *A raw step-sequence does not count as a construction of a determinate object merely in virtue of being a sequence.*

Proof. A raw sequence may be written, executed, or transformed while leaving open what object, if any, it fixes; whether its steps are admissible steps of the same act; and whether later repair preserves or changes the act being assessed. Those are exactly the governance matters at issue. So raw sequentiality alone yields, at most, a trace. It does not yet yield construction of a determinate same-domain object. $\square$

Lemma 3.20 (Same-act repair is impossible). *If a later intervention changes an inadmissible performance into an admissible one, then the later intervention does not preserve the same act.*

Proof. By Definition 3.6, act identity requires preservation of the admissibility-relevant invariant bundle together with preservation of act-time admissibility status. A later intervention that converts inadmissibility into admissibility changes that status and therefore fails to preserve act identity. What results is either a distinct act or a distinct construction sequence, not repair of the same act. $\square$

Lemma 3.21 (Reference is necessary for determinate construction output). *If a purported construction act lacks determinate reference, then it does not yield a determinate object of the regime.*

Proof. A construction must be about or directed toward a target in a fixed way in order to preserve identity through its steps. If the act's expressions refer equivocally or vacuously, then no single governed target is fixed: different readings, or none at all, remain equally compatible with the same mark sequence. What remains is a trace, not a determinate object. Hence determinate construction output requires determinate reference. $\square$

Lemma 3.22 (Standing is necessary for admissible stephood). *If a purported step lacks standing, it does not function as an admissible construction step, but only as an unsupported inscription or transition.*

Proof. A construction step is not merely something that occurs between two inscriptions. It purports to advance a target under some licensed governance status. Without standing, there is no governed construction act whose role could be assessed. The item may still be written, computed, or recorded, but it does not count as an admissible step. Therefore standing is necessary for constructional stephood. $\square$

Lemma 3.23 (Irreversibility is necessary for stable construction status). *If invalid steps are repairable post hoc without loss, then the distinction between admissible continuation and retrospective rescue is not stably fixed at act-time.*

Proof. Suppose invalidity could later be removed while preserving the same act as fully admissible. By Lemma 3.20, that preservation claim is false: any intervention that changes inadmissibility into admissibility changes the act identity of what is being assessed. So if repair is allowed, the later admissible outcome is not the same act but a distinct act or distinct sequence. The original act therefore remains inadmissible, and any regime that treats later repair as preserving the same act loses a stable act-time distinction between continuation and rescue. Stable constructional status consequently requires irreversibility. $\square$

Lemma 3.24 (Admissibility is the governance boundary of joint enforceability). *Whenever determinate reference, standing, and irreversibility are jointly enforceable on the same construction acts, admissibility is already in force there as the governance boundary of that joint enforceability.*

Proof. By Lemmas 3.21–3.23, a regime that treats its acts as determinate, admissibility-assessable construction acts must already maintain determinate reference, licit step-status, and stable non-repairability of invalid steps. The point is not that one first has three free-standing ingredients and then adds admissibility as an extra fifth clause. Rather, once those three conditions are jointly enforceable on the same acts as conditions of construction, the regime already has a governance boundary distinguishing admissible from non-admissible acts and distinguishing continuation from retrospective rescue. That governance-boundary status is exactly what is here called admissibility. Conversely, a regime lacking that joint governance boundary does not govern construction as non-degenerate construction in the present sense. Hence admissibility is already in force wherever the three conditions are jointly enforceable. $\square$

Proposition 3.25 (Boundary closure is forced by admissibility). *Let a regime support admissibility-assessable, identity-preserving construction with irreversibility. Then any admissibility-relevant discriminator, selector, or classification rule must be invariant under identity-preserving transformation on the fixed domain.*

Consequently, any attempt to introduce additional admissibility-relevant structure not preserved under such transformations either:

- (i) changes admissibility classification and so introduces a competing gate, or*
- (ii) leaves admissibility classification unchanged and is therefore inert bookkeeping.*

Hence the admissibility classification invariant is closed under same-domain construction: no additional admissibility-relevant discriminator can be introduced without either collapsing to governance-equivalence or changing the domain of evaluation.

Proof. If an admissibility-relevant discriminator is not invariant under identity-preserving transformation, then two presentations of what purports to be the same target on the same fixed domain receive different admissibility significance. If that difference changes what counts as admissible continuation, standing-bearing reuse, or prohibited repair, then the regime has introduced a second admissibility classifier for the same fixed-domain acts. But the distinction between admissible continuation and prohibited repair is exactly the governance work fixed by admissibility together with irreversibility; reclassifying same-domain acts therefore yields a competing gate rather than a mere refinement. If, by contrast, the additional discriminator does not change any admissibility verdict, then it contributes no independent governance work and is only bookkeeping. There is no third possibility: a same-domain addition either changes admissibility significance or it does not. Accordingly, admissibility-assessable, identity-preserving construction with irreversibility already forces closure of the admissibility classification invariant against further same-domain discriminators. $\square$

Theorem 3.26 (Minimal assessability boundary). *Let R be any system whose outputs are treated as determinate objects and whose transitions are evaluated as construction steps. Then determinate reference, admissible stephood, and non-repairable invalidity are jointly necessary and minimal for that evaluability. More exactly:*

- (i) *if determinate reference is removed, object identity is lost and the outputs are no longer evaluable as determinate content;*
- (ii) *if standing is removed, no transition qualifies as an admissible construction step and the outputs are no longer evaluable as construction steps;*
- (iii) *if irreversibility is removed, there is no stable distinction between admissible continuation and retrospective rescue, and the outputs are no longer evaluable as constructions; and*
- (iv) *if an additional condition is imposed as an independent governance requirement beyond those three, then the regime is governed more tightly than evaluability itself requires.*

Accordingly, these three conditions mark the minimal assessability boundary for identity-preserving construction as such. The minimality claim is a claim about governance work on the fixed domain, not about uniqueness of one bookkeeping decomposition.

Proof. Items (i)–(iii) are exactly the failure modes identified in Lemmas 3.21–3.23. Without determinate reference there is no fixed object identity and therefore no evaluable content; without standing there is no licensed act and therefore no evaluable construction step; without irreversibility there is no stable admissibility distinction and therefore no evaluable construction rather than retrospective normalization. The point is not merely that these failures are bad outcomes. It is that, once any one of them occurs, the target phenomenon itself is no longer present: what remains is trace-production, transformation, or bookkeeping rather than governed construction of a determinate same-domain object. This also blocks nearby weakenings and substitutes. Any purported weaker replacement for one of the three conditions must either preserve exactly the same governance work, in which case it is merely a notational variant of that condition; or else it must relax some part of that work, in which case one of the same excluded failures reappears—presentation-dependent object identity, indeterminate target, or collapse of the distinction between continuation and repair. If a purported weakening avoids those failures only by adding a new admissibility-relevant ranking, selector, tolerance, or repair-sensitive middle status, then it no longer weakens the package but supplements it by fresh governance structure. For minimality, suppose an additional condition Q is imposed as an independent governance requirement on every admissibility-assessable construction act. Then either Q is already enforced by the joint assessability work of reference, standing, and irreversibility, in which case it is not additional; or else Q excludes some act-sequences that already satisfy those three conditions, in which case Q governs more tightly than evaluability itself requires. There is no third possibility. So the three conditions are jointly necessary and minimal for that evaluability. $\square$

Corollary 3.27 (Non-arbitrariness of the target class). *The class of meaningful regimes is not defined by pre-selecting the kernel. It is exactly the class of systems in which determinate, admissibility-assessable construction is possible for the present target. Restriction to that class is therefore not protective but forced by the target of inquiry.*

Proof. By Theorem 3.26, determinate reference, standing, and irreversibility are not optional entry criteria appended in advance of the argument. They are the minimal conditions under which outputs and transitions are evaluable as construction at all. So to restrict attention to regimes in which those conditions are assessable is simply to restrict attention to the subject matter of the paper rather than to protect the thesis from counterexample by fiat. $\square$

Theorem 3.28 (The target class is forced, not stipulated). *Let R be any regime of raw step-sequences. If R lacks determinate reference, standing, or irreversibility in the senses fixed above, then R does not support governed construction of determinate same-domain objects. It supports only raw transformation, encoding, or bookkeeping processes.*

Proof. If determinate reference is lacking, Lemma 3.21 shows that no determinate target object is fixed. If standing is lacking, Lemma 3.22 shows that no step counts as an admissible construction step. If irreversibility is lacking, Lemma 3.23 together with Lemma 3.20 shows that act-time distinction between continuation and repair collapses. By Lemma 3.19, raw sequentiality does not by itself restore those losses. So a regime lacking any one of these conditions may still execute sequences, but it does not instantiate governed construction of determinate same-domain objects. $\square$

Lemma 3.29 (Exhaustion of failure modes for non-kernel regimes). *Let R be a regime of raw step-sequences. Suppose R lacks at least one member of K while claiming to support determinate, identity-preserving construction on a fixed domain. Then exactly one of the following holds:*

- (i) *object identity is not fixed, because determinate reference fails;*
- (ii) *step status is not evaluable as admissible or inadmissible, because standing fails;*
- (iii) *the distinction between continuation and repair is not stable at act-time, because irreversibility fails; or*
- (iv) *the regime restores one or more of these distinctions only by adding additional governance-bearing structure not fixed on the original domain, and so proceeds by illicit import rather than same-domain construction.*

These cases cover the preservation failures relevant to the present target.

Proof. If R lacks reference, then by Lemma 3.21 no determinate target is fixed, giving (i). If R lacks standing, then by Lemma 3.22 no step counts as an admissible construction step, giving (ii). If R lacks irreversibility, then by Lemmas 3.20 and 3.23 the distinction between admissible continuation and retrospective rescue is unstable at act-time, giving (iii). The only remaining possibility is that R attempts to preserve determinate identity, admissible stephood, or stable act-status while omitting one of the kernel roles. But then those distinctions are being reintroduced by fresh governance-bearing structure not fixed on the original domain. That is illicit import rather than a same-domain realization below the kernel, giving (iv). Within the present target, every purported non-kernel regime either loses one of the three assessability goods directly or restores it by extra same-domain authority. $\square$

Lemma 3.30 (Reduction to kernel failure modes). *Let R be any regime that deviates from the kernel while claiming to preserve determinate, identity-preserving construction on a fixed domain. Then every such deviation induces at least one of the following:*

- (i) *non-uniqueness of target under identity-preserving transformation;*
- (ii) *indeterminacy of admissible stephood;*
- (iii) *instability of admissibility under continuation; or*
- (iv) *introduction of a selector, ranking, or auxiliary structure not invariant under identity-preserving transformation.*

Hence every deviation reduces to a kernel failure mode.

Proof. If a deviation changes which target is fixed under identity-preserving transformation, then same-target identity is not preserved, giving (i). If it leaves target fixed but no longer preserves evaluable admissible stephood, then licensed step status becomes indeterminate, giving (ii). If it

preserves target and stephood while allowing admissibility status to vary under purported same-act continuation, then the act-time distinction between continuation and repair is unstable, giving (iii). The only remaining possibility is that the regime preserves these goods by introducing some selector, ranking, tolerance class, or other auxiliary governance-bearing discriminator not already fixed by the original admissibility-relevant invariant bundle. But such a discriminator is fresh same-domain authority rather than preservation of the original regime, giving (iv). Since every deviation changes target, step-status, continuation-status, or the authority by which these are fixed, the partition covers the relevant ways a same-domain deviation can proceed. Therefore every deviation reduces to a kernel failure mode. $\square$

Theorem 3.31 (Non-degenerate construction forces the kernel). *If a meaningful regime admits non-degenerate construction in the sense of Definition 3.7, then the full kernel K is already in force in that regime. Equivalently, no meaningful regime supports non-degenerate construction in that sense while lacking admissibility, standing, reference, irreversibility, or some governance-equivalent replacement for one of them.*

Proof. Let R be a meaningful regime admitting non-degenerate construction in the sense of Definition 3.7. By Theorem 3.26, any regime in which outputs are assessable as determinate, admissibility-assessable constructions already requires determinate reference, admissible stephood, and irreversibility as the minimal assessability boundary of that evaluability. By Lemma 3.24, the joint enforceability of those conditions already places admissibility in force as the governance boundary constituted by their coherence on the same acts. Hence R already enforces Ref, St, Irr, and Adm, that is, the full kernel K . $\square$

Corollary 3.32 (No non-degenerate regime below the kernel). *No meaningful regime can both support non-degenerate construction in the sense of Definition 3.7 and generate any member of K from a strictly lower governance-free basis while remaining within that same fixed domain.*

Proof. Assume some meaningful regime R supports non-degenerate construction in the sense of Definition 3.7 and nevertheless generates some $P \in K$ from a strictly lower governance-free basis. By Theorem 3.31, the full kernel K is already in force in R before any such generation begins. So the alleged generator does not produce P from below; it operates only within a regime whose meaningfulness already depends on P . That contradicts the claim that P was first generated from a lower basis. $\square$

Remark 3.33. The point of Theorem 3.31 is not to build the package into construction by stipulation. It is to show that, for the target domain fixed in the fixed-domain construction subsection, any regime that already supports non-degenerate construction already instantiates the kernel by necessity. The derivation lemmas of Section 3 therefore record a local reflection of a prior construction-level fact.

3.2 Presupposition Structure of Derivation

With Theorem 3.31 in place, derivation is the proposition-targeted special case of licensed construction. The present section records the derivation-level presupposition facts. These lemmas do not generate the package. They state what the fixed-domain construction subsection result looks like once the focus narrows from non-degenerate construction in general to derivation in particular: a sequence counts here as a derivation only if it already operates with admissibility, standing, reference, and irreversibility in place.

Lemma 3.34 (Derivation presupposes admissibility). *If $\text{Der}(P)$ holds for any proposition P , then admissibility is already in force.*

Proof. By Definition 3.8, a governed derivation is not merely a formal trace but a governed construction act whose target is a proposition. Its steps therefore count as steps only under an admissibility regime. So if $\text{Der}(P)$ exists, admissibility has already been presupposed. $\square$

Lemma 3.35 (Derivation presupposes standing). *If $\text{Der}(P)$ holds, then standing is already in force.*

Proof. A derivation with no standing would be a sequence of acts with no admissible target-bearing status. Such a sequence might be written down, but it would not count as a derivation of anything. Licensed derivation therefore presupposes standing. $\square$

Lemma 3.36 (Derivation presupposes reference). *If $\text{Der}(P)$ holds, then determinate reference is already in force.*

Proof. A derivation whose expressions fail to refer determinately does not derive a determinate conclusion. It supplies marks, not governed content. Since $\text{Der}(P)$ names a derivation of a proposition rather than a mere syntactic trail, reference is presupposed. $\square$

Lemma 3.37 (Derivation presupposes irreversibility). *If $\text{Der}(P)$ holds, then irreversibility is already in force.*

Proof. Without irreversibility, invalid steps could be repaired retroactively into acceptable ones, and the distinction between admissible continuation and post hoc rescue would collapse. In that case the derivation would not possess a stable governance status at all. Hence governed derivation presupposes irreversibility. $\square$

Corollary 3.38 (Global presupposition lemma). *For every proposition P , if $\text{Der}(P)$ holds, then the full kernel K is already in force.*

Proof. Combine the four previous lemmas. $\square$

Theorem 3.39 (Definability is not governance generation). *Let P be any governance condition. A system may define, encode, or represent P without generating P from below. Generation from below occurs only if the system contributes new governance work sufficient to produce P for the original acts on the original fixed domain.*

Proof. A definition or encoding may restate a governance condition, give it a symbol, embed it in a meta-language, or characterize its extension. None of those activities by itself changes the class of governed constructions on the original fixed domain. By definition, governance generation requires independent governance work. So definability, representation, and coding are not by themselves generation from below. $\square$

Remark 3.40. Accordingly, the non-derivability claims below are not claims that one cannot write down or characterize admissibility, standing, reference, or irreversibility. They are claims that no lower same-domain basis generates the governance work those conditions perform for the original acts.

3.3 Non-Derivability of the Kernel

The results of this section have two directions. Internally, they show that no lower same-domain basis generates the governance work performed by the kernel. Externally, they identify the construction-level form of a broader meta-necessity: in any non-degenerate irreversible compositional regime, the invariant required to make irreversibility well-defined is exactly the invariant characterized here under the fixed-domain conditions of identity-preserving construction. The kernel is therefore not an optional refinement layered on top of admissibility. It is the minimal decomposition of the admissibility invariant once the target object is fixed.

The argumentative standard in this section is deliberately adversarial. A proposed defeat of the kernel must either supply a faithful lower generator for one of its roles, or supply a same-regime faithful counterexample that preserves the target phenomenon while lacking that role. The former fails because any licensed construction of the role already relies on the role or a governance-equivalent substitute. The latter fails because any same-regime omission either loses target identity, loses licit stephood, loses act-time irreversibility, imports a replacement selector, or collapses extensionally back to the same governance work.

Theorem 3.41 (Non-derivability of admissibility). *Admissibility is not derivable from below.*

Proof. Assume for contradiction that admissibility is derivable from below. Then there exists a governed construction of *Adm* whose admissibility does not already presuppose *Adm* or a governance-equivalent condition. But by the previous section, any licensed construction already presupposes admissibility. So the alleged construction uses what it claims to produce. This is precisely the forbidden circularity in the definition of derivability from below. Therefore admissibility is not derivable from below. $\square$

Theorem 3.42 (Non-derivability of standing). *Standing is not derivable from below.*

Proof. Suppose standing were derivable from below. Then there would exist a governed construction of *St* whose admissibility did not already presuppose standing. By the previous section, however, the very existence of such a construction already presupposes standing. The construction therefore fails the “from below” condition by collapsing into the very status it was supposed to generate. $\square$

Theorem 3.43 (Non-derivability of reference). *Reference is not derivable from below.*

Proof. If reference were derivable from below, there would be a governed construction of *Ref* whose admissibility did not already rely on determinate reference. But by the previous section, no licensed construction exists without determinate reference. Hence the putative construction again presupposes what it claims to derive. Reference is therefore not derivable from below. $\square$

Theorem 3.44 (Non-derivability of irreversibility). *Irreversibility is not derivable from below.*

Proof. Assume irreversibility is derivable from below. Then a governed construction of *Irr* exists whose admissibility does not already presuppose irreversibility. The previous section rules this out: a licensed construction already relies on the stable difference between admissible continuation and invalid repair. So irreversibility cannot be derived from beneath itself. $\square$

Corollary 3.45 (Kernel non-derivability). *No member of K is derivable from below.*

Proof. Immediate from the four preceding theorems. $\square$

Theorem 3.46 (No faithful lower generator / structural non-derivability). *Let $P \in K$. No system supporting non-degenerate construction can generate P from a genuinely lower condition while remaining within the same fixed domain. Equivalently, any such system already has P in force before internal construction begins.*

Proof. Assume for contradiction that some system or meaningful regime R supports non-degenerate construction in the sense of Definition 3.7 and nevertheless generates some $P \in K$ from a purportedly lower package. There are only two cases.

First, the alleged generator operates on the same fixed domain as the original acts. Then by Theorem 3.31, the full package K is already in force in R , and hence P is already in force there. So the alleged lower package does not generate P from below; it operates only inside a regime whose meaningfulness already depends on P . In this same-domain case the remaining escape routes are exhausted. A purported lower generator must either (i) introduce additional same-domain structure not already licensed on the fixed domain, (ii) collapse to triviality by adding no independent governance work, (iii) alter identity-preserving transformation, reference conditions, or act-identity conditions so that the original target phenomenon is no longer preserved, or (iv) reproduce the original admissibility boundary extensionally under a different presentation. Route (i) is illicit same-domain import rather than lower generation. Route (ii) is not generation at all. Route (iii) changes domain rather than generating the kernel within the same one. Route (iv) is extensional redescription of what was already in force. Any further same-domain route would have to do independent governance work while avoiding all four branches, which is precisely what the preceding results exclude.

Second, the alleged generator operates outside the original fixed domain and then attempts to re-enter it. But then the resulting construction does not govern the original acts on their original fixed domain without an additional transfer principle, selector, or authority connecting the external regime back to the old one. That additional bridge is exactly fresh structure not fixed by the original domain. More sharply: the admissibility boundary together with standing classification fixes the identity of the domain under evaluation. So any purported derivation that changes that boundary or that classification has not generated the kernel for the original domain; it has changed the object of evaluation and is therefore a scope change rather than a lower derivation. The maneuver is consequently scope change or illicit import rather than lower generation of the kernel for the original acts.

Thus neither same-domain nor cross-domain generation succeeds. Re-describing the alleged generator as belonging to a regime that lacks P therefore does not help, because either it fails to govern the original domain or, if it does govern it, P was already in force there. The obstruction is structural rather than a peculiarity of one proof format. $\square$

Theorem 3.47 (Regime-invariant non-derivability). *Let $P \in K$. No system Σ can derive P from below unless the admissibility conditions under which a Σ -construction counts as a construction already enforce P , something governance-equivalent to P , or a condition that already requires P .*

Proof. Let $P \in K$, and suppose some system Σ is offered as a construction or derivation system for P . There are two cases.

First, the admissibility conditions under which a Σ -construction counts as a licensed construction already enforce P , a governance-equivalent condition, or a condition whose own admissibility already requires P . Then Σ is kernel-respecting with respect to P by Definition 3.14, and the alleged construction does not generate P from below.

Second, those conditions do not already enforce P in any of those ways. If Σ nevertheless supported non-degenerate construction in the same fixed domain, Theorem 3.31 would force the

whole package, including P , already to be in force there. So a non-kernel-respecting Σ cannot both lack P and still count as a system of non-degenerate construction for the present target. Its output is therefore, at best, an external coding, a bookkeeping device, or a purely formal trace; it is not a construction of P from below for the target phenomenon at issue.

Changing formal dress, moving to a meta-theory, or recoding the language therefore does not remove the need for the work that P names. By Theorem 3.39, such recodings may define or represent P without generating it from below. $\square$

Theorem 3.48 (No faithful lower generator and no faithful same-domain extension). *For the admissibility boundary on a fixed domain, there is no faithful lower generator and no faithful same-domain extension that converts admissibility, standing, reference, or irreversibility into lower theorems for the original acts on the original fixed domain. Any purported attempt ends in exactly one of four outcomes: illicit structure import, triviality, violation of the base goods, or extensional collapse back to the gate itself.*

Proof. Any genuinely pre-boundary constraining device must be available and meaningful on the fixed pre-admissible side. If it is not, it is illicit import. If it is, then under the same-domain invariance conditions relevant here it either has no independent constraining force on individual acts, destroys reference, identity, or irreversibility, or coincides extensionally with the gate already being determined. These four branches cover the ways a purported pre-boundary device could relate to the original acts. A further case would have to contribute independent same-domain governance work while neither importing structure, trivializing the construction, violating the base goods, nor collapsing extensionally to the gate. But independent same-domain governance work is exactly what the non-derivability and no-faithful-same-domain-extension results exclude. The same exhaustion therefore remains in force under faithful same-domain extension, because fresh same-domain authority for the original acts is itself illicit structure import unless it changes nothing or collapses back to the original gate. $\square$

Proposition 3.49 (Cross-domain closure and admissibility transport). *The non-derivability results above exclude not only same-domain lower generators but also cross-domain simulation attempts. In particular, let Σ be any external system and let ϕ be a mapping from Σ into the fixed domain of the original acts. If ϕ preserves (i) target identity under identity-preserving transformation, (ii) admissibility classification, and (iii) standing-bearing continuation, then ϕ is governance-equivalent to the original domain on the relevant image. If ϕ fails to preserve any of (i)–(iii), then the attempted transport changes the admissibility-relevant invariant bundle and constitutes a scope change rather than a derivation. Accordingly, cross-domain simulation does not provide a lower generator for admissibility on the fixed domain: it either collapses to governance-equivalence or exits the domain of evaluation.*

Proof. The fixed domain is identified by preservation of target identity, admissibility classification, and the class of governed constructions up to governance equivalence (Definition 3.7 and the fixed-domain clause of the fixed-domain construction subsection). By Proposition 3.25, admissibility-assessable, identity-preserving construction with irreversibility already forces closure of the admissibility classification invariant against further same-domain discriminators. So if a mapping ϕ from an external system preserves target identity, admissibility classification, and standing-bearing continuation for the original acts, then it preserves exactly the invariant bundle that individuates the fixed domain for the present target. On the relevant image, ϕ therefore contributes no independent governance work beyond what is already counted as the same domain; it is governance-equivalent rather than a lower generator.

If, by contrast, ϕ fails to preserve any one of those items, then the transported construction no longer governs the original acts as the same fixed-domain objects with the same admissibility classification and standing-bearing continuation. That is not derivation of admissibility for the original acts, but change of the object under evaluation. Hence cross-domain transport either collapses to governance-equivalence or exits the domain of evaluation. $\square$

Remark 3.50. The previous results do not show that one may never write down statements asserting admissibility, standing, reference, or irreversibility. They show something sharper: these conditions cannot be generated as foundations by a construction that does not already rely on them. Their non-derivability is thus a minimality result, not an expressive prohibition.

Theorem 3.51 (No governance-effective deeper same-domain invariant). *Let a system support admissibility-assessable, identity-preserving construction with irreversibility. Then no governance-effective invariant strictly beneath admissibility exists for that same fixed-domain target.*

More exactly, any purported invariant that claims to ground, constrain, or underwrite admissibility must fall into exactly one of the following cases:

- (i) *it imports structure not invariant under identity-preserving transformation and therefore changes domain rather than grounding admissibility on the fixed domain;*
- (ii) *it carries no constraining power on admissibility-assessable construction and is therefore vacuous with respect to the present target; or*
- (iii) *it is governance-equivalent to admissibility or standing under renaming and therefore performs no deeper governance work.*

Accordingly, no governance-effective deeper same-domain invariant remains compatible with irreversible, identity-preserving construction on the fixed domain.

Proof. Any purported deeper invariant must, by hypothesis, do one of two things: either it constrains admissibility, or it fails to constrain it. If it fails to constrain admissibility, then it does no governance work for the target phenomenon and is vacuous relative to identity-preserving construction. So only the constraining case remains.

Now any invariant that constrains admissibility must be available wherever admissibility is being fixed for the same acts on the same domain. If it is introduced only by adding structure not preserved under identity-preserving transformation, then it does not underwrite admissibility on the fixed domain but changes the admissibility-relevant invariant bundle and thus changes domain. If instead it is presented as same-domain and same-target while avoiding such imported structure, then by the non-derivability and no-faithful-same-domain-extension results above, it cannot contribute independent deeper governance work beneath or prior to admissibility. In that case either it leaves the admissibility classification unchanged, in which event it is inert bookkeeping, or it reproduces the same governance work already performed by admissibility or standing under a different presentation, in which event it is governance-equivalent under renaming.

A further option would have to constrain admissibility on the same fixed domain while neither importing structure, nor being vacuous, nor collapsing to governance-equivalent admissibility work. But that is exactly the kind of independent deeper same-domain invariant excluded by the preceding non-derivability results together with Proposition 3.49 and the fixed-domain-extension result. Therefore no governance-effective invariant strictly beneath admissibility exists for the present fixed-domain target. $\square$

Remark 3.52 (Dependency status of invariant impossibility). Here and throughout, “deeper invariant” means any invariant that purports to ground or constrain admissibility without being reducible to it.

The preceding theorem is eliminative, not generative. It does not serve as a premise for the existence or necessity of admissibility. Rather, it constrains the space of possible alternatives once admissibility is already required by irreversibility and identity-preserving construction and once the fixed-domain kernel has been established in the kernel forcing and non-derivability subsections.

In particular, the impossibility of deeper invariants is not used to derive admissibility itself, but only to exclude competing invariant structures that might otherwise be proposed in its place. Any dependence on architectural commitments such as non-transmissive closure or boundary discipline is therefore downstream: within the argumentative order of the present paper, those commitments are consequences of the admissibility structure already established in the kernel forcing and non-derivability subsections or eliminative restatements of what those sections exclude, not independent assumptions on which the kernel results rely.

3.4 Mutual Closure and Minimality

Proposition 3.53 (Admissibility requires standing). *Any regime lacking standing is not an admissibility regime.*

Proof. A regime without standing does not distinguish licit construction from unsupported utterance or transition. It therefore lacks the governance force required to count as admissibility. At best it describes transitions; it does not govern construction. $\square$

Proposition 3.54 (Standing requires reference). *Standing without determinate reference is incoherent.*

Proof. Standing is the licitness of a construction act. But where reference is indeterminate, there is no determinate target whose licitness could be assessed. A purported standing relation with no reference relation underneath it licenses emptiness or equivocation, not governed construction. $\square$

Proposition 3.55 (Governed reference requires irreversibility). *The reference relevant to non-degenerate construction is not bare denotation but reference fixed for governed use at act-time. That form of reference cannot be maintained if invalid construction acts are repairable post hoc without loss.*

Proof. A symbol may still bear a denotational relation under later reinterpretation or repair. That is not yet the kernel notion of reference. The issue here is whether the act, as performed, fixes a determinate target for governed constructive use. If later repair can retroactively convert the same invalid act into a fully admissible one without loss, then the constructive status under which the act's target is fixed is not settled at act-time. What remains may be denotation, but not governed reference in the sense required for non-degenerate construction. Hence kernel-reference requires irreversibility of invalidity. $\square$

Proposition 3.56 (Governance irreversibility requires admissibility). *The irreversibility relevant here is not mere temporal one-wayness or procedural non-rewindability. It is the governance distinction between licensed continuation and prohibited repair, and that distinction is available only under an admissibility boundary.*

Proof. One may define other senses of irreversibility—causal, temporal, computational, or procedural—without invoking admissibility. Those are not the senses at issue here. The kernel member Irr concerns irreversible invalidity of construction acts. Without admissibility there is no principled governance boundary under which one continuation counts as licensed while another counts as illicit repair. One can still speak of sequence or alteration, but not of irreversible invalidity in the governance sense. Thus governance irreversibility presupposes admissibility. $\square$

Theorem 3.57 (Mutual closure of the kernel). *The members of K form a mutually closed necessity set: none can be coherently asserted while denying the others' governance role.*

Proof. Propositions 3.53–3.56 exhibit a governance-specific necessity loop: admissibility requires standing, standing requires reference, governed reference requires irreversibility, and governance irreversibility requires admissibility. Hence no member can be detached from the rest while retaining its intended governance content. What remains after detachment is either vacuous, equivocal, or degenerate. $\square$

Remark 3.58. The last two arrows of the loop concern the governance senses fixed in the fixed-domain construction subsection. Proposition 3.55 is not a claim about bare denotation independently of construction; it concerns reference as fixed for governed constructive use at act-time. Proposition 3.56 is not a claim about every temporal or procedural notion of irreversibility; it concerns irreversible invalidity of construction acts. In those senses, the loop is closed.

Theorem 3.59 (Minimality of the kernel). *The kernel K is minimal for non-degenerate construction: no proper subset of K suffices for non-degenerate construction.*

Proof. Assume a proper subset $K_0 \subsetneq K$ suffices for non-degenerate construction. Let $X \in K \setminus K_0$ be an omitted member. Since non-degenerate construction allegedly remains possible, either X is unnecessary or X is generated from the remaining members. The first option contradicts Theorem 3.57, since the governance role of each member depends on the rest. The second option contradicts Corollary 3.45, since X would then be derivable from below from a lower basis. Both possibilities fail. Therefore no proper subset suffices. $\square$

Corollary 3.60 (No lower generator). *There exists no governance condition or package strictly below K that generates K .*

Proof. If such a lower generator existed, at least one member of K would be produced from below by a package not already equivalent to that member. That contradicts Corollary 3.45. $\square$

Theorem 3.61 (Uniqueness of governance work, not of bookkeeping decomposition). *If K' is any governance package sufficient for non-degenerate construction in the sense of Definition 3.7, then K' must cover the same governance roles as K and is governance-equivalent to K at that role level. Coarser or finer decompositions may exist; what cannot exist is a governance-non-equivalent alternative package, where governance-non-equivalent means inducing a different class of governed constructions.*

Proof. Any package sufficient for non-degenerate construction must, by Theorem 3.31, already secure admissibility, standing, reference, and irreversibility, or governance-equivalent replacements for those roles. If it fails to supply any one of those governance roles, non-degenerate construction collapses by Theorem 3.57. A coarser package may compress multiple roles into one clause, and a finer package may split one role across several clauses. But if the total governance work and the resulting class of governed constructions remain unchanged, then by Definition 3.9 the package is governance-equivalent to K at the level that matters here. If the governance work changes, the package is governance-non-equivalent and is not sufficient for non-degenerate construction in the present sense. Therefore the uniqueness claim concerns required governance work, not the uniqueness of one four-part bookkeeping decomposition. $\square$

Remark 3.62. This theorem concerns governance-role equivalence, not representational identity and not uniqueness of a particular factorization. It does not exclude alternative formal encodings,

type-theoretic or categorical implementations, game-semantic presentations, or coarser/finer decompositions that preserve the same governance work. It excludes only packages that alter the class of governed constructions and therefore perform different governance work.

3.5 Fixed-Domain Closure Results

In the present section, “same domain” always means preservation of the admissibility-relevant invariant bundle and hence preservation of the class of governed constructions up to governance equivalence. The point is not to classify arbitrary statuses or metrics, but to describe what fixed-domain admissibility can and cannot look like once the kernel is in place.

The bivalence and AMetricity claims below concern only gate status for standing-bearing commitment, licensed reuse, and admissible continuation. They do not deny object-language many-valued semantics, paraconsistent valuations, probabilistic confidence, proof-search rankings, approximation orders, proof-state metadata, type refinements, or graded measures internal to an already admitted construction regime. Such structures are admissible only when they do not themselves decide standing-bearing entry. If they do decide entry, they are part of the admissibility gate and fall under the present results; if they do not, they are downstream bookkeeping or internal structure.

These results do more than describe one convenient interface. Taken together, they close the remaining underdetermination question for the admissibility invariant itself. Once a fixed domain already requires an admissibility boundary, every same-domain realization must either reproduce the interface proved in this section up to governance-equivalence or else fall into one of the failure modes already excluded: hidden selector work, repair-sensitive middle status, illicit structure import, domain change, or inert bookkeeping. The point of the section is therefore structural rigidity: necessity plus uniqueness of same-domain realization.

Lemma 3.63 (No intermediate admissibility state). *Let S be any admissibility-relevant status distinct from admitted and not-admitted on a fixed domain. Then exactly one of the following holds:*

- (i) *S affects admissible continuation, commitment, or licensed reuse, in which case S performs hidden selector work or reopens a repair-sensitive middle status; or*
- (ii) *S affects none of those, in which case S is inert bookkeeping rather than an admissibility-relevant status.*

Hence no third admissibility-relevant status exists.

Proof. Take any status S distinct from admitted and not-admitted. If possession of S changes what may later count as licensed continuation, commitment, or reuse, then S adds governance-relevant discrimination beyond the gate itself. In that case it either selects among otherwise eligible acts or preserves a repair-sensitive middle state between rejection and admission. If instead possession of S changes nothing about later licensed use, then S contributes no governance work and is merely descriptive bookkeeping. Since every admissibility-relevant status must either affect licensed use or fail to affect it, these two cases are exhaustive. Therefore no third admissibility-relevant status survives. $\square$

Theorem 3.64 (Necessity and bivalence of admissibility). *On every fixed domain supporting licensed construction, a prior admissibility predicate is forced, and any admissibility-relevant status interface on that same fixed domain is extensionally equivalent to a binary one. Equivalently, every admissibility-relevant status falls into exactly one of two classes: admitted for commitment, reuse, and standing-bearing continuation, or not admitted.*

Proof. Without a prior admissibility predicate, entry into licensed composition would have to occur before evaluation, by post hoc repair, by scope transport, or by injected parameterization. Each route breaks reference, composition, or irreversibility. So a prior gate is forced. Now suppose the resulting admissibility interface were not extensionally binary. Then some additional same-domain status would have to survive between, or alongside, admitted and not admitted. If that status changes later admissible use, it is hidden gate work or a repair-sensitive middle state and therefore contradicts the fail-closed consequences of irreversibility. Any state whose admissibility status can change without constituting a numerically distinct act violates irreversibility and is therefore excluded. If it changes no later admissible use, it is inert bookkeeping and not an admissibility-relevant status at all. There is no third possibility. So every admissibility-relevant interface collapses extensionally to admitted or not admitted. $\square$

Theorem 3.65 (AMetric boundary and non-parameterization). *At the admissibility boundary, no admissibility-relevant selector, ranking, ordering, grading, metric, or discrete indexing survives. In particular, there is no admissibility-relevant family of distinct positive statuses beyond the binary admitted / not-admitted split.*

Proof. On the pre-boundary side, no representative, coordinate, or labeling choice may carry admissibility-relevant authority for the same fixed domain. This label-neutrality is not a free modeling preference. If admissibility were allowed to depend on labels, coordinates, or presentation indices as such, then two identity-preserving presentations of the same candidate would receive different admissibility significance solely by representation. That would make representation itself a same-domain selector and would violate the requirement that admissibility be invariant under identity-preserving transformation. So any admissibility-relevant differentiator must be invariant under governance-preserving relabeling. Under that invariance condition, only equality-pattern structure survives: there is no nontrivial unary grading, no admissibility-relevant order, no selector of a privileged positive route, and no meaningful metric beyond discrete equality. Suppose, then, that a parameterized or ranked boundary nevertheless existed. If the parameter or ranking makes a governance-relevant difference, it selects among otherwise admissible candidates or routes and therefore introduces a same-domain selector not preserved by the relabeling symmetry just described. If it makes no governance-relevant difference, it is bookkeeping rather than boundary structure. Likewise, if the extra structure is used to encode an intermediate positive status rather than a selector, it reopens a repair-sensitive middle state between admitted and not admitted, contrary to the fail-closed consequence of irreversibility. Thus every purported admissibility-relevant parameterization or metric either contradicts the boundary invariance conditions or collapses to bookkeeping. Accordingly, any purported admissibility-relevant parameterization at the boundary is forbidden. $\square$

Theorem 3.66 (Standing equals reuse-stable admissibility). *Let Stand_D be any predicate claimed to mark those evaluated acts on a fixed domain D that are eligible for standing-bearing continuation, commitment, or licensed reuse. Then for every act $a \in D$,*

$$\text{Stand}_D(a) = 1 \iff a \text{ is reuse-stably admissible on } D.$$

Equivalently, standing is not an extra primitive or interface-level classifier beyond admissibility on a fixed scope. It is the persistence condition of admissibility under licensed same-scope continuation.

Proof. If $\text{Stand}_D(a) = 1$, then standing is already doing admissibility-relevant gate work on the fixed scope, so by Theorem 3.64 there is no additional governance-relevant classifier beyond the unique gate. Thus a is admitted, and if it were not reuse-stable some licensed same-scope continuation

would silently retarget or reclassify the act. Conversely, if a is reuse-stably admissible but denied standing, then admitted acts are split into two substantive positive subclasses. If that split affects later continuation, it is forbidden same-side gate work; if it affects nothing, it is inert bookkeeping. So the split is impossible as a substantive classification. $\square$

Theorem 3.67 (Extensional uniqueness of the admissible interior). *For a fixed system S on a fixed domain, the admissible interior is unique up to extensional, or lawful, equivalence. More precisely, if I_1 and I_2 are predicates on acts such that each contains exactly the standing acts of S , then for every act a ,*

$$I_1(a) \iff I_2(a).$$

Equivalently: in the standing-instantiated regime, there are no plural admissible interiors in extension; any adequate encoding of the admissible interior collapses pointwise to the standing predicate.

Proof. Assume that I_1 and I_2 both capture exactly the standing acts on the same fixed domain but that they disagree on some act a . Then the same act receives two different standing verdicts on the same fixed scope. If one now adds a selector deciding which interior governs a , the selector itself becomes a new same-domain admissibility-relevant parameter, contrary to Theorems 3.64 and 3.65. If one adds no such selector, then the fixed-domain standing classification is indeterminate, contrary to the role of standing in Theorem 3.66. So two extensional interiors capturing exactly the standing acts cannot disagree on any act. Their apparent plurality therefore collapses extensionally to one admissible interior. $\square$

Theorem 3.68 (Conservation of standing). *For each fixed domain, no internal operation of the licensed construction regime creates, amplifies, transfers, or recovers standing. Standing can only be preserved or destroyed.*

Proof. Recovery would be same-act repair; creation would be standing from a failed act without fresh evaluation; amplification would require standing to admit degrees despite Theorem 3.64; and transfer would require an independent carrier of standing distinct from the gate. Each branch is excluded by the preceding kernel packet. So only preservation or destruction remain. $\square$

Theorem 3.69 (Structural rigidity of the admissibility invariant). *Let D be any fixed domain supporting governed construction. Let J_D be any candidate same-domain realization of the admissibility invariant sufficient to fix eligibility, standing-bearing continuation, and reference-preserving commitment on D . Then exactly one of the following holds:*

- (i) J_D is governance-equivalent to the interface already fixed by the present results: a binary admissibility gate with an AMetric boundary, standing equal to reuse-stable admissibility, a unique admissible interior, and conservation of standing;
- (ii) J_D introduces fresh same-domain selector work, repair-sensitive middle status, or other illicit structure and therefore fails as a faithful realization on D ;
- (iii) J_D changes the admissibility-relevant invariant bundle and therefore changes domain rather than realizing the same invariant on the same domain; or
- (iv) J_D contributes no independent governance work and collapses to bookkeeping.

In particular, the admissibility invariant required for standing-bearing construction admits no governance-non-equivalent same-domain realization.

Proof. By Theorem 3.61, any package sufficient for non-degenerate construction must already cover the same governance roles as K up to governance-equivalence; there is no further governance-non-equivalent package that preserves the same class of governed constructions. Theorems 3.64 and

3.65 fix the interface shape of any such package on a fixed domain: admissibility-relevant status collapses extensionally to admitted or not admitted, and no same-domain selector, ranking, metric, or parameter survives at the boundary. Theorem 3.66 then eliminates any additional positive-side classifier beyond reuse-stable admissibility; Theorem 3.67 eliminates plural same-domain admissible interiors; and Theorem 3.68 eliminates creation, amplification, transfer, and same-act recovery of standing.

Suppose, then, that J_D is a candidate realization of the required invariant on the same fixed domain. If it preserves all of the governance work just listed, then by Theorem 3.61 it is governance-equivalent to the interface already proved, giving (i). If it differs by introducing fresh selector work, middle statuses, repair channels, or imported authority, then it contradicts Theorems 3.64, 3.65, and 3.68, yielding (ii). If it preserves some other boundary only by altering what counts as the same target, the same standing-bearing continuation, or the same admissibility-relevant invariant bundle, then it is no longer realizing the same invariant on the same fixed domain, yielding (iii). If it changes no governance verdict at all, then it is an extensional redescription or bookkeeping layer, yielding (iv). Any further candidate would have to change governance work while avoiding the listed branches, which is exactly what the rigidity result excludes. $\square$

3.6 Consequences for Admissible Structure

The previous sections establish that the foundational layer is fixed, minimal, and non-derivable. The present section derives several structural consequences once that package is in place. These are not additional assumptions; they are strict consequences of the fixed package itself. All consequences in this section follow strictly from the impossibility of same-domain selector introduction, repair-sensitive status, or invariant violation established in Sections 4–6. The forcing pattern is uniform. A purported same-domain alternative to the results below must either (i) introduce an additional admissibility-relevant parameter or selector, (ii) reopen a repair channel or middle status between admission and rejection, or (iii) preserve all same-domain governance work and therefore collapse extensionally to the original interface. The first two branches contradict the kernel results already established; the third yields only bookkeeping redescription. The arguments below therefore proceed by contradiction against those three possibilities rather than by stipulating an interface in advance.

Definition 3.70 (Invariant bundle and same target). An invariant bundle is a family of admissibility-relevant invariants sufficient to determine when two acts have the same governed target. For acts a and b , write $a \sim b$ to mean that they agree on every invariant in the bundle.

Definition 3.71 (Scope-preserving continuation). A continuation C is scope-preserving iff it remains within one admissibility-relevant frame and does not change target by illicit redescription.

Definition 3.72 (Admissible operator). An operator or relation is admissible iff its application respects the governance of standing and admissibility rather than introducing distinctions by redescription alone.

Theorem 3.73 (Scope-preserving invariance). *If C is scope-preserving, then for every admissible act a , one has $a \sim C(a)$.*

Proof. Assume C is scope-preserving but changes an admissibility-relevant invariant of a . Then a and $C(a)$ do not fix the same target. The continuation would therefore have introduced a governance-bearing difference while still claiming to remain within the same scope. That is precisely a silent redescription or hidden reparameterization, which admissibility excludes. Hence every scope-preserving continuation preserves the invariant bundle pointwise. $\square$

Theorem 3.74 (Transport closure). *Let R be an admissible relation. If $b_1 \sim b_2$, then for every q ,*

$$R(q, b_1) \iff R(q, b_2).$$

Proof. If R could distinguish b_1 from b_2 despite $b_1 \sim b_2$, then the distinction would arise from presentation rather than governed target. But admissible construction cannot allow standing-bearing content to depend on mere redescription. Once target is fixed, admissible transport must be closed under that fixing. Therefore admissible relations cannot distinguish same-target presentations. $\square$

Theorem 3.75 (Domain-defined operators). *An admissible operator is defined exactly on the admissible domain, and in particular must be defined on every standing point applied to itself.*

Proof. If an admissible operator were undefined on a standing point applied to itself, then a standing act would fail to lie in its own governed domain. The operator would not govern the admissible act as such. Conversely, if the operator had governance-bearing force outside the admissible domain, it would confer structure where standing is absent and thereby violate the admissibility boundary. Hence admissible operators are defined exactly on the admissible domain. $\square$

Theorem 3.76 (Quotient identity). *Identity of admissible presentation is exactly same-target identity. Equivalently, the admissible redescription orbit of an act coincides with its invariant bundle class.*

Proof. Suppose two presentations lie in the same admissible orbit but differ in same-target identity. Then admissible redescription would change governed target, violating Theorem 3.73. Conversely, suppose two presentations agree on every admissibility-relevant invariant but fail to count as identical for admissible purposes. Then admissibility would preserve a surplus distinction not grounded in target, violating Theorem 3.74. Therefore orbit identity and invariant identity coincide. $\square$

Corollary 3.77 (No silent redescription). *Any purported continuation that changes admissibility-relevant status while claiming same scope is illicit.*

Proof. A same-scope continuation must preserve the invariant bundle by Theorem 3.73. A status-changing continuation would therefore either leave scope or introduce illicit redescription. In either case it is not admissible as same-scope transport. $\square$

Remark 3.78. The formal consequences listed here—transport closure, quotient identity, domain-defined operators, and scope-preserving invariance—therefore do not stand as fresh assumptions. They are structural fallout from the fixedness of the kernel. They belong downstream of foundational closure, not alongside it.

3.7 Closure Against Additional Foundational Conditions

In this section, an *additional foundational condition* means a putative further bearer of irreducible constructional work. It does not mean a merely declared symbol, shorthand, or presentational starting point. The question is whether anything beyond K can function as a further base-level condition of non-degenerate construction.

Any such purported further condition can enter only in one of three ways: it can refine admissibility classification on the same fixed domain, purport to generate admissibility-relevant governance from a lower basis, or serve as an independent same-domain invariant alongside the kernel. These possibilities cover the relevant ways an additional condition could relate to fixed-domain admissibility: it may change admissibility classification, supply or generate it, or leave it

untouched. If it leaves admissibility untouched, then it performs no independent governance work and is derivative rather than foundational.

Theorem 3.79 (Exhaustion of foundational conditions). *Let Q be any purported additional foundational condition. Then exactly one of the following holds:*

- (i) Q performs no independent governance work beyond the kernel K , in which case Q is derivative rather than foundational;
- (ii) Q purports to generate admissibility-relevant governance from a lower same-domain basis; or
- (iii) Q purports to persist as an independent same-domain classifier or invariant alongside the kernel.

Cases (ii) and (iii) are impossible: generation from below is excluded by the non-derivability and no-faithful-lower-generator results, while independent same-domain persistence is excluded by the no-faithful-same-domain-extension results together with the absence of any faithful same-domain counterexample. Therefore no additional foundational condition survives on the same fixed domain as an independent bearer of governance work.

Proof. Take any proposed foundational candidate Q . If Q makes no difference to the class of governed constructions once K is fixed, then by definition it is not foundational; it is descriptive, not load-bearing. Suppose instead that Q performs independent governance work on the same fixed domain. Then, by the classification just stated, that extra work must enter in one of two nontrivial ways: either Q is supposed to generate admissibility-relevant governance from a lower same-domain basis, or it is supposed to survive as an independent same-domain classifier or invariant alongside the kernel.

If Q is claimed to be generated from a lower same-domain basis, then Corollary 3.45 together with Theorem 3.46 excludes the claim: no member of the kernel, and no genuinely foundational governance role, is generated from below while remaining on the same fixed domain.

If Q is instead claimed to persist as an independent same-domain classifier or invariant, then it would constitute an additional same-domain bearer of governance work. But this is excluded by Theorem 3.48: there is no faithful same-domain extension that converts the original acts into a second independent governance architecture rather than collapsing back to the original admissibility boundary, importing illicit structure, violating the base goods, or proving trivial.

A purported fourth option would have to leave admissibility untouched while still doing independent foundational work. But a condition that leaves admissibility untouched leaves the class of governed constructions untouched as well, and so is derivative rather than foundational. Therefore the listed cases are complete. Hence any such condition either does no independent work or falls into one of the two excluded branches. Therefore no additional foundational condition survives as an independent bearer of governance work on the same fixed domain. $\square$

Corollary 3.80 (Foundational closure). *The foundational layer of non-degenerate construction is complete in the following sense: no admissible extension below or alongside K survives on the same fixed domain as an independent bearer of governance work.*

Proof. Immediate from Theorem 3.79. $\square$

3.8 Mechanization Boundary and Proof-Theoretic Scope

Theorem 3.81 (Internal mechanization boundary). *No proof system can establish the package K as a genuinely new internal theorem from a lower layer while preserving the target domain fixed in the fixed-domain construction subsection.*

Proof. Any proof system that already counts its outputs as admissibility-assessable constructions belongs to the target class of the fixed-domain construction subsection. By Theorem 3.31, any such system already operates with admissibility, standing, reference, and irreversibility in place. An “internal construction” of K therefore begins under the very conditions it purports to generate. By Theorems 3.46, 3.47, and 3.48, this is not generation from below but restatement, encoding, or circular recapture. $\square$

Remark 3.82. The point is not anti-formal. It is to mark where formal work properly begins. Once the fixed package is in place, the remaining consequence layer is ordinary derivable mathematics and is an entirely appropriate target for mechanization.

Corollary 3.83 (No lower internal substrate). *A faithful mechanization of the present framework begins at the consequence layer rather than beneath admissibility and standing.*

Proof. If a lower internal layer still counted as part of the same construction regime, then Theorem 3.31 would already place admissibility and standing there. If it did not count as part of that regime, it would not amount to a lower constructional substrate for the present target. So the correct starting point for mechanization is the downstream consequence layer. $\square$

Remark 3.84. A companion formalization is included only as corroboration of this downstream layer. It is not a premise in the argument of the self-contained kernel proof, and the non-derivability results do not depend on repository-level or file-level detail. For present purposes it is enough that representative families—split-collapse, standing/reuse closure, no same-act repair, no generators, no independent carriers, uniqueness of the admissible interior, conservation, and the supporting transport/invariance lemmas—have been machine-checked. That mechanized layer matters here not as a substitute for the present proofs, but as an external check that the equivalence-level uniqueness claims do not rest on merely verbal factorization.

3.9 Endpoint-facing kernel wrappers

Definition 3.85 (Theorem-bearing endpoint use). For a fixed endpoint context R and a branch B ,

$$\text{EndpointUse}_R(B)$$

means that B is offered as a theorem-bearing resolution of the official endpoint image of R . Endpoint use is not raw inscription and not mere report language; it is a governed mathematical act with target, carrier, theorem-status, redescription stability, and downstream reuse.

Theorem 3.86 (Endpoint use is governed construction). *For every fixed endpoint context R and branch B ,*

$$\text{EndpointUse}_R(B) \Rightarrow \text{Constr}(B).$$

Consequently endpoint use is subject to the kernel of non-degenerate construction.

Proof. A theorem-bearing endpoint must fix the target it resolves, preserve the carrier on which the problem is stated, distinguish theorem-status from raw trace or failed proof, permit downstream mathematical reuse, and remain stable under admissible redescription. These are precisely the features of governed construction for a determinate object on a fixed domain. Therefore endpoint use is governed construction. $\square$

Theorem 3.87 (Kernel necessity for theorem-bearing endpoints). *If EndpointUse, then the full kernel*

$$K = \{\text{Adm}, \text{St}, \text{Ref}, \text{Irr}\}$$

is already in force for that endpoint use.

Proof. By Theorem 3.86, endpoint use is governed construction. By Theorem 3.31, every meaningful non-degenerate governed construction already instantiates admissibility, standing, reference, and irreversibility. Thus the kernel is already in force for theorem-bearing endpoint use. $\square$

Theorem 3.88 (Ordinary theoremhood is kernel-internal). *For any proposition P , if $\text{Proof}_R(P)$ is a genuine proof rather than raw trace, then*

$$\text{Proof}_R(P) \Rightarrow \text{Der}(P) \Rightarrow K_R.$$

In particular, an ordinary theorem of $\neg\text{Pos}_{\text{raw}}$, if it exists as a theorem, is not rejected merely because it is negative.

Proof. A genuine proof is a proposition-targeted governed derivation. By Definition 3.8, derivation is the proposition-targeted special case of governed construction. Corollary 3.38 states that every governed derivation already presupposes the full kernel. Therefore ordinary theoremhood is kernel-internal. The sign or polarity of the proposition does not change this conclusion. $\square$

4 A+ as Kernel-Forced Consequence Layer

Definition 4.1 (A+ endpoint consequence package). For an endpoint context R and branch B , A^+ denotes the fixed-domain consequence package forced by kernel-instantiated theorem-bearing endpoint use. It consists of boundary fixation, endpoint bivalence, AMetric boundary, standing as reuse-stable admissibility, conservation of standing, unique admissible interior, no raw trace sufficiency, no selector import, no carrier/domain transfer, no third endpoint status, no auxiliary endpoint-status refinement, and report-preserving output discipline.

Theorem 4.2 (Kernel forces A+ consequences). *For every fixed endpoint context R and branch B ,*

$$\text{EndpointUse}_R(B) \wedge \text{FixedDomain}(R) \Rightarrow A_R^+(B).$$

Proof. By Theorem 3.87, endpoint use instantiates the kernel. Fixed-domain closure then applies the consequences proved in the kernel section. Boundary fixation is admissibility as the joint governance boundary. Endpoint bivalence follows from Theorem 3.64. The AMetric boundary follows from Theorem 3.65. Standing as reuse-stable admissibility follows from Theorem 3.66. Unique admissible interior follows from Theorem 3.67. Conservation follows from Theorem 3.68. No raw trace sufficiency follows from Lemma 3.19. No selector import and no carrier/domain transfer follow from Proposition 3.49 and Theorem 3.69. No third endpoint status follows from Lemma 3.63. No auxiliary endpoint-status refinement follows from structural rigidity. Report-preserving output discipline is the claim-report interface of the same reference, standing, admissibility, and irreversibility roles. Thus A+ is a consequence package, not an independent premise. $\square$

Corollary 4.3 (AMetricity from endpoint use). *On the fixed SAT endpoint image,*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{EndpointUse}_R(B) \wedge \text{FixedDomain}(R) \Rightarrow \text{AMetricBoundary}(R).$$

Proof. The context supplies the carrier and branch slots. Endpoint use plus fixed-domain closure supplies A^+ by Theorem 4.2; the AMetric boundary is one of the listed kernel-forced A+ consequences. $\square$

5 Local SAT Kernel Packet and Weakening Resistance

The self-contained kernel proof above establishes the general necessity of the kernel for non-degenerate construction. The present section records the SAT-local consequences actually used by the endpoint proof and audits the possible weakening routes. The claim is local: the manuscript does not require every formalism to use the same vocabulary. It requires that, on the fixed unrestricted finite CNF-SAT carrier, a theorem-bearing endpoint regime that preserves the same carrier, candidate image, branch slots, endpoint role, and act-time finality already performs the governance work named by the local kernel packet.

5.1 Local SAT kernel packet

Node	General necessity	Local SAT form
K1	Fixed-domain kernel	A theorem-bearing SAT endpoint requires the fixed unrestricted finite CNF carrier, encoded candidate image, branch slots, admissibility boundary, and act-time irreversibility.
K2	Reference fixation	A SAT report cannot be licensed by silently replacing arbitrary finite CNF-SAT with fixed-width k-SAT, a promise fragment, nonuniform circuits, finite benchmarks, oracle behavior, per-length advice, or a different encoding carrier.
K3	Standing-readout separation	<code>CnfSATInPolyTime</code> is the positive endpoint readout. Lower-bound data, solver hardness, failure traces, or candidate obstruction data are not that readout by definition.
K4	Bivalence and fail-closed status	On the fixed endpoint role, a theorem-bearing branch is admitted or not admitted. Unknown, hard, unproved, heuristic, benchmark-failed, partial, or open status is not a third endpoint truth branch.
K5	Unique admissible interior	The same SAT endpoint slot cannot carry plural independent endpoint-governing interiors. A second endpoint-status classifier must collapse to the bridge interface, reset carrier, or fail.
K6	Standing-admissibility identity	Separator standing cannot float outside admissibility. A negative endpoint branch with standing must be admissibly typed; otherwise it is proof support, bookkeeping, or carrier drift.
K7	No generators	Writing that all encoded polynomial-time candidates fail does not self-authorize endpoint standing.
K8	No carriers	Finite tests, solver hardness, oracle separations, nonuniform fragments, proof-complexity fragments, lower-bound templates, or circuit families cannot carry CNF-SAT endpoint status without a lawful bridge back to the fixed carrier.

K9	No post-selection repair	Later candidate discovery, encoding shift, oracle shift, reclassification, or benchmark extension creates a new certified act; it does not repair an earlier unbridged endpoint report as the same act.
K10	AMetric no-selector boundary	No preferred encoding, candidate order, search depth, benchmark set, ranking, heuristic metric, or finite table decides endpoint status before admissibility is fixed.
K11	Report preservation	A report may not exceed its carrier, candidate image, and bridge support. Candidate failure evidence alone cannot replace endpoint proof; candidate-image exclusion becomes endpoint governance only under endpoint-resolving use.
K12	ATS skin non-authority	Labels, syntax, model names, code orderings, proof sketches, and bookkeeping do not become theorem-bearing by being written.
K13	Endpoint exhaustion	Once the carrier, branch slots, endpoint role, and admissibility boundary are fixed, endpoint-bearing branches are exhausted by positive endpoint occupation or governed separator use. Proof support, bookkeeping, unknown status, deferred status, repair, carrier shift, unsupported separator status, and hidden-fifth statuses do not occupy endpoint truth.

Theorem 5.1 (Local SAT kernel packet). *For every theorem-bearing endpoint use on the fixed unrestricted finite CNF-SAT carrier, the local SAT consequences K1–K13 are in force.*

Proof. By Theorem 3.87, endpoint use instantiates reference, standing, admissibility, and irreversibility. K1 records their joint fixed-domain role. K2 and K3 are reference and role-separation consequences. K4–K6 are fail-closed, unique-interior, and standing-admissibility consequences. K7–K9 block self-authorization, unlicensed carrier transfer, and post-selection repair. K10 blocks selector authority at the endpoint boundary. K11 is the SAT report-preservation rule. K12 prevents skin-level notation from becoming theorem-bearing. K13 is endpoint exhaustion: after the SAT endpoint coordinates are fixed, a purported negative branch is positive occupation, proof support, bookkeeping, repair, carrier reset, selector/surrogate import, lawful external comparison, or forbidden second endpoint authority. No additional same-domain endpoint truth occupation remains. $\square$

5.2 Weakening-resistance of the local SAT packet

Definition 5.2 (Core SAT endpoint adequacy packet). The core SAT endpoint adequacy packet is

$$C_{0,\text{SAT}}(R, \text{model}) = \{\text{Ref}_R, \text{St}_R, \text{Adm}_R, \text{Irr}_R\},$$

together with target determinacy, step evaluability, act-time finality, same-regime fidelity, the fixed unrestricted finite CNF-SAT carrier, and the context-bound encoded candidate image of Subsection 2.3.

Definition 5.3 (Separator-governance-permissive weakening). A separator-governance-permissive weakening is a regime W_{sep} such that:

1. W_{sep} preserves the same unrestricted finite CNF-SAT carrier;
2. W_{sep} preserves the same encoded candidate image, positive endpoint slot, and raw negative branch slot;
3. W_{sep} preserves theorem-bearing target-slot fixation, minimal report evaluability, same-regime fidelity, and act-time finality;
4. W_{sep} preserves the core packet $C_{0,\text{SAT}}(R, \text{model})$;
5. W_{sep} permits endpoint-resolving separator governance that is not positive endpoint occupation, proof support only, bookkeeping, carrier shift, repair, or a lawful coequal target role.

Target-slot fixation in this definition is weaker than K13: it fixes what is being evaluated but does not itself exclude unknown, deferred, bookkeeping, or hidden-fifth endpoint statuses. Minimal report evaluability is weaker than K11: it attaches reports to the fixed target and classifies their role, but it does not itself impose no-overreporting. K11 and K13 are proved below as local necessities; they are not smuggled into the definition of W_{sep} .

Lemma 5.4 (Plural SAT data are permitted; plural endpoint interiors are not). *K5 does not prohibit multiple encodings, candidate procedures, proof routes, reductions, lower-bound attempts, solver traces, oracle comparisons, circuit families, proof-complexity systems, or heuristic studies. It prohibits only plural endpoint-governing admissible interiors for the same fixed SAT endpoint slot when those interiors assign endpoint status while preserving the same carrier, candidate image, branch roles, and endpoint target.*

Proof. Plural SAT data may be proof support, auxiliary structure, representation, or bridge material without deciding endpoint status. If two alleged endpoint interiors agree on all endpoint-bearing acts, the second is endpoint bookkeeping. If they disagree while preserving the same carrier and endpoint slot, then the same SAT endpoint has two governing authorities. That is not harmless mathematical plurality; it is plural endpoint governance. $\square$

Lemma 5.5 (Local necessity of K5). *Under $C_{0,\text{SAT}}(R, \text{model})$, a weakening that permits two distinct endpoint-governing admissible interiors for the same SAT endpoint slot either collapses to bookkeeping, splits the domain, or creates a second same-domain endpoint classifier.*

Proof. Let I_1 and I_2 be endpoint-governing interiors for the same fixed endpoint slot. If they agree on all endpoint-bearing acts, they are extensionally identical for endpoint purposes. If they disagree, then the disagreement must be resolved by a higher selector, a domain split, or a second classifier. A higher selector is endpoint governance above the gate; a domain split exits same-carrier use; a second classifier is precisely the forbidden plural endpoint authority. Thus K5 is locally necessary for non-degenerate SAT endpoint determinacy. $\square$

Lemma 5.6 (Local necessity of K6). *Under $C_{0,\text{SAT}}(R, \text{model})$, separator standing outside admissibility is either support-only, carrier shift, or a second endpoint-status gate.*

Proof. Suppose a separator branch has endpoint standing while not being admissibility-bound. If it does not affect endpoint status, it is support, bookkeeping, or lower-status report material. If it affects endpoint status on the same carrier, it performs endpoint-status work outside the admissibility boundary and therefore acts as a second gate. If it avoids that conclusion by changing the carrier, encoding image, branch role, or endpoint slot, it is carrier or domain shift. $\square$

Lemma 5.7 (Local necessity of K11). *Under $C_{0,\text{SAT}}(R, \text{model})$ plus minimal report evaluability, a theorem-bearing SAT endpoint report that exceeds its carrier, encoded candidate image, bridge position, and support is unsupported endpoint authority, silent redescription, or no-carrier transfer.*

Proof. Minimal report evaluability says that a report is attached to the fixed target and can be classified as theorem-bearing, support-level, bookkeeping, or out-of-domain. K11 adds no-overreporting: endpoint force may not exceed the fixed carrier, candidate image, bridge position, and tensor support. A finite obstruction, heuristic lower-bound trace, or candidate failure datum may support a proof; it does not by itself settle the endpoint. If it is reported as endpoint failure on the same carrier without lawful bridge support, it overstates its support. The overstatement is unsupported endpoint authority, silent redescription of a surrogate as the endpoint, or transfer from another carrier. $\square$

Lemma 5.8 (Local necessity of K13). *K13 does not deny epistemic uncertainty, open problem status, incomplete proof search, heuristic evidence, or review status. It states that unknown, deferred, unsupported, bookkeeping, support-level, and hidden-fifth statuses are report or epistemic statuses, not endpoint truth branches on the fixed SAT endpoint slot.*

Proof. Target-slot fixation fixes what is being evaluated. It does not by itself exhaust the endpoint truth slot. K13 supplies the fail-closed endpoint-exhaustion rule: once carrier, branch slots, endpoint role, and admissibility boundary are fixed, an endpoint branch is admitted for endpoint standing or not. Unknown status may describe the state of a problem; support status may describe evidence; bookkeeping may organize notation. None of these occupies endpoint truth. Treating any of them as a third theorem-bearing endpoint branch degenerates the fixed endpoint regime. $\square$

Theorem 5.9 (No strict same-carrier weakening permits independent SAT separator governance). *Under the core SAT endpoint adequacy packet $C_{0,\text{SAT}}(R, \text{model})$, no separator-governance-permissive weakening W_{sep} preserves non-degenerate same-carrier SAT endpoint use.*

Proof. Assume such a weakening exists, and let G_{sep} be the endpoint-resolving separator act it permits. Since G_{sep} is endpoint-resolving, it changes the endpoint status of the fixed SAT slot. Since it is not proof support or bookkeeping, it is theorem-bearing. Since it is not carrier shift or repair, it acts on the same carrier and same endpoint act. Since it is not positive endpoint occupation, it does not occupy the constructive SAT endpoint. Since it is not a lawful coequal target role, it cannot be a separately declared alternative endpoint inside the same positive endpoint-image carrier.

If G_{sep} shares the same admissible interior as the positive bridge, it collapses into bridge occupation and cannot supply separator exclusion. If it has a distinct admissible interior, Lemma 5.5 yields plural endpoint governance, a second classifier, or domain split. If it has standing without admissibility, Lemma 5.6 makes it support-only, carrier shift, or second gate. If its report force exceeds carrier and bridge support, Lemma 5.7 gives unsupported endpoint authority, silent redescription, or no-carrier transfer. If it is neither admitted nor rejected while remaining endpoint-resolving, Lemma 5.8 gives third-status degeneration. In every case the alleged weakening either preserves the original endpoint behavior extensionally, lowers the separator to support or bookkeeping, shifts carrier, or reproduces forbidden second endpoint authority. Hence no strict same-carrier weakening survives. $\square$

Corollary 5.10 (Strict SAT weakening either preserves the packet extensionally or exits endpoint use). *Let W_{sep} be a weaker same-carrier SAT regime preserving core endpoint adequacy. If W_{sep} does not permit independent endpoint governance, it is extensionally equivalent to the local SAT packet for endpoint purposes. If it does permit independent endpoint governance, it fails reference, standing, admissibility, irreversibility, target-slot fixation, minimal report evaluability, no-overreporting, endpoint exhaustion, or same-carrier fidelity.*

Proof. If the weakening does not alter endpoint-bearing statuses, its differences are support-level or presentation-level for the endpoint. If it alters endpoint status by permitting independent separator governance, Theorem 5.9 shows that it exits non-degenerate same-carrier endpoint use by one of the listed failures. $\square$

6 Endpoint-Status Discriminator Discipline

Definition 6.1 (Endpoint-status discriminator). An endpoint-status discriminator for a branch B on context R , written

$$\text{EndpointDisc}_R(C, B),$$

is a status factor, rule, classifier, role-marker, selector, or admissibility-relevant distinction by which B is asserted to occupy an endpoint status on the fixed endpoint image of R .

Definition 6.2 (Independent same-domain discriminator). A discriminator C is independent same-domain governance work for B , written $\text{IndependentDisc}_R(C, B)$, iff it changes endpoint-status, admissibility, standing-bearing continuation, or reportable theorem use on the same fixed domain while not being governance-equivalent to the already fixed admissibility/standing interface.

Theorem 6.3 (Endpoint discriminator trichotomy with domain exit). *Let C be any endpoint-status discriminator attached to branch B on a fixed domain R . Exactly one of the following governance cases applies:*

$$\begin{array}{ll} \text{Bookkeeping}_R(C, B), & \text{GovEquivalent}_R(C, B), \\ \text{IndependentDisc}_R(C, B), & \text{DomainShift}_R(C, B). \end{array}$$

That is, C either changes no endpoint-governance verdict, reproduces the existing kernel-forced interface, performs independent same-domain governance work, or changes the admissibility-relevant invariant bundle and leaves the domain.

Proof. If C changes no admissibility, standing, continuation, endpoint-status, or reportability verdict, it is bookkeeping. If it changes verdicts only in a way already fixed by the kernel-forced admissibility/standing interface, it is governance-equivalent. If it changes endpoint-status verdicts on the same invariant bundle without being governance-equivalent, it performs independent same-domain governance work. If it changes the invariant bundle by which target, carrier, standing-bearing continuation, or admissibility classification are individuated, it changes domain. These cases exhaust whether a discriminator changes governance work and whether the change remains on the fixed domain. $\square$

Theorem 6.4 (Independent same-domain discriminator is forbidden). *For theorem-bearing endpoint use on a fixed domain,*

$$\text{EndpointUse}_R(B) \wedge \text{FixedDomain}(R) \Rightarrow \neg \exists C \text{IndependentDisc}_R(C, B).$$

Proof. Endpoint use plus fixed-domain closure yields the A+ consequence package by Theorem 4.2. The AMetric boundary excludes selector, ranking, metric, or parameter work at the endpoint boundary. Bivalence excludes third endpoint status. Standing as reuse-stable admissibility and the unique admissible interior exclude a second same-domain endpoint gate. Conservation excludes creation, recovery, transfer, or amplification of standing by an extra carrier. Transport closure excludes same-domain redescription changes. Therefore no independent same-domain endpoint-status discriminator survives. By Theorem 3.69, any attempted realization is governance-equivalent, illicit same-domain selector work, domain-changing, or bookkeeping; the independent same-domain case is the illicit case. $\square$

Corollary 6.5 (Role-specific classification is consequence, not source). *If a branch has a role-specific endpoint status, that status is admissible only as a consequence of endpoint use under the kernel-forced fixed-domain interface. It does not create standing for the branch.*

Proof. By Theorem 3.87, endpoint use already presupposes the kernel. By Theorem 4.2, the endpoint-status interface is downstream of that kernel. Hence role-specific classification may record the status a branch has under the fixed interface, but cannot be a source of kernel standing. $\square$

7 SAT Branch Asymmetry and Separator Occupation

Definition 7.1 (Ordinary negative theorem).

$$\text{OrdNegThm}_R := \text{Proof}_R(\neg \text{Pos}_{\text{raw}}).$$

This denotes ordinary theoremhood of the negative SAT proposition inside a governed theorem regime. It is not, by itself, a distinct separator endpoint-status discriminator.

Theorem 7.2 (Negative theoremhood is not automatically defective).

$$\text{OrdNegThm}_R \Rightarrow K_R.$$

Thus the manuscript does not reject $\text{Proof}_R(\neg \text{Pos}_{\text{raw}})$ merely for being negative.

Proof. This is Theorem 3.88 applied to $P = \neg \text{Pos}_{\text{raw}}$. Genuine theoremhood is governed derivation and is already kernel-internal. The excluded object below is not theoremhood of a negative proposition; it is the same-domain separator discrimination induced by the SAT image-separator branch. $\square$

Theorem 7.3 (Ordinary negative theoremhood is not the rejected discriminator). *Ordinary theoremhood of the negative proposition is not, merely as theoremhood, the independent same-domain separator discriminator excluded below:*

$$\text{OrdNegThm}_R \text{ by theorem status alone does not equal } \mathcal{D}_{\text{sep}}(\text{model}).$$

When the content proved is the raw negative branch Sep_{bare} , the SAT-specific lower-bound/no-positive bridge transports that content to candidate endpoint-image exclusion. Only when the negative content is offered as the official endpoint resolution is that support-level exclusion promoted to $\text{Sep}_{\text{img}}^{\text{occ}}$. The resulting contradiction targets the discriminator induced by endpoint occupation, not negative theoremhood as such.

Proof. Ordinary theoremhood is a proposition-targeted governed derivation, hence kernel-internal by Theorem 7.2. The discriminator excluded below is not a theorem-status predicate. Theorem 7.26 supplies only candidate endpoint-image exclusion; endpoint-resolution use is required before that support-level content becomes separator endpoint-image occupation. Thus an ordinary negative theorem is not rejected for polarity or for being theoremhood; the rejected object is the same-domain separator endpoint-status discrimination induced only after official endpoint-resolution use promotes the negative content to $\text{Sep}_{\text{img}}^{\text{occ}}$. $\square$

Theorem 7.4 (Ordinary SAT falsity is not lost; endpoint-bearing negation is role-classified). *The manuscript does not deny that $\neg \text{Pos}_{\text{raw}}$ is the ordinary logical negation of the positive SAT endpoint. It denies only that this negation can occupy theorem-bearing endpoint status on the same fixed SAT*

carrier without taking one of the audited roles. If it is proposition-targeted theoremhood only, it is kernel-internal and not rejected. If it is proof support, it does not resolve the official endpoint. If it changes the carrier, it is not the same endpoint. If it is offered as the official negative endpoint resolution, it becomes endpoint use of Sep_{bare} , and the SAT lower-bound/no-positive bridge transports it first into $\text{CandImageExcl}(\text{model})$ and then, under that endpoint-resolution use, into separator endpoint-image occupation $\text{Sep}_{\text{img}}^{\text{occ}}$.

Proof. Theorem 7.2 grants ordinary negative theoremhood as kernel-internal theoremhood. Theorem 7.3 separates that theoremhood from the rejected separator discriminator. The additional act of offering the negative theorem as the official endpoint resolution is classified by Definition 7.7. Under that use, Theorem 7.8 yields endpoint use of Sep_{bare} ; the SAT-native bridge then routes Sep_{bare} through lower-bound normal form to $\text{CandImageExcl}(\text{model})$, and official endpoint-resolution use promotes that content to $\text{Sep}_{\text{img}}^{\text{occ}}$. Thus ordinary negation remains ordinary negation, while endpoint-bearing use of that negation is role-classified. $\square$

Theorem 7.5 (The SAT separator branch is not a lawful coequal endpoint role). *Inside the fixed unrestricted finite CNF-SAT endpoint-image carrier, Sep_{bare} is not a separately declared lawful coequal target role alongside positive endpoint occupation. The positive branch is direct constructive occupation of CnfSATInPolyTime . A same-carrier negative branch that resolves the official endpoint is proof support, bookkeeping, carrier/domain shift, or endpoint-status governance by candidate-image exclusion. It is not an additional coequal truth slot inside the same positive endpoint-image carrier.*

Proof. The official problem admits the ordinary mathematical alternatives $P = NP$ and $P \neq NP$. The present proof spine, however, evaluates the unrestricted finite CNF-SAT endpoint carrier whose positive endpoint image is polynomial-time CNF-SAT decidability. On that carrier, Theorem 7.10 identifies the positive branch as direct constructive endpoint occupation. The negative CNF branch is the raw complement of that positive image and, by the SAT-native lower-bound/no-positive bridge, becomes candidate endpoint-image exclusion and, under endpoint-resolution use, separator endpoint-image occupation. If this negative branch is not endpoint-resolving, it is support or bookkeeping. If it changes the carrier, it is not the same endpoint. If it resolves the same carrier negatively, it governs endpoint status from outside positive occupation and therefore enters independent-discriminator analysis. Hence it is not a lawful coequal endpoint role inside the same endpoint-image carrier. $\square$

Theorem 7.6 (Separator exclusion is not a ban on negative complexity theory). *The proof does not forbid negative complexity-theoretic mathematics. Circuit lower bounds, proof-complexity lower bounds, relativization or oracle separations, natural-proofs and algebrization barriers, failed candidate algorithms, finite solver-hardness evidence, local obstructions, and proposition-targeted negative theoremhood are legitimate when they remain in their lawful roles. The excluded structure is narrower: same-carrier endpoint-resolving separator governance that does not occupy the positive CNF-SAT endpoint, does not remain proof support or bookkeeping, does not change carrier, and nevertheless governs endpoint status through an independent same-domain discriminator.*

Proof. Negative mathematical data may be proof support, local obstruction, barrier analysis, domain diagnosis, or ordinary theoremhood. None of those roles is excluded by the no-independent-discriminator theorem. The excluded case is the residue after lawful roles have been removed: a same-carrier endpoint-resolving negative branch that does endpoint-status work outside positive occupation while preserving the same carrier. By Theorem 7.4 this is endpoint-bearing negative governance, and by Theorem 7.5 it is not a lawful coequal endpoint role inside the same positive endpoint-image carrier. It is therefore routed to the independent-discriminator closure. $\square$

Definition 7.7 (Official negative endpoint resolution).

$$\text{OfficialEndpointResolution}_R(\neg\text{Pos}_{\text{raw}})$$

means that an ordinary proof of the negative proposition is offered not merely as a proposition-targeted derivation, but as the official negative resolution of the P versus NP endpoint on the unrestricted finite CNF-SAT carrier. This predicate records use of the proof as an endpoint act. It is not a source of theorem standing and does not make negative theoremhood defective.

Theorem 7.8 (Official negative theoremhood becomes endpoint use). *There is no standing intermediate position between ordinary negative theoremhood and negative endpoint use when the theorem is offered as a resolution of the official P versus NP endpoint. Formally, on the fixed SAT carrier,*

$$\begin{aligned} &C_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{OrdNegThm}_R \\ &\wedge \text{OfficialEndpointResolution}_R(\neg\text{Pos}_{\text{raw}}) \Rightarrow \text{EndpointUse}_R(\text{Sep}_{\text{bare}}). \end{aligned}$$

Proof. An ordinary proof of $\neg\text{Pos}_{\text{raw}}$, considered only as a proposition-targeted derivation, is governed theoremhood and is not rejected for being negative. But the additional premise $\text{OfficialEndpointResolution}_R(\neg\text{Pos}_{\text{raw}})$ changes the act under assessment: the proof is now being offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint. By Definition 3.85, theorem-bearing endpoint use is precisely the governed mathematical act that fixes the target, carrier, theorem-status, redescription stability, and downstream reuse of a branch. Since the branch offered is the raw negative branch Sep_{bare} on the fixed SAT carrier, the act is $\text{EndpointUse}_R(\text{Sep}_{\text{bare}})$. If the proof refuses that endpoint-use status, then it is no longer being used as the official endpoint resolution of the same target; it is merely non-endpoint theoremhood, bookkeeping negation, carrier shift, or a degenerate act relative to the fixed endpoint. $\square$

Corollary 7.9 (No floating official negative endpoint). *A proof of $\neg\text{Pos}_{\text{raw}}$ cannot simultaneously be offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint and refuse negative endpoint-use status. If it is not endpoint use, it does not resolve the official endpoint; if it is endpoint use, it enters the SAT negative-occupation and endpoint-status-bivalence results below.*

Proof. Immediate from Theorem 7.8. The only harmless form of ordinary negative theoremhood is theoremhood considered without official endpoint-resolution use. Once the theorem is offered as the endpoint resolution, refusing the endpoint-use classification withholds the very status required for the act to be the same theorem-bearing resolution act. $\square$

Theorem 7.10 (Positive branch is direct constructive endpoint occupation). *The positive CNF-SAT branch directly occupies the constructive endpoint carrier:*

$$\text{CnfPositiveEndpoint} \Rightarrow \text{CnfSATDirectPositiveEndpointOccupation}.$$

Equivalently, when Pos_{raw} is obtained as CnfSATInPolyTime , its endpoint role is the direct polynomial-time CNF-SAT endpoint. It is not a same-domain separator classifier and does not require an independent endpoint-status discriminator.

Proof. The positive branch asserts the existence of a deterministic polynomial-time decision procedure for arbitrary finite CNF-SAT. That assertion directly occupies the constructive endpoint carrier named by CnfSATInPolyTime . No further separator relation over the endpoint image is introduced: the branch does not classify the positive route from outside; it is the positive route. The corresponding audited formal theorem is listed in Appendix A. $\square$

Theorem 7.11 (Raw positive branch equals the formal CNF positive endpoint). *On any context-bound CNF-SAT endpoint model,*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \\ & \Rightarrow (\text{Pos}_{\text{raw}} \leftrightarrow \text{CnfPositiveEndpoint}). \end{aligned}$$

Proof. The premise $\text{Model}_{\text{CNF}}(\text{model}, R)$ fixes the formal CNF-SAT endpoint model associated with the manuscript’s raw carrier. By Definition 1.5, $\text{Pos}_{\text{raw}} := \text{CnfSATInPolyTime}$, the assertion that arbitrary finite CNF-SAT is decidable in deterministic polynomial time. In the formal SAT operator layer, $\text{CnfPositiveEndpoint}$ names that same positive endpoint on the encoded CNF model. Thus the raw positive branch and the formal positive endpoint are equivalent on the fixed model. $\square$

Corollary 7.12 (Bare negative branch equals the formal bare negative branch). *On any context-bound CNF-SAT endpoint model,*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{bare}} \\ & \Rightarrow \text{CnfSATBareNegativeBranch}(R, \text{model}). \end{aligned}$$

Proof. By Theorem 9.1, $\text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}}$. By Theorem 7.11, $\text{Pos}_{\text{raw}} \leftrightarrow \text{CnfPositiveEndpoint}$ on the context-bound CNF model. Hence Sep_{bare} gives $\neg \text{CnfPositiveEndpoint}$, which is exactly $\text{CnfSATBareNegativeBranch}(R, \text{model})$ in the formal SAT operator notation. $\square$

Theorem 7.13 (Canonical CNF model binding is instantiated, not arbitrary). *The fixed SAT context supplies its own canonical encoded CNF model, denoted m_{SAT} , rather than making an arbitrary model context-bound:*

$$\mathcal{C}_{\text{SAT}}(R, m)_{R_{\text{SAT}}^{\text{ctx}}} \wedge \text{FixedDomain}(R_{\text{SAT}}^{\text{ctx}}) \Rightarrow \text{Model}_{\text{CNF}}(m_{\text{SAT}}, R_{\text{SAT}}^{\text{ctx}}).$$

All branch-local theorems below keep $\text{Model}_{\text{CNF}}(\text{model}, R)$ as an explicit premise until the final instantiated SAT context supplies m_{SAT} .

Proof. The CNF model is fixed by the official SAT endpoint carrier: arbitrary finite CNF instances over finite Boolean alphabets, evaluated by formula size, with admissible encodings preserving the same endpoint-image carrier. By Definition 2.2, context binding includes the model-adequacy packet of Subsection 2.3: coverage, soundness, failure adequacy, and image exactness. This determines the canonical context-bound model for $R_{\text{SAT}}^{\text{ctx}}$. It does not license the stronger and false reading that every arbitrary *model* is context-bound. $\square$

7.1 SAT-native lower-bound normal form and endpoint-status governance

The remaining SAT-local burden is to make the raw negative branch visible in ordinary lower-bound language before the endpoint-status exclusion theorem is applied. The point of this subsection is not to import Cook–Levin/Karp machinery and not to add a new source of AASC standing. It records the SAT-native bridge formalized across `AnticheckerReduction.lean` and `SATOperatorProofQueue.lean`: the context-bound candidate model is adequate for ordinary deterministic polynomial-time CNF-SAT candidates; the raw negative branch is standard lower-bound normal form over that exact encoded candidate image; that normal form is candidate endpoint-image exclusion; candidate endpoint-image exclusion supplies theorem-level endpoint-status discrimination; and an endpoint-resolving negative use of that exclusion is endpoint-status governance, not a harmless proof-support observation.

Definition 7.14 (Standard SAT lower-bound normal form). For a context-bound encoded CNF-SAT candidate model, $\text{StdLB}(\text{model})$ denotes the standard lower-bound normal form:

$$\forall c (\text{PolyCand}_{\text{model}}(c) \Rightarrow \text{FailsSAT}_{\text{model}}(c)).$$

Here c ranges over encoded deterministic polynomial-time candidate SAT procedures, $\text{PolyCand}_{\text{model}}(c)$ says that the code is polynomial-time in the fixed encoded CNF model, and $\text{FailsSAT}_{\text{model}}(c)$ says that the decoded procedure fails to decide arbitrary finite CNF-SAT on that carrier. The corresponding Lean predicate is `CnfSATStandardLowerBoundNormalForm`.

Definition 7.15 (Candidate endpoint-image exclusion). $\text{CandImageExcl}(\text{model})$ denotes universal exclusion of every encoded polynomial-time candidate from occupying the positive SAT endpoint image. In Lean this is `CnfSATCandidateEndpointImageExclusion`. It is a theorem-level statement about the fixed encoded candidate image; it is not the assertion that a practical witness selector or uniform failing-instance generator has been constructed.

Definition 7.16 (Theorem-level endpoint-status discriminator). $\text{ThmLevelDisc}(\text{model})$ denotes the theorem-level discriminator induced by candidate endpoint-image exclusion: the global status assignment under which each encoded polynomial-time candidate is classified as a non-occupant of the positive CNF-SAT endpoint image. The corresponding Lean predicate is `CnfSATTheoremLevelEndpointStatusDiscriminator`. This is not an algorithmic separator and not witness selection; it is endpoint-status classification at theorem level.

Theorem 7.17 (Bare negative branch has standard lower-bound normal form). *On the fixed context-bound CNF-SAT endpoint carrier,*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{bare}} \Rightarrow \text{StdLB}(\text{model}).$$

The audited Lean anchor is `cnfSATBareNegativeBranch_standardLowerBoundNormalForm`.

Proof. By Corollary 7.12, Sep_{bare} gives the formal bare negative branch:

$$\text{CnfSATBareNegativeBranch}(R, \text{model}).$$

This is the negation of the positive CNF-SAT endpoint on the context-bound encoded model. In ordinary machine-language form, it is precisely the universal lower-bound normal form: every encoded deterministic polynomial-time candidate fails to occupy the positive SAT endpoint. The cited Lean anchor records the same route from the formal bare negative branch to standard lower-bound normal form. $\square$

Theorem 7.18 (Lower-bound normal form is candidate endpoint-image exclusion). *On the fixed encoded CNF-SAT candidate image,*

$$\text{StdLB}(\text{model}) \Rightarrow \text{CandImageExcl}(\text{model}).$$

The Lean counterpart is `cnfSATImageSeparatorBranch_standardLowerBoundNormalForm` together with `CnfSATCandidateEndpointImageExclusion`.

Proof. The standard lower-bound normal form says that every encoded polynomial-time candidate fails SAT on the fixed unrestricted finite CNF carrier. By Theorem 2.9, the positive endpoint image is exactly the image of encoded polynomial-time candidates with no SAT failure. Universal failure of those candidates is therefore universal exclusion from the positive endpoint image. This is candidate endpoint-image exclusion. $\square$

Theorem 7.19 (Candidate endpoint-image exclusion induces theorem-level candidate-image classification). *For the context-bound CNF-SAT endpoint model,*

$$\text{CandImageExcl}(\text{model}) \Rightarrow \text{ThmLevelDisc}(\text{model}).$$

The Lean anchor is `cnfSATTheoremLevelEndpointStatusDiscriminator_of_candidateImageExclusion`; the direct bare-negative form is `cnfSATBareNegativeBranch_theoremLevelDiscriminator`.

Proof. Candidate endpoint-image exclusion globally assigns each encoded polynomial-time candidate the endpoint-relevant status “non-occupant of the positive CNF-SAT endpoint image.” This is theorem-level candidate-image classification. It does not compute a failing formula uniformly, does not select a canonical witness, and does not claim an algorithmic separator. It becomes endpoint-status governance only when the classification is used as the endpoint-resolving negative branch on the fixed carrier; otherwise it remains lower-bound or proof-support content. The cited Lean anchor records the theorem-level classification surface. $\square$

Definition 7.20 (Legitimate invariant use versus endpoint-status governance). The Lean type `CnfSATInvariantUseKind` separates three uses of invariant or obstruction material:

`auxiliaryInvariant`, `localObstruction`, `endpointStatusGovernance`.

Auxiliary invariants and local obstructions are legitimate proof-support material. Endpoint-status governance on the fixed SAT carrier is the forbidden same-domain endpoint classifier. This is recorded by `CnfSATInvariantUseLegitimate` and `cnfSATEndpointStatusGovernance_not_legitimate`.

Use kind	Legitimate?	Reason
Auxiliary invariant	Yes	Supports a proof without governing the endpoint-status surface.
Local obstruction	Yes	Records local incompatibility or proof support inside an already fixed route.
Endpoint-status governance	No	Acts as independent same-domain authority over endpoint occupation on the fixed SAT carrier.

Definition 7.21 (Candidate-image use classification). The Lean candidate-image-use type distinguishes four uses of candidate-image exclusion:

`ProofSupportOnly`, `EndpointResolvingNegThm`, `EndpointResolvingNonGov`, `CarrierChangingLB`.

The corresponding Lean classifier is `CnfSATCandidateImageExclusionUseClassification`. In manuscript language: proof-support-only use does not resolve the endpoint; endpoint-resolving negative theorem use is official negative endpoint use; endpoint-resolving non-governance is the hidden fifth case; and carrier-changing lower-bound use exits the fixed carrier.

Theorem 7.22 (Endpoint-resolving non-governance is the hidden fifth case). *There is no fixed-carrier use of candidate-image exclusion that both resolves the official endpoint negatively and avoids endpoint-status governance:*

$$\neg \text{EndpointResolvingNonGov}.$$

The Lean anchor is `cnfSATEndpointResolvingNonGovernance_hiddenFifthCase_impossible`.

Proof. If candidate-image exclusion is only proof support, it does not resolve the official endpoint. If it resolves the official endpoint negatively on the fixed carrier, it governs the endpoint-status surface by classifying candidate positive endpoint occupants as non-occupants. If it avoids that governance by changing the carrier, it is a carrier/domain shift. The alleged fourth possibility—endpoint-resolving but not endpoint-governing on the same carrier—is exactly the hidden fifth case, and the Lean classifier assigns it no inhabitant. $\square$

Definition 7.23 (Candidate-image endpoint governance). $\text{CandImageGov}_R(\text{model})$ means that candidate-image exclusion is used as endpoint-status governance on the fixed CNF-SAT carrier:

$$\text{CandImageGov}_R(\text{model}) := \text{FixedDomain}(R) \wedge \neg \text{CnfSATDirectPositiveEndpointOccupation} \\ \wedge \text{ThmLevelDisc}(\text{model}).$$

This matches the Lean predicate `CnfSATEndpointStatusGovernanceByCandidateImageExclusion`: the carrier is fixed, the use does not occupy the positive endpoint, and theorem-level candidate-status discrimination is present.

Theorem 7.24 (Official negative endpoint use is candidate-image endpoint-status governance). *For the context-bound finite CNF-SAT carrier,*

$$\text{CnfSATOfficialNegativeEndpointUse}(R, \text{model}) \Rightarrow \text{CandImageGov}_R(\text{model}).$$

The Lean anchor is `cnfSATOfficialNegativeEndpointUse_endpointStatusGovernance`.

Proof. Official negative endpoint use contains the fixed endpoint domain, the context-bound CNF model, and the bare negative branch. The bare negative branch yields candidate-image exclusion and then the theorem-level endpoint-status discriminator by Theorems 7.17–7.19. Since the use is negative, it does not occupy the direct positive endpoint. Therefore it is endpoint-status governance by candidate-image exclusion, exactly as stated by the Lean theorem. $\square$

Corollary 7.25 (Endpoint-resolving negative theorem use is endpoint-status governance). *If candidate-image exclusion is used as the endpoint-resolving negative theorem on the fixed CNF-SAT carrier, then it is endpoint-status governance:*

$$\text{EndpointResolvingNegThm} \Rightarrow \text{CandImageGov}_R(\text{model}).$$

The Lean anchor is `cnfSATEndpointResolvingNegativeTheorem_is_endpointStatusGovernance`.

Proof. By Definition 7.21, the endpoint-resolving negative-theorem case is official negative endpoint use. Theorem 7.24 then gives endpoint-status governance by candidate-image exclusion. Proof-support-only use does not resolve the endpoint, and carrier-changing use exits the fixed carrier; the alleged endpoint-resolving non-governance use is ruled out by Theorem 7.22. $\square$

Theorem 7.26 (Raw negative branch forces candidate endpoint-image exclusion). *On the fixed context-bound CNF-SAT endpoint carrier,*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{bare}} \Rightarrow \text{CandImageExcl}(\text{model}).$$

The audited Lean support is the route through `cnfSATBareNegativeBranch_standardLowerBoundNormalForm` and the candidate-image exclusion bridge.

Proof. The implication is not a reportability principle and does not assert that raw truth creates a Clay report act. By Theorem 7.17, Sep_{bare} gives standard SAT lower-bound normal form over the fixed encoded candidate image. By Theorem 7.18, that normal form is candidate endpoint-image exclusion. Therefore $\text{CandImageExcl}(\text{model})$ follows. No endpoint-image separator occupation is inferred at this stage. $\square$

Theorem 7.27 (Candidate-image exclusion becomes separator occupation only under official endpoint resolution). *On the fixed context-bound CNF-SAT endpoint carrier,*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{CandImageExcl}(\text{model}) \\ & \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}}) \Rightarrow \text{Sep}_{\text{img}}^{\text{occ}}. \end{aligned}$$

Proof. Candidate-image exclusion by itself is lower-bound content: it may be proof support, a local obstruction, ordinary theorem content, or carrier-diagnostic information. The additional hypothesis $\text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})$ changes the act under assessment. It says that the negative content is being used as the official endpoint-resolving branch for the same unrestricted finite CNF-SAT endpoint. Under that use, the support-level image exclusion is promoted to separator endpoint-image occupation. This is precisely $\text{Sep}_{\text{img}}^{\text{occ}}$. No support-level lower-bound statement is promoted merely by being true or written. $\square$

Lemma 7.28 (Separator occupation contains candidate-image exclusion). *On the fixed context-bound CNF-SAT endpoint carrier, separator endpoint-image occupation includes the support-level candidate-image exclusion content:*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{img}}^{\text{occ}} \Rightarrow \text{CandImageExcl}(\text{model}).$$

Proof. By Definition 1.6, $\text{Sep}_{\text{img}}^{\text{occ}}$ is candidate-image exclusion after that content is used as the official endpoint-resolving negative branch on the fixed carrier. The occupation status therefore contains, rather than replaces, the support-level content $\text{CandImageExcl}(\text{model})$. This lemma is an unpacking lemma only: it does not promote lower-bound content into endpoint occupation and does not assert endpoint use from $\text{CandImageExcl}(\text{model})$ alone. $\square$

7.2 SAT Negative Occupation Exhaustion and Endpoint-Status Bivalence

The preceding bridge is not a representational convention. It is the SAT-local lower-bound/no-positive equivalence on the context-bound unrestricted finite CNF-SAT endpoint model. The audited Lean formalization separates two locks that make the negative-occupation step explicit. First, official negative endpoint use is exhausted into four cases: image-separator occupation, ordinary negative alternative, bookkeeping-only negative, or carrier/domain shift. Second, governed endpoint use on the fixed finite CNF-SAT carrier has only two endpoint statuses: positive occupation or separator occupation. These locks are included to block the claim that the official raw negative may remain a third theorem-bearing endpoint status that is neither positive nor separator.

The phrase “official negative endpoint use” is not a new escape hatch and not an AASC-added source of standing. By Theorem 7.8, ordinary theoremhood of $\neg \text{Pos}_{\text{raw}}$ becomes negative endpoint use exactly when it is offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint. If it is not so offered, it does not resolve the official target. If it is so offered, it enters the occupation-exhaustion and bivalence locks in this subsection.

Definition 7.29 (Official negative occupation alternatives). For the context-bound CNF-SAT endpoint model, $\text{CnfSATOfficialNegativeEndpointUse}(R, \text{model})$ denotes theorem-bearing official negative endpoint use of the finite CNF-SAT carrier. The Lean surface distinguishes four exhaustive negative-occupation alternatives:

$$\text{Sep}_{\text{img}}^{\text{occ}}, \quad \text{OrdNeg}_R^{\text{alt}}, \quad \text{CnfSATBookkeepingNegativeOnly}(R, \text{model}), \quad \text{CnfSATCarrierShift}(R, \text{model}).$$

The first is separator endpoint-image occupation. The second is a putative ordinary negative endpoint alternative that remains governed but does not occupy separator status. The third is bookkeeping-only negation, not theorem-bearing endpoint use. The fourth is a change of carrier or domain. The corresponding Lean objects are $\text{CnfSATOfficialNegativeEndpointUse}$, $\text{CnfSATNegativeOccupationExhaustion}$, $\text{CnfSATBookkeepingNegativeOnly}$, and $\text{CnfSATCarrierShift}$.

Theorem 7.30 (SAT negative occupation exhaustion). *Official negative endpoint use on the context-bound finite CNF-SAT carrier is exhausted by image-separator occupation, ordinary negative alternative, bookkeeping-only negative, or carrier/domain shift. In Lean this is the theorem $\text{cnfSATNegativeOccupation_exhaustion}$.*

Proof. The exhaustion is branch-complete by construction of the fixed SAT endpoint image. A theorem-bearing negative endpoint either occupies the image-separator branch, claims a distinct governed ordinary negative endpoint status, remains a bookkeeping-only negation without endpoint use, or changes the carrier/domain on which the official endpoint is being evaluated. These alternatives cover whether the negative is endpoint-bearing and, if endpoint-bearing, whether it remains on the fixed carrier and whether it occupies separator status. The cited Lean anchor records this exact exhaustion. $\square$

Theorem 7.31 (Non-optionality of SAT negative occupation). *On the fixed context-bound finite CNF-SAT endpoint carrier, official negative endpoint use together with fixed endpoint domain, context-bound CNF model, and the AMetric boundary entails image-separator occupation:*

$$\text{CnfSATOfficialNegativeEndpointUse}(R, \text{model}) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{AMetricBoundary}(R) \Rightarrow \text{Sep}_{\text{img}}^{\text{occ}}.$$

The Lean anchor is $\text{cnfSATNegativeOccupation_nonoptional}$.

Proof. Apply the occupation exhaustion of Theorem 7.30. Fixed endpoint domain excludes carrier/-domain shift; official negative endpoint use excludes bookkeeping-only negation; and the AMetric boundary excludes the ordinary-negative-alternative branch. The corresponding Lean anchors are:

```
cnfSATNoCarrierShift_of_fixedEndpointDomain
cnfSATOfficialNegativeEndpointUse_not_bookkeepingOnly
cnfSATNoOrdinaryNegativeAlternative_of_ametricBoundary
cnfSATNegativeOccupation_nonoptional
```

The remaining exhaustive branch is separator endpoint-image occupation. $\square$

Definition 7.32 (SAT endpoint-status bivalence). The Lean status type $\text{CnfSATEndpointStatus}$ has exactly two constructors, `positive` and `separator`. The predicates $\text{CnfSATGovernedEndpointUse}$ and $\text{CnfSATGovernedEndpointStatusUse}$ mark governed endpoint use and governed endpoint-status use on the context-bound finite CNF-SAT carrier.

Theorem 7.33 (Endpoint-status bivalence for governed CNF-SAT endpoint use). *Governed endpoint use on the fixed finite CNF-SAT carrier is endpoint-status bivalent: it has positive status or separator status, and no third governed endpoint status. The Lean anchors are `cnfSATEndpointStatus_exhaustive` and `cnfSATGovernedEndpointUse_bivalent`.*

Proof. The endpoint-status surface contains only the positive and separator constructors. A governed endpoint use must occupy one of these statuses. Any alleged third governed endpoint status would either be bookkeeping, a carrier shift, or an independent same-domain status refinement; those alternatives are excluded by the fixed-domain endpoint interface already established in Sections 6 and 7. The Lean anchors record the status-exhaustive and governed-use bivalence forms. $\square$

Theorem 7.34 (Negative governed endpoint use has separator status). *For the governed finite CNF-SAT endpoint,*

$$\begin{aligned} & \text{CnfSATGovernedEndpointUse}(R, \text{model}) \wedge \neg \text{CnfPositiveEndpoint} \\ & \Rightarrow \text{CnfSATImageSeparatorBranch}(R, \text{model}). \end{aligned}$$

Equivalently, a governed endpoint use that is not positive occupies separator status and therefore the image-separator branch. The Lean anchors are `cnfSATNegativeGovernedEndpointUse_has_separatorStatus` and `cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse`.

Proof. By Theorem 7.33, governed endpoint use is positive or separator. The assumption $\neg \text{CnfPositiveEndpoint}$ eliminates positive endpoint occupation. The only remaining governed endpoint status is separator status, and separator status is the SAT image-separator branch on the context-bound CNF model. This is the direct branch form formalized by `cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse`. $\square$

Consequently, the objection that the official raw negative may be theorem-bearing while avoiding image-separator occupation is not a remaining third route. Mere proposition-targeted negative theoremhood is harmless but does not by itself resolve the official endpoint. Once the same theorem is offered as the official negative endpoint resolution, Theorem 7.8 makes it endpoint use; on the fixed governed finite CNF-SAT carrier, a negative endpoint is not positive; endpoint-status bivalence forces separator status; separator status occupies the image-separator branch; and separator endpoint-image occupation is the branch closed by the no-independent-discriminator route below.

Definition 7.35 (SAT-local independence bridge). $\mathcal{I}_{\text{sep}}(\text{model})$ denotes the SAT-local same-domain independence bridge: a separating candidate-status classifier for the context-bound CNF-SAT model is independent same-domain endpoint-status discrimination. In the Lean surface this is the predicate `CnfSeparatingClassifierIsIndependentSameDomain(model)`. It is not a source of standing and not an A+ premise; it records the SAT-local condition under which an image-separator branch produces the independent discriminator excluded by kernel-forced rigidity.

Definition 7.36 (Separator discriminator and no-independent closure). For the context-bound CNF-SAT model, $\mathcal{D}_{\text{sep}}(\text{model})$ denotes the independent same-domain endpoint-status discriminator induced by the SAT image-separator branch:

$$\mathcal{D}_{\text{sep}}(\text{model}) := \text{CnfSATIndependentSeparatorEndpointStatusDiscriminator}(\text{model}).$$

The predicate $\mathcal{N}_{\text{sep}}(\text{model})$ denotes the kernel-forced no-independent-separator-classifier closure for that same model:

$$\mathcal{N}_{\text{sep}}(\text{model}) := \text{CnfNoIndependentSeparatingClassifier}(\text{model}).$$

It is used only at the endpoint-use consequence layer, and its operative content is that no independent same-domain separator discriminator survives on the fixed CNF-SAT endpoint carrier.

Theorem 7.37 (No-independent closure excludes the separator discriminator). *For the context-bound CNF-SAT model,*

$$\mathcal{N}_{\text{sep}}(\text{model}) \Rightarrow \neg \mathcal{D}_{\text{sep}}(\text{model}).$$

Proof. By Definition 7.36, $\mathcal{D}_{\text{sep}}(\text{model})$ is the formal independent separator endpoint-status discriminator and $\mathcal{N}_{\text{sep}}(\text{model})$ is the corresponding no-independent-separator-classifier closure. The latter is precisely the negation of the former at the fixed SAT endpoint carrier: independent same-domain separator discrimination is excluded. Thus $\mathcal{N}_{\text{sep}}(\text{model}) \Rightarrow \neg \mathcal{D}_{\text{sep}}(\text{model})$. $\square$

Theorem 7.38 (Endpoint-image exclusion is independent only under endpoint-resolving use). *Candidate endpoint-image exclusion is not automatically forbidden and is not automatically an independent same-domain discriminator. On the fixed unrestricted finite CNF-SAT carrier, $\text{CandImageExcl}(\text{model})$ remains legitimate when it is used as proof support, a local obstruction, lower-bound lemma, ordinary theorem content not offered as the official endpoint resolution, or carrier-diagnostic information. It invokes the SAT-local independence bridge only when it is used to resolve the fixed endpoint negatively while preserving the same carrier and while not occupying the positive endpoint:*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \\ & \quad \wedge \text{CandImageExcl}(\text{model}) \wedge \text{Sep}_{\text{img}}^{\text{occ}} \wedge \text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}) \\ & \Rightarrow \mathcal{I}_{\text{sep}}(\text{model}). \end{aligned}$$

Proof. Proof-support use of candidate-image exclusion does not settle the official endpoint; it remains lawful lower-bound or obstruction material. Carrier-changing use is not the same endpoint. Bookkeeping use changes no endpoint status. The only remaining case in which candidate-image exclusion does endpoint work on the fixed carrier is endpoint-resolving use of the separator endpoint-image occupation $\text{Sep}_{\text{img}}^{\text{occ}}$. In that case the report classifies the positive endpoint image from outside positive occupation. By Theorems 7.24 and 7.5, such use is not positive endpoint occupation, proof support, bookkeeping, carrier shift, or a lawful coequal target role. It is same-domain endpoint-status governance by candidate-image exclusion, and that is exactly the SAT-local independence bridge $\mathcal{I}_{\text{sep}}(\text{model})$. Thus lower-bound content is not globally banned; only endpoint-resolving use on the fixed carrier invokes the independence bridge. $\square$

Theorem 7.39 (SAT-local bridge instantiated by endpoint-resolving separator occupation). *In the fixed SAT context, the SAT-local independence bridge is invoked by candidate-image exclusion only when that content is in endpoint-resolving separator occupation:*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{CandImageExcl}(\text{model}) \\ & \quad \wedge \text{Sep}_{\text{img}}^{\text{occ}} \wedge \text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}) \Rightarrow \mathcal{I}_{\text{sep}}(\text{model}). \end{aligned}$$

Proof. The model-binding clause fixes the encoded CNF-SAT carrier: arbitrary finite CNF instances over finite Boolean alphabets, evaluated by formula size, with admissible encodings preserving the same SAT endpoint image. The support-level lower-bound content is $\text{CandImageExcl}(\text{model})$; it may remain legitimate proof support. The hypothesis $\text{Sep}_{\text{img}}^{\text{occ}}$ identifies endpoint-image separator occupation, not mere lower-bound content. The hypothesis $\text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}})$ says that this occupation is being used to resolve the fixed endpoint negatively. Under these conditions, Theorem 7.38 applies: candidate-image exclusion classifies the positive endpoint image from outside

positive occupation. Hence the separating classifier is same-domain endpoint-status work rather than transported proof support or report bookkeeping. This is the SAT-local bridge represented in Lean by `CnfSeparatingClassifierIsIndependentSameDomain(model)`. The theorem therefore does not assert an independent discriminator from lower-bound content alone; it states the endpoint-use condition under which the image-separator theorem applies. $\square$

Theorem 7.40 (Separator endpoint occupation requires independent separator discriminator). *For the fixed SAT endpoint carrier,*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \mathcal{I}_{\text{sep}}(\text{model}) \wedge \text{Sep}_{\text{img}}^{\text{occ}} \Rightarrow \mathcal{D}_{\text{sep}}(\text{model}).$$

Proof. The formal SAT operator theorem supplies the Lean-side rule for the image-separator branch. In manuscript terms, that branch is invoked here only as separator endpoint occupation $\text{Sep}_{\text{img}}^{\text{occ}}$, together with the explicit SAT-local independence bridge. Under those two conditions, it yields $\mathcal{D}_{\text{sep}}(\text{model})$. This is the exact branch asymmetry: the positive branch is direct endpoint occupation, while endpoint-resolving separator occupation induces independent same-domain endpoint-status discrimination. $\square$

Theorem 7.41 (Official endpoint-resolving bare negative branch requires independent separator discriminator). *For the fixed SAT endpoint carrier,*

$$\begin{aligned} \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{bare}} \\ \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}}) \Rightarrow \mathcal{D}_{\text{sep}}(\text{model}). \end{aligned}$$

Proof. By Theorem 7.26, Sep_{bare} gives $\text{CandImageExcl}(\text{model})$, not endpoint occupation by itself. The additional endpoint-resolution premise is essential. Together with candidate-image exclusion, Theorem 7.27 gives $\text{Sep}_{\text{img}}^{\text{occ}}$. Theorem 7.42 gives $\text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}})$. With endpoint use explicit, Theorem 7.39 gives $\mathcal{I}_{\text{sep}}(\text{model})$, and Theorem 7.40 gives $\mathcal{D}_{\text{sep}}(\text{model})$. Thus the independent discriminator is required only for the official endpoint-resolving use of the bare negative branch, not for lower-bound content or ordinary negative theoremhood as such. $\square$

Theorem 7.42 (Separator endpoint-image occupation is endpoint use). *In the fixed SAT context, occupation of the separator endpoint-image branch is endpoint use:*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{Sep}_{\text{img}}^{\text{occ}} \Rightarrow \text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}).$$

Proof. The support-level lower-bound content is $\text{CandImageExcl}(\text{model})$. It may appear as a lower-bound lemma, obstruction, or proof-support statement without endpoint use. The symbol $\text{Sep}_{\text{img}}^{\text{occ}}$, by contrast, denotes endpoint-image separator occupation: candidate-image exclusion after it is used as the official endpoint-resolving negative branch on the fixed carrier. Thus the theorem does not promote arbitrary lower-bound content into endpoint governance. It only unpacks the occupation notation: when separator endpoint-image occupation is present on the fixed endpoint carrier, that presence is theorem-bearing endpoint use of the separator branch. $\square$

Theorem 7.43 (Branch-local no independent SAT separator classifier). *For the fixed SAT endpoint carrier, separator endpoint-image occupation in endpoint use triggers the kernel-forced no-independent-discriminator consequence:*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}}) \Rightarrow \mathcal{N}_{\text{sep}}(\text{model}).$$

Proof. Endpoint use is governed construction, hence it instantiates the kernel by Theorem 3.87. The fixed-domain A+ consequence layer then supplies the AMetric boundary, unique admissible interior, standing conservation, no-selector discipline, transport closure, and structural rigidity. On a fixed SAT endpoint carrier, an endpoint-status discriminator must be bookkeeping, governance-equivalent, domain-changing, or independent same-domain governance work by Theorem 6.3. The first two cases do not yield $\mathcal{D}_{\text{sep}}(\text{model})$, the third is not the same SAT endpoint domain, and the fourth is excluded by the kernel-forced consequence layer. Thus separator endpoint-image use on the context-bound SAT model yields $\mathcal{N}_{\text{sep}}(\text{model})$. $\square$

Theorem 7.44 (Separator endpoint-image occupation impossible). *In the fixed SAT context,*

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \Rightarrow \neg \text{Sep}_{\text{img}}^{\text{occ}}.$$

Proof. Assume $\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R)$ and suppose $\text{Sep}_{\text{img}}^{\text{occ}}$. Theorem 7.42 gives $\text{EndpointUse}_R(\text{Sep}_{\text{img}}^{\text{occ}})$. Lemma 7.28 gives the support-level content $\text{CandImageExcl}(\text{model})$. With endpoint use explicit, Theorem 7.39 gives $\mathcal{I}_{\text{sep}}(\text{model})$. Theorem 7.43 gives $\mathcal{N}_{\text{sep}}(\text{model})$. By Theorem 7.37, $\neg \mathcal{D}_{\text{sep}}(\text{model})$. But Theorem 7.40 gives $\mathcal{D}_{\text{sep}}(\text{model})$. Contradiction. Therefore $\neg \text{Sep}_{\text{img}}^{\text{occ}}$. $\square$

No fifth negative occupation. Thus the alleged third governed negative occupation is not a fifth case. If it has no endpoint-governance effect, it is bookkeeping. If it has endpoint-governance effect on the fixed carrier while being neither positive nor separator, it is hidden selector work or forbidden third-status refinement. If it depends on later reclassification of the same act, it is repair-sensitive middle status. If it avoids those consequences only by changing the admissibility-relevant invariant bundle, it is carrier or domain shift. Hence every purported governed negative endpoint occupation that is not separator either fails to be endpoint-bearing, violates the fixed-domain endpoint-status discipline, reopens repair, or leaves the fixed CNF-SAT carrier. No coherent fifth occupation remains.

Theorem 7.45 (Official endpoint-resolving bare negative branch impossible). *In the fixed SAT context,*

$$\begin{aligned} &\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \\ &\Rightarrow \neg(\text{Sep}_{\text{bare}} \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})). \end{aligned}$$

Proof. Assume $\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R)$ and suppose $\text{Sep}_{\text{bare}} \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})$. Theorem 7.26 gives $\text{CandImageExcl}(\text{model})$. By Theorem 7.27, the conjunction of candidate-image exclusion with official endpoint-resolution use gives $\text{Sep}_{\text{img}}^{\text{occ}}$. But Theorem 7.44 excludes $\text{Sep}_{\text{img}}^{\text{occ}}$ under the same fixed context. Contradiction. Thus the bare negative branch is impossible as the official endpoint-resolving negative branch. Non-endpoint lower-bound content and ordinary negative theoremhood remain governed by the non-explosiveness theorem. $\square$

8 Route-Locus Exhaustion for SAT Separator Use

Definition 8.1 (Endpoint-resolving negative SAT route). An endpoint-resolving negative SAT route is a proof trace, invariant, obstruction, lower-bound theorem, model comparison, classifier, or report that is used to make Sep_{bare} the official negative resolution of the unrestricted finite CNF-SAT endpoint on the fixed carrier.

Definition 8.2 (SAT route loci). The SAT route-locus basis consists of the following loci:

Locus	SAT form	Disposition
L1	Carrier identity: unrestricted finite CNF versus fixed-width, promise, oracle, nonuniform, or benchmark carriers.	Carrier/domain shift unless bridged back.
L2	Candidate image: uniform deterministic polynomial-time machine/bound pairs.	Adequate model by Subsection 2.3.
L3	Encoding or presentation artifacts, duplicate codes, and representative choices.	Bookkeeping unless they change endpoint status, in which case selector/import failure.
L4	Positive endpoint occupation by a deterministic polynomial-time SAT decider.	Direct positive branch.
L5	Standard lower-bound normal form: every encoded polynomial-time candidate fails.	Proof support until endpoint-resolving; then enters image exclusion.
L6	Candidate endpoint-image exclusion.	Endpoint-governing if used as official negative resolution.
L7	Candidate-status classifier or theorem-level endpoint discriminator.	Routes to independent-discriminator closure.
L8	Nonuniform, oracle, promise, average-case, parameterized, or restricted-fragment shift.	Not the same endpoint carrier.
L9	Finite benchmark, empirical solver behavior, or heuristic hardness.	Support only; not endpoint occupation.
L10	Repair or later reclassification of an earlier endpoint act.	Violates act-time finality.
L11	Selector or ranking over candidates, encodings, search depths, or benchmarks.	AMetric no-selector violation if endpoint-bearing.
L12	Endpoint-status governance outside positive occupation.	Forbidden independent same-domain authority.

Theorem 8.3 (SAT route-locus exhaustion). *Every same-carrier endpoint-resolving negative SAT route enters at least one of L1–L12. If it enters none, it does no endpoint work and cannot resolve the official endpoint.*

Proof. A negative SAT route that resolves the endpoint must either alter the carrier, quantify over the polynomial-time candidate image, depend on encodings or representatives, deny positive endpoint occupation, express lower-bound normal form, exclude the candidate endpoint image, classify candidate status, shift to nonuniform/oracle/promise scope, rely on finite evidence, repair a prior report, select among candidates, or govern endpoint status. These cases cover the ways a same-carrier negative route can do endpoint work. L1 and L8 are carrier shifts; L3 and L11 are bookkeeping or selector failures; L5 and L9 are proof support until endpoint-resolving; L6 and L7 route to endpoint governance; L10 violates irreversibility; L12 is the forbidden same-domain discriminator case. If none of the loci is entered, the route preserves carrier, candidate image, encoding status, branch slot, lower-bound status, image occupation, classifier status, auxiliary scope, evidential status, act-time status, selector status, and endpoint-status governance. No endpoint-relevant work remains. $\square$

Corollary 8.4 (No surviving same-carrier separator route). *No same-carrier endpoint-resolving negative SAT route survives the route-locus audit outside the image-separator route already excluded by Theorem 7.44.*

Proof. By Theorem 8.3, every route enters L1–L12 or does no endpoint work. The non-endpoint routes are proof support or bookkeeping. The carrier-shift routes do not preserve the official

endpoint. Repair and selector routes violate irreversibility or AMetricity. The endpoint-governing routes enter candidate-image exclusion, theorem-level discrimination, or independent-discriminator closure and are excluded by the image-separator argument. Therefore no additional same-carrier endpoint-resolving separator route survives. $\square$

9 Endpoint-Image Resolution

Theorem 9.1 (Bare SAT endpoint complementarity). *At the raw unrestricted finite CNF-SAT layer,*

$$\text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}}.$$

Proof. By Definition 1.5, $\text{Pos}_{\text{raw}} := \text{CnfSATInPolyTime}$, the assertion that arbitrary finite CNF-SAT is decidable by a deterministic polynomial-time machine. The bare negative branch Sep_{bare} is the official raw complement branch: failure of polynomial-time decidability for that same unrestricted finite CNF-SAT carrier. Definition 2.1 prevents any shift to a restricted CNF fragment. Therefore the raw branches are exact complements on the same carrier. $\square$

Theorem 9.2 (Bare endpoint bivalence).

$$\text{Pos}_{\text{raw}} \vee \text{Sep}_{\text{bare}}.$$

Proof. Classical bivalence gives $\text{Pos}_{\text{raw}} \vee \neg \text{Pos}_{\text{raw}}$. By Theorem 9.1, $\neg \text{Pos}_{\text{raw}}$ is equivalent to Sep_{bare} . Hence $\text{Pos}_{\text{raw}} \vee \text{Sep}_{\text{bare}}$. $\square$

Definition 9.3 (Official SAT endpoint evaluation of the raw bivalent split). $\text{OSEval}(R, \text{model})$ means that the bivalent case split between Pos_{raw} and Sep_{bare} is being evaluated as the official P versus NP endpoint on the fixed unrestricted finite CNF-SAT carrier. It is a target-evaluation condition, not a truth assumption: it says that the negative branch of the split, if reached, is considered as the official endpoint-resolving negative branch rather than as lower-bound support, ordinary proposition-targeted theoremhood, or bookkeeping.

Lemma 9.4 (Negative branch in official endpoint evaluation carries endpoint-resolution use). *On the fixed context-bound SAT carrier,*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{OSEval}(R, \text{model}) \wedge \text{Sep}_{\text{bare}} \\ & \Rightarrow \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}}). \end{aligned}$$

Proof. By Definition 9.3, official SAT endpoint evaluation is the official endpoint case split itself. The positive case is direct positive endpoint occupation. The negative case, if entered on the same fixed carrier, is not being used merely as a lower-bound lemma, proof-support observation, ordinary proposition-targeted theorem, or bookkeeping expression. It is the negative branch of the official endpoint evaluation. Hence it carries the endpoint-resolution use $\text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})$. The lemma does not promote arbitrary lower-bound content to endpoint use; it applies only inside the official endpoint evaluation of the raw bivalent split. $\square$

Theorem 9.5 (Positive SAT endpoint). *In the fixed SAT endpoint evaluation,*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \wedge \text{OSEval}(R, \text{model}) \\ & \Rightarrow \text{Pos}_{\text{raw}}. \end{aligned}$$

Proof. Assume the four hypotheses displayed in the theorem. By Theorem 9.1, $\text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}}$, and by Theorem 9.2, either Pos_{raw} or Sep_{bare} . If Pos_{raw} , the conclusion is immediate. If Sep_{bare} , Lemma 9.4 gives $\text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})$. Theorem 7.45 excludes $\text{Sep}_{\text{bare}} \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}})$ under the same fixed context. Contradiction. Therefore the negative endpoint branch is unavailable in the official endpoint evaluation, and Pos_{raw} follows. $\square$

Theorem 9.6 (Instantiated SAT endpoint context). *The official unrestricted finite CNF-SAT endpoint carrier instantiates the fixed context, fixed-domain, bound-model, and official endpoint-evaluation hypotheses:*

$$\begin{aligned} & \mathcal{C}_{\text{SAT}}(R, m)_{R_{\text{SAT}}^{\text{ctx}}} \wedge \text{FixedDomain}(R_{\text{SAT}}^{\text{ctx}}) \wedge \text{Model}_{\text{CNF}}(m_{\text{SAT}}, R_{\text{SAT}}^{\text{ctx}}) \\ & \wedge \text{OSEval}(R_{\text{SAT}}^{\text{ctx}}, m_{\text{SAT}}). \end{aligned}$$

Proof. The official SAT endpoint carrier is arbitrary finite CNF over finite Boolean alphabets with formula length as input size; the positive branch is deterministic polynomial-time CNF-SAT decidability; the raw negative branch is its complement on the same carrier; and admissible encodings preserve the carrier and endpoint image. This is exactly the fixed context of Definition 1.3 together with fixed-domain preservation, the context-bound CNF model of Definition 2.2, and the official endpoint-evaluation condition of Definition 9.3. $\square$

Theorem 9.7 (Instantiated positive SAT endpoint).

$$\text{CnfSATInPolyTime}.$$

Proof. By Theorem 9.6, the official SAT carrier supplies the fixed context, fixed-domain, bound-model, and official endpoint-evaluation hypotheses. Applying Theorem 9.5 with $R = R_{\text{SAT}}^{\text{ctx}}$ and $model = m_{\text{SAT}}$ yields Pos_{raw} . By Definition 1.5, $\text{Pos}_{\text{raw}} := \text{CnfSATInPolyTime}$. $\square$

10 Positive Report Classification

Definition 10.1 (Positive Clay report act). $\text{Report}_{\text{Clay}}(\text{Pos}_{\text{raw}})$ is the report act that presents Pos_{raw} as the positive mathematical endpoint after the separator branch has been excluded.

Theorem 10.2 (Positive report act).

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(model, R) \wedge \text{Pos}_{\text{raw}} \Rightarrow \text{Report}_{\text{Clay}}(\text{Pos}_{\text{raw}}).$$

Proof. Once the fixed SAT context is in force and the raw positive endpoint has been derived after exclusion of the separator branch, the manuscript's theorem-bearing output is the report of Pos_{raw} as the positive mathematical endpoint. That report act does not supply standing; it records the already derived endpoint under the report-preserving interface forced by the kernel. $\square$

Theorem 10.3 (Positive endpoint classification).

$$\mathcal{C}_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(model, R) \wedge \text{Pos}_{\text{raw}} \wedge \text{Report}_{\text{Clay}}(\text{Pos}_{\text{raw}}) \Rightarrow \mathbf{A}_{\text{pos}}.$$

Proof. The positive branch is a theorem-bearing endpoint report on the fixed SAT carrier. By Theorem 7.10, the positive branch is direct constructive endpoint occupation rather than separator classification. By Theorem 3.87, endpoint use is kernel-instantiated; by Theorem 4.2, the report-preserving A+ consequence layer applies. The positive classification $\mathbf{A}_{\text{pos}}$ records that downstream status. It is not a source of standing and is not an independent separator discriminator. $\square$

Corollary 10.4 (Positive Clay mathematical endpoint).

$$C_{\text{SAT}}(R, m) \wedge \text{FixedDomain}(R) \wedge \text{Model}_{\text{CNF}}(\text{model}, R) \Rightarrow \text{Pos}_{\text{Clay}}.$$

Proof. Theorem 9.5 gives Pos_{raw} . Theorem 10.2 gives $\text{Report}_{\text{Clay}}(\text{Pos}_{\text{raw}})$. Theorem 10.3 gives A_{pos} . Therefore $\text{Pos}_{\text{Clay}} := \text{Pos}_{\text{raw}} \wedge A_{\text{pos}}$. $\square$

11 AASC Closure and Endpoint Correspondence

The manuscript does not redefine P , NP , CNF, satisfiability, polynomial time, reduction, or NP-completeness. AASC proves the positive raw SAT endpoint by excluding the separator endpoint image through kernel-forced fixed-domain discriminator closure. Once CnfSATInPolyTime is obtained, the final move to $P = NP$ is the standard Cook–Levin/Karp endpoint correspondence. Cook–Levin/Karp is not the branch-exclusion method; it identifies the official endpoint consequence after the AASC proof has delivered unrestricted finite CNF-SAT in deterministic polynomial time.

Theorem 11.1 (Cook–Levin/Karp endpoint correspondence).

$$\text{CnfSATInPolyTime} \Rightarrow P = NP.$$

Proof. Arbitrary finite CNF-SAT is NP-complete under the standard Cook–Levin/Karp polynomial-time many-one reduction framework. If CNF-SAT has a deterministic polynomial-time decider, then every language in NP is decided in deterministic polynomial time by composing its polynomial-time reduction to CNF-SAT with that decider. Hence $NP \subseteq P$. Since $P \subseteq NP$ by definition, $P = NP$. $\square$

Corollary 11.2 (Positive P versus NP endpoint).

$$P = NP.$$

Proof. Theorem 9.7 gives CnfSATInPolyTime . Theorem 11.1 gives $P = NP$. $\square$

12 Denial Burden and Exhaustion of Remaining Routes

Dependency-inversion objections

An objection that treats the kernel as an optional AASC overlay is not a separate denial route. It is a dependency-inversion objection. Such an objection has standing only if it identifies a failure in the kernel-neutral adequacy argument: target determinacy, step evaluability, act-time finality, or same-regime fidelity must be shown not to be required for theorem-bearing endpoint use, or a same-regime faithful counterexample must be supplied that preserves all four while omitting at least one kernel role. Without that showing, the objection merely renames a forced precondition as optional vocabulary.

Cost of abandoning the kernel

A denial of the kernel is not a harmless refusal of terminology. The kernel is the role-decomposition of the four adequacy conditions required for determinate same-domain theorem-bearing construction. To deny the kernel while still claiming to assess the same endpoint, an objector must identify which adequacy condition is being abandoned and accept the corresponding loss.

Denied adequacy condition	Kernel role lost	Consequence of denial
Target determinacy	Reference	No fixed endpoint object, carrier, candidate image, or branch slot is preserved.
Step evaluability	Standing	No transition has theorem-bearing stephood; lower-bound traces are inscriptions rather than endpoint acts.
Act-time finality	Irreversibility	Same-act repair, later reclassification, and retrospective status change remain open.
Same-regime fidelity	Admissibility boundary	Carrier, domain, encoding image, or endpoint classification may shift under redescription.

Thus kernel denial has only two possible forms. Either it denies none of the four adequacy conditions, in which case the kernel has already been conceded under different vocabulary; or it denies at least one of them, in which case the objector is no longer preserving determinate same-domain theorem-bearing endpoint use. There is no third position from which one may reject the kernel while retaining the same official endpoint-resolution act.

Right-mode objection targets

A standing mathematical objection must attack one of the following load-bearing claims.

1. AASC mathematical status: identify a precise failure of a definition, theorem, inference, kernel dependency, or formal constraint.
2. Lean support boundary: identify a mismatch between a Lean-facing support claim and the manuscript theorem chain, if Lean support is invoked.
3. Kernel-neutral target adequacy: show that target determinacy, step evaluability, act-time finality, or same-regime fidelity is not required for theorem-bearing endpoint use.
4. Non-degenerate SAT endpoint regime forcing the kernel: preserve fixed target, carrier, theorem-status distinction, downstream reuse, redescription stability, and act-time finality while lacking one of *Adm*, *St*, *Ref*, *Irr*.
5. Local SAT packet: show that a used *A+* component is not forced by the kernel plus fixed-domain closure.
6. Strict weakening of *K5*, *K6*, *K11*, or *K13*: define a weaker regime preserving the same endpoint roles while allowing separator governance.
7. Standard encoded CNF model adequacy: identify a concrete failure of coverage, soundness, failure adequacy, or candidate-image exactness.
8. Raw complementarity: show $\text{Sep}_{\text{bare}} \not\leftrightarrow \neg \text{Pos}_{\text{raw}}$ on the unrestricted finite CNF-SAT carrier.
9. SAT-native bridge: show that Sep_{bare} does not yield standard lower-bound normal form, or that standard lower-bound normal form is not candidate endpoint-image exclusion.
10. Theorem-level discriminator: show that candidate endpoint-image exclusion used at endpoint level does not classify endpoint status.
11. Ordinary negation role-classification: show that the proof rejects ordinary negative theoremhood merely for being negative.
12. Coequal-role denial: show that Sep_{bare} is a lawful coequal endpoint role inside the same positive endpoint-image carrier rather than endpoint-status governance.
13. No hidden fifth case: inhabit endpoint-resolving, same-carrier, non-governing candidate-image exclusion.

14. Endpoint governance: show that official negative endpoint use can remain theorem-bearing while avoiding the occupation-exhaustion and endpoint-status-bivalence locks.
15. Independent discriminator: show that the image-separator branch can avoid inducing independent same-domain endpoint-status discrimination under the SAT-local independence bridge.
16. No independent discriminator: show that such an independent same-domain endpoint-status discriminator survives AMetricity, unique interior, standing conservation, transport closure, and structural rigidity.
17. Non-explosiveness: show that the proof globally bans negative complexity mathematics rather than excluding only same-carrier endpoint-resolving separator governance.
18. Route-locus exhaustion: show a same-carrier endpoint-resolving negative SAT route outside L1–L12.
19. Cook–Levin/Karp endpoint correspondence: show that CnfSATInPolyTime does not imply $P = NP$ under the standard NP-completeness correspondence.

Faithful same-regime counterexample worksheet

A successful same-mode objection must complete at least one of the following tasks.

Target	Successful objection must produce
Kernel denial	A theorem-bearing SAT endpoint act preserving fixed carrier, candidate image, branch slots, target-slot fixation, report evaluability, same-regime fidelity, and act-time finality while abandoning a named kernel role.
Strict weakening	A weaker regime W_{sep} preserving the unrestricted finite CNF carrier, encoded candidate image, target-slot fixation, minimal report evaluability, standing-admissibility discipline, act-time finality, and no carrier shift while permitting endpoint-resolving separator governance that is not positive occupation, proof support, bookkeeping, carrier shift, repair, lawful coequal target role, or second endpoint authority.
Model adequacy	A deterministic polynomial-time SAT candidate lost by the encoded image, an unlawful candidate admitted as polynomial-time, an encoding artifact that changes SAT-failure status, or a mismatch between CnfSATInPolyTime and the encoded non-failing candidate image.
Raw negative route	A raw negative SAT branch on the unrestricted finite CNF carrier that does not reduce to standard lower-bound normal form over the fixed candidate image.
Candidate-image bridge	A standard lower-bound normal form that is not candidate endpoint-image exclusion.
Ordinary negation	A proof that the manuscript treats negative theoremhood as defective merely because it is negative.
Coequal endpoint	A proof that Sep_{bare} is a lawful coequal endpoint role inside the same positive endpoint-image carrier.
Endpoint governance	Endpoint-resolving candidate-image exclusion that is not proof support, not bookkeeping, not carrier shift, not positive occupation, and not endpoint-status governance.
Independent discriminator	Image-separator endpoint use that does not induce $\mathcal{D}_{\text{sep}}(\text{model})$, or an induced $\mathcal{D}_{\text{sep}}(\text{model})$ compatible with $\mathcal{N}_{\text{sep}}(\text{model})$.
Final endpoint	A failure of $\neg(\text{Sep}_{\text{bare}} \wedge \text{OfficialEndpointResolution}_R(\neg\text{Pos}_{\text{raw}}))$ in the official endpoint evaluation, or of $(\text{Sep}_{\text{bare}} \leftrightarrow \neg\text{Pos}_{\text{raw}}) \Rightarrow \text{Pos}_{\text{raw}}$, or a failure of the Cook–Levin/Karp correspondence from CnfSATInPolyTime to $P = NP$.

Objections outside these classes are non-standing unless they identify a theorem-level failure in one of the cited bridges. The eliminated negative route is the raw SAT lower-bound/no-positive branch, because that branch transports directly to candidate endpoint-image exclusion and, under endpoint-resolution use, separator endpoint-image occupation; endpoint-resolving separator occupation then induces independent same-domain endpoint-status discrimination under the SAT-local bridge; and the kernel-forced consequence layer excludes such discrimination.

13 Conclusion

The branch-exclusion argument is AASC-internal and kernel-first. The kernel is not an optional overlay; it is forced by the conditions under which any proof, counterproof, endpoint resolution, or theorem-bearing act remains a determinate same-carrier mathematical act. Raw traces do not become constructions merely by being written. Genuine theoremhood is governed derivation and is therefore kernel-internal. Negative theoremhood is not rejected for being negative.

On the fixed unrestricted finite CNF-SAT carrier, the positive branch is direct constructive

endpoint occupation. The raw negative branch is ordinary logical negation at the bare level, but endpoint-bearing use of that negation is role-classified. If it remains proof support or ordinary proposition-targeted theoremhood, it does not resolve the official endpoint. If it changes carrier, it is not the same endpoint. If it is offered as the official negative endpoint resolution, it becomes endpoint use of the negative branch. The SAT-native lower-bound/no-positive bridge then transports it to standard lower-bound normal form, candidate endpoint-image exclusion, and the image-separator branch.

The image-separator branch is not a lawful coequal endpoint role inside the same positive endpoint-image carrier. When used to resolve the endpoint, it performs endpoint-status governance by candidate-image exclusion. The hidden-fifth alternative-endpoint resolution without endpoint-status governance is impossible. Separator endpoint-image occupation invokes the branch-local SAT independence bridge and induces an independent same-domain endpoint-status discriminator. The kernel-forced consequence layer excludes such discriminators. Therefore separator endpoint-image occupation is unavailable, the bare negative branch is excluded as the official endpoint-resolving branch, and official endpoint evaluation of the raw bivalent split plus raw complementarity yields Pos_{raw} . Since Pos_{raw} is CnfSATInPolyTime , arbitrary finite CNF-SAT is decidable in deterministic polynomial time. The standard Cook–Levin/Karp endpoint correspondence then gives $P = NP$.

A Lean4 SAT Operator Bridge Appendix

The manuscript proof above remains the primary proof. The Lean4 material in this appendix is support for the local SAT-operator bridge used by the AASC endpoint proof spine:

$$\begin{aligned} \text{Sep}_{\text{bare}} &\Rightarrow \text{CandImageExcl}(\text{model}), \\ \text{CandImageExcl}(\text{model}) \wedge \text{OfficialEndpointResolution}_R(\neg \text{Pos}_{\text{raw}}) &\Rightarrow \text{Sep}_{\text{img}}^{\text{occ}} \Rightarrow \mathcal{D}_{\text{sep}}(\text{model}) \Rightarrow \perp. \end{aligned}$$

It is not used as a substitute for the external Cook–Levin/Karp endpoint correspondence from CnfSATInPolyTime to $P = NP$. Its role is reproducibility support for the AASC foundation-to-SAT endpoint proof spine: the release archive includes the Minimal Conditions/A+ and non-degenerate-kernel audit layers together with the SAT endpoint-operator bridge and negative-branch closure formalized in the release archive [9, 10].

A.1 Release, environment, and audit runner

The Lean audit archive included with this project is the following source-material zip:

source_material/AASC-PvsNP-SAT-Endpoint-Lean-Audit-1.0.0.zip

The pinned build data recorded in the release are:

Item	Value
Lean toolchain	leanprover/lean4:v4.28.0
Lake project	AASCPvsNPSATEndpointLeanAudit
mathlib requirement	mathlib, revision tag v4.28.0
Pinned mathlib revision	8f9d9cff6bd728b17a24e163c9402775d9e6a365
Main proof queue	MaleyLean/Papers/PvsNP/SATOperatorProofQueue.lean
Focused audit runner	scripts/check-pvsnp-sat-operator-bridge-audit.ps1
Release root suffix	062a2a6

The verification command supplied by the release is:

```
powershell -ExecutionPolicy Bypass -File scripts/check-pvsnp-sat-operator-bridge-audit.ps1
```

The runner prints the Lean toolchain, prints the pinned mathlib manifest revision, builds MaleyLean.Papers.PvsNP.SATOperatorAuditRunners, evaluates the SAT-operator formalization-status and progress summaries, scans the audited AASC foundation plus P vs NP SAT endpoint surface for live axiom, sorry, admit, or unsafe declarations, builds the AASC foundation/kernel support modules, builds the SAT proof queue, antichecker/model-adequacy bridge, and bridge-callability modules, and runs ten audit files: the Minimal Conditions/A+ audit, the non-degenerate-kernel audit, the SAT endpoint checks, and the combined full-stack AASC/SAT check. The release notes record complete closure for the AASC SAT endpoint and CNF-SAT-in-polynomial-time status summaries. Focused axiom output for the bridge anchors is reported against only the standard Lean/classical/mathlib base dependencies `propext`, `Classical.choice`, and `Quot.sound`.

A.2 Declaration locations

The following declaration locations are taken from the released Lean files. They are included to make the manuscript-to-Lean correspondence checkable without treating Lean as a replacement for the manuscript proof.

```
SameDomainEndpoint.lean:22
  abbrev CnfPositiveEndpoint : Prop := CnfSATInPolyTime

FoundationalClassifierBridge.lean:44
  def CnfSeparatingClassifierIsIndependentSameDomain

FoundationalClassifierBridge.lean:70
  def CnfNoIndependentSeparatingClassifier

SATOperatorProofQueue.lean:10031
  def CnfSATImageSeparatorBranch

SATOperatorProofQueue.lean:10042
  def CnfSATBareNegativeBranch

SATOperatorProofQueue.lean:10049
  def CnfSATBareSeparator

SATOperatorProofQueue.lean:10142
  def CnfSATImageSeparatorEndpointUse

SATOperatorProofQueue.lean:10161
  def CnfSATDirectPositiveEndpointOccupation

SATOperatorProofQueue.lean:10169
  def CnfSATIndependentSeparatorEndpointStatusDiscriminator

SATOperatorProofQueue.lean:10184
  def CnfSATBookkeepingNegativeOnly
  def CnfSATCarrierShift
  def CnfSATOfficialNegativeEndpointUse
  def CnfSATNegativeOccupationExhaustion

SATOperatorProofQueue.lean:10313
  inductive CnfSATEndpointStatus
  def CnfSATEndpointStatusOccupation
  def CnfSATGovernedEndpointUse
  def CnfSATGovernedEndpointStatusUse
```

The proof-spine theorem locations are:

```
SATOperatorProofQueue.lean:10075
```

```

cnfBareNegativeBranch_forces_imageSeparatorBranch

SATOperatorProofQueue.lean:10184
  cnfSATNegativeOccupation_exhaustion
  cnfSATNoCarrierShift_of_fixedEndpointDomain
  cnfSATOfficialNegativeEndpointUse_not_bookkeepingOnly
  cnfSATNoOrdinaryNegativeAlternative_of_ametricBoundary
  cnfSATNegativeOccupation_nonoptional

SATOperatorProofQueue.lean:10313
  cnfSATEndpointStatus_exhaustive
  cnfSATGovernedEndpointUse_bivalent
  cnfSATNegativeGovernedEndpointUse_has_separatorStatus
  cnfSATGovernedEndpointStatusUse_eq_separator_of_not_positive
  cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse

SATOperatorProofQueue.lean:10087
  cnfImageSeparatorBranch_of_not_positiveEndpoint

SATOperatorProofQueue.lean:10104
  cnfSATBareSeparator_forces_imageSeparatorBranch

SATOperatorProofQueue.lean:10148
  cnfSATImageSeparatorBranch_endpointUse

SATOperatorProofQueue.lean:10179
  cnfPositiveEndpoint_directEndpointOccupation

SATOperatorProofQueue.lean:10190
  cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator

SATOperatorProofQueue.lean:10212
  cnfBareNegativeBranch_requires_independentSeparatorDiscriminator

SATOperatorProofQueue.lean:10246
  cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10267
  cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10286
  cnfNotPositiveEndpoint_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10308
  cnfPositiveEndpoint_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10331
  cnfSATInPolyTime_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10351
  cnfSATInPolyTime_of_context_noIndependentDiscriminator

SATOperatorProofQueue.lean:10372
  cnfSATBareSeparator_impossible_of_context_noIndependentDiscriminator

```

A.3 Manuscript-to-Lean correspondence

The manuscript-to-Lean symbol correspondence is:

PosRaw	<-> CnfPositiveEndpoint (abbrev. CnfSATInPolyTime)
SepBare	<-> CnfSATBareNegativeBranch R model
SepImg	<-> CnfSATImageSeparatorBranch R model (Lean-side image branch)
SepImgOcc	<-> endpoint-occupation use of CnfSATImageSeparatorBranch R model
DirPos	<-> CnfSATDirectPositiveEndpointOccupation
SepDisc(model)	<-> CnfSATIndependentSeparatorEndpointStatusDiscriminator model
NoSepDisc(model)	<-> CnfNoIndependentSeparatingClassifier model

```

SepIndBridge(model)
  <-> CnfSeparatingClassifierIsIndependentSameDomain model
OfficialNegUse  <-> CnfSATOfficialNegativeEndpointUse R model
NegOccExhaust  <-> CnfSATNegativeOccupationExhaustion R model
GovernedUse     <-> CnfSATGovernedEndpointUse R model
EndpointStatus  <-> CnfSATEndpointStatus

```

The formal correspondence of the central manuscript chain is:

```

CnfPositiveEndpoint
  -> CnfSATDirectPositiveEndpointOccupation

CnfSATBareNegativeBranch R model
  -> CnfSATImageSeparatorBranch R model

CnfSATOfficialNegativeEndpointUse R model
  + CnfSATFixedEndpointDomain R
  + CnfSATContextBoundCNFModel R model
  + AMetricBoundary R
  -> CnfSATImageSeparatorBranch R model

CnfSATGovernedEndpointUse R model
  + Not CnfPositiveEndpoint
  -> CnfSATImageSeparatorBranch R model

CnfSeparatingClassifierIsIndependentSameDomain model
  -> CnfSATImageSeparatorBranch R model
  -> CnfSATIndependentSeparatorEndpointStatusDiscriminator model

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> Not (CnfSATImageSeparatorBranch R model)

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> Not (CnfSATBareNegativeBranch R model)

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> CnfSATInPolyTime

```

A.4 Selected source excerpts

The following excerpts are reproduced from `SATOperatorProofQueue.lean`. They are the Lean-side anchors for the raw-negative bridge, endpoint-use marker, branch asymmetry, and final SAT endpoint closeout.

```

def CnfSATImageSeparatorBranch
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  cnfDirectGateLowerBoundResidualTarget

def CnfSATBareNegativeBranch
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  Not CnfPositiveEndpoint

theorem cnfBareNegativeBranch_forces_imageSeparatorBranch
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hBare : CnfSATBareNegativeBranch R model) :
  CnfSATImageSeparatorBranch R model :=

```

<pre> (cnfDirectGateLowerBoundResidualTarget_iff_noSatInPolyTime).2 hBare theorem cnfImageSeparatorBranch_of_not_positiveEndpoint {Act Object : Type} {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object} {model : CnfEncodedCandidateModel} (hNotPositive : Not CnfPositiveEndpoint) : CnfSATImageSeparatorBranch R model := cnfBareNegativeBranch_forces_imageSeparatorBranch (R := R) (model := model) hNotPositive </pre>
<pre> def CnfSATDirectPositiveEndpointOccupation : Prop := CnfPositiveEndpoint def CnfSATIndependentSeparatorEndpointStatusDiscriminator (model : CnfEncodedCandidateModel) : Prop := exists classifier : CnfCandidateStatusClassifier model, CnfClassifierSeparatesPolynomialCandidates model classifier /\ Nonempty (CnfClassifierIndependentSameDomain model classifier) theorem cnfPositiveEndpoint_directEndpointOccupation (hPositive : CnfPositiveEndpoint) : CnfSATDirectPositiveEndpointOccupation := hPositive theorem cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator {Act Object : Type} {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object} {model : CnfEncodedCandidateModel} (hIndependent : CnfSeparatingClassifierIsIndependentSameDomain model) (hSep : CnfSATImageSeparatorBranch R model) : CnfSATIndependentSeparatorEndpointStatusDiscriminator model := by have hSameDomain : CnfSameDomainSeparator := (cnfDirectGateLowerBoundResidualTarget_iff_sameDomainSeparator).1 hSep let classifier := cnfClassifierOfSameDomainSeparator model hSameDomain have hSeparates : CnfClassifierSeparatesPolynomialCandidates model classifier := cnfClassifierSeparates_of_sameDomainSeparator model hSameDomain exact Exists.intro classifier (And.intro hSeparates (hIndependent classifier hSeparates)) </pre>
<pre> theorem cnfBareNegativeBranch_requires_independentSeparatorDiscriminator {Act Object : Type} {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object} {model : CnfEncodedCandidateModel} (hIndependent : CnfSeparatingClassifierIsIndependentSameDomain model) (hBare : CnfSATBareNegativeBranch R model) : CnfSATIndependentSeparatorEndpointStatusDiscriminator model := cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator (R := R) hIndependent (cnfBareNegativeBranch_forces_imageSeparatorBranch hBare) theorem cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator {Act Object : Type} {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object} {model : CnfEncodedCandidateModel} (hNoIndependent : CnfNoIndependentSeparatingClassifier model) (hIndependent : CnfSeparatingClassifierIsIndependentSameDomain model) : Not (CnfSATImageSeparatorBranch R model) := by intro hSep exact Exists.elim </pre>

```

(cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator
  (R := R)
  hIndependent hSep)
(fun classifier hClassifier =>
  hNoIndependent classifier hClassifier.1 hClassifier.2)

```

```

theorem cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
  (hIndependent :
    CnfSeparatingClassifierIsIndependentSameDomain model) :
  Not (CnfSATBareNegativeBranch R model) := by
intro hBare
exact
  (cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator
    (R := R)
    hNoIndependent hIndependent)
  (cnfBareNegativeBranch_forces_imageSeparatorBranch hBare)

theorem cnfNotPositiveEndpoint_impossible_of_noIndependentDiscriminator
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
  (hIndependent :
    CnfSeparatingClassifierIsIndependentSameDomain model)
  (hNotPositive : Not CnfPositiveEndpoint) :
  False :=
(cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator
  (R := R)
  (model := model)
  hNoIndependent hIndependent)
  hNotPositive

```

```

theorem cnfSATInPolyTime_of_noIndependentDiscriminator
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
  (hIndependent :
    CnfSeparatingClassifierIsIndependentSameDomain model) :
  CnfSATInPolyTime :=
cnfPositiveEndpoint_of_noIndependentDiscriminator
  (R := R)
  (model := model)
  hNoIndependent hIndependent

theorem cnfSATInPolyTime_of_context_noIndependentDiscriminator
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (_hCtx : CnfSATClayEndpointImageContext R model)
  (_hFixed : CnfSATFixedEndpointDomain R)
  (_hModel : CnfSATContextBoundCNFModel R model)
  (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
  (hIndependent :
    CnfSeparatingClassifierIsIndependentSameDomain model) :
  CnfSATInPolyTime :=
cnfSATInPolyTime_of_noIndependentDiscriminator
  (R := R)
  (model := model)
  hNoIndependent hIndependent

```

```

def CnfSATBookkeepingNegativeOnly

```

```

    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

def CnfSATCarrierShift
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

def CnfSATOfficialNegativeEndpointUse
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  CnfSATBareNegativeBranch _R _model

def CnfSATNegativeOccupationExhaustion
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  CnfSATImageSeparatorBranch _R _model \ /
  CnfOrdinaryNegativeAlternative _R _model \ /
  CnfSATBookkeepingNegativeOnly _R _model \ /
  CnfSATCarrierShift _R _model

```

```

theorem cnfSATNegativeOccupation_nonoptional
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hOfficial : CnfSATOfficialNegativeEndpointUse R model)
  (hFixed : CnfSATFixedEndpointDomain R)
  (hModel : CnfSATContextBoundCNFModel R model)
  (hAMetric : AMetricBoundary R) :
  CnfSATImageSeparatorBranch R model :=
  -- formal proof in SATOperatorProofQueue.lean

```

```

inductive CnfSATEndpointStatus where
  | positive
  | separator

def CnfSATGovernedEndpointUse
  {Act Object : Type}
  (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
  (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

theorem cnfSATNegativeGovernedEndpointUse_has_separatorStatus
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hGov : CnfSATGovernedEndpointUse R model)
  (hNotPositive : Not CnfPositiveEndpoint) :
  CnfSATImageSeparatorEndpointUse R model :=
  -- formal proof in SATOperatorProofQueue.lean

theorem cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse
  {Act Object : Type}
  {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
  {model : CnfEncodedCandidateModel}
  (hGov : CnfSATGovernedEndpointUse R model)
  (hNotPositive : Not CnfPositiveEndpoint) :
  CnfSATImageSeparatorBranch R model :=
  -- formal proof in SATOperatorProofQueue.lean

```

A.5 Focused axiom-output table

The focused audit files in the release issue `#print axioms` commands for the AASC foundation anchors and the SAT endpoint bridge anchors below. The release records the following focused axiom surface for these proof-spine theorems: the AASC foundation anchors report no project axioms, and the SAT bridge anchors report only the standard Lean/classical/mathlib base dependencies `propext`, `Classical.choice`, and `Quot.sound`. No project-specific axiom, `sorry`, `admit`, or `unsafe` declaration is used by these anchors.

The model-adequacy audit in `AnticheckerReduction.lean` names the ordinary coding bridge consumed by the SAT-native route. It records coverage, sound decoding, failure adequacy, candidate-image exactness, and the positive/negative endpoint equivalences for encoded deterministic polynomial-time CNF-SAT candidates. The final Lean audit also names the SAT-native form of the remaining bridge objection. The lower-bound and theorem-level discriminator anchors record that the bare negative branch gives lower-bound normal form over the fixed encoded candidate image, and that the resulting discriminator is theorem-level rather than algorithmic. The invariant-use and candidate-image-use classifications separate legitimate proof support from forbidden endpoint-status governance. The closing hinge is formalized by the following Lean anchors:

```
CnfSATCandidateImageExclusionUseKind
CnfSATCandidateImageExclusionUseClassification
CnfSATEndpointStatusGovernanceByCandidateImageExclusion
cnfSATEndpointResolvingNonGovernance_hiddenFifthCase_impossible
cnfSATOfficialNegativeEndpointUse_endpointStatusGovernance
cnfSATEndpointResolvingNegativeTheorem_is_endpointStatusGovernance
```

Lean theorem	Focused axiom output
cnfBareNegativeBranch	[propext, Classical.choice, Quot.sound]
forces_imageSeparatorBranch	[propext, Classical.choice, Quot.sound]
cnfImageSeparatorBranch	[propext, Classical.choice, Quot.sound]
of_not_positiveEndpoint	[propext, Classical.choice, Quot.sound]
cnfSATImageSeparatorBranch	[propext, Classical.choice, Quot.sound]
endpointUse	[propext, Classical.choice, Quot.sound]
cnfSATImageSeparatorBranch	[propext, Classical.choice, Quot.sound]
requires_independentSeparatorDiscriminator	[propext, Classical.choice, Quot.sound]
cnfSATImageSeparatorBranch	[propext, Classical.choice, Quot.sound]
impossible_of_noIndependentDiscriminator	[propext, Classical.choice, Quot.sound]
cnfBareNegativeBranch	[propext, Classical.choice, Quot.sound]
requires_independentSeparatorDiscriminator	[propext, Classical.choice, Quot.sound]
cnfBareNegativeBranch	[propext, Classical.choice, Quot.sound]
impossible_of_noIndependentDiscriminator	[propext, Classical.choice, Quot.sound]
cnfNotPositiveEndpoint	[propext, Classical.choice, Quot.sound]
impossible_of_noIndependentDiscriminator	[propext, Classical.choice, Quot.sound]
cnfSATInPolyTime	[propext, Classical.choice, Quot.sound]
of_context_noIndependentDiscriminator	[propext, Classical.choice, Quot.sound]
cnfSATNegativeOccupation	[propext, Classical.choice, Quot.sound]
exhaustion	[propext, Classical.choice, Quot.sound]
cnfSATNegativeOccupation	[propext, Classical.choice, Quot.sound]
nonoptional	[propext, Classical.choice, Quot.sound]
cnfSATGovernedEndpointUse	[propext, Classical.choice, Quot.sound]
bivalent	[propext, Classical.choice, Quot.sound]
cnfSATNegativeGovernedEndpointUse	[propext, Classical.choice, Quot.sound]
has_separatorStatus	[propext, Classical.choice, Quot.sound]
cnfSATImageSeparatorBranch	[propext, Classical.choice, Quot.sound]
of_negativeGovernedEndpointUse	[propext, Classical.choice, Quot.sound]
cnfSATCandidateModelCoverage	[propext, Classical.choice, Quot.sound]
cnfSATCandidateModelSoundness	[propext, Classical.choice, Quot.sound]
cnfSATCandidateFailureAdequacy	[propext, Classical.choice, Quot.sound]
cnfSATCandidateImageExactness	[propext, Classical.choice, Quot.sound]
cnfEncodedCandidateModelAdequate	[propext, Classical.choice, Quot.sound]
cnfSATInPolyTime_iff_exists	[propext, Classical.choice, Quot.sound]
_encoded_nonfailing_candidate	[propext, Classical.choice, Quot.sound]
cnfSATNotInPolyTime_iff	[propext, Classical.choice, Quot.sound]
_all_encoded_candidates_fail	[propext, Classical.choice, Quot.sound]
cnfSATBareNegativeBranch	[propext, Classical.choice, Quot.sound]
standardLowerBoundNormalForm	[propext, Classical.choice, Quot.sound]
cnfSATBareNegativeBranch	[propext, Classical.choice, Quot.sound]
theoremLevelDiscriminator	[propext, Classical.choice, Quot.sound]
cnfSATOfficialNegativeEndpointUse	[propext, Classical.choice, Quot.sound]
endpointStatusGovernance	[propext, Classical.choice, Quot.sound]
cnfSATEndpointResolvingNonGovernance	[propext, Classical.choice, Quot.sound]
hiddenFifthCase_impossible	[propext, Classical.choice, Quot.sound]
cnfSATEndpointResolvingNegativeTheorem	[propext, Classical.choice, Quot.sound]
is_endpointStatusGovernance	[propext, Classical.choice, Quot.sound]

A.6 Audit scope and boundary

The Lean archive formalizes the AASC foundation audit surface consumed by the SAT route together with the SAT-operator bridge and negative-branch closeout used in the manuscript proof spine. The SAT-operator proof queue also includes negative-occupation-exhaustion anchors and endpoint-status-bivalence anchors, corresponding to commits 282d4ad and a938648 in the public Lean repository; commit f2af564 exposes the combined full-stack AASC/SAT audit check. It does not attempt to formalize the external Cook–Levin/Karp theorem inside Lean. Its support role is local and reproducibility-oriented: it checks that the AASC foundation anchors and the formal SAT-side branch chain used by the manuscript have the advertised Lean anchors and that the audit surface is free of live project-level incomplete declarations.

B Standard CNF-SAT to $P=NP$ Correspondence

The manuscript proof derives CnfSATInPolyTime . Here “bounded-CNF” means finitely encoded CNF formulas of arbitrary instance size over a finite Boolean alphabet; it does not mean a fixed-width, fixed-size, bounded-variable, fixed-clause, bounded-depth, promise-restricted, parameterized, or otherwise tractable restricted CNF fragment. The equality $P = NP$ then uses the standard Cook–Levin/Karp endpoint correspondence: arbitrary finite CNF-SAT is NP-complete under polynomial-time many-one reductions; every $L \in NP$ reduces to CNF-SAT; composition of a polynomial reduction with a polynomial SAT decider gives a polynomial decider for L ; and $P \subseteq NP$. Hence $NP \subseteq P$ and $P = NP$. This correspondence is standard Cook–Levin/Karp background. It is not the proof of separator exclusion; AASC supplies that proof.

C Reference Integrity Appendix

The proof ladder is ordered so that each theorem cites prior results only. The final endpoint theorem cites the context definition, official endpoint-evaluation definition, the standard encoded-model instantiation, the encoded candidate model-adequacy packet, positive direct-occupation theorem, model binding, the branch-local SAT image-separator bridge, raw complementarity, the raw-negative-to-image bridge, image-separator exclusion, and the standard CNF-SAT completeness bridge. No theorem cites itself.

D Theorem Ladder and Anti-Circularity Audit

Theorem ladder

1. Mathematical-status lock and Lean support boundary fix the proof as a manuscript theorem chain with Lean as audit support.
2. Kernel-neutral target adequacy fixes target determinacy, step evaluability, act-time finality, and same-regime fidelity.
3. Non-degenerate construction forces Adm , St , Ref , Irr , and endpoint use is governed construction.
4. The local SAT kernel packet records the SAT-facing K1–K13 consequences.
5. Weakening-resistance shows that K5, K6, K11, and K13 cannot be strictly weakened while preserving same-carrier endpoint use and allowing separator governance.
6. Encoded candidate model adequacy fixes coverage, soundness, failure adequacy, and candidate-image exactness for the unrestricted finite CNF-SAT carrier.
7. Raw complementarity gives $\text{Sep}_{\text{bare}} \leftrightarrow \neg \text{Pos}_{\text{raw}}$.
8. Official SAT endpoint evaluation of the raw bivalent split supplies endpoint-resolution use for the negative branch only when that branch is reached inside the official endpoint case split.
9. The SAT-native bridge routes Sep_{bare} to standard lower-bound normal form and candidate endpoint-image exclusion; official endpoint-resolution use promotes that content to $\text{Sep}_{\text{img}}^{\text{occ}}$, and separator occupation explicitly contains the support-level candidate-image exclusion content.
10. Ordinary negation remains ordinary negation; endpoint-bearing negation is role-classified.
11. The separator branch is not a lawful coequal endpoint role inside the same positive endpoint-image carrier.
12. Endpoint-resolving candidate-image exclusion is endpoint-status governance; endpoint-resolving non-governance is the hidden fifth case and is impossible.

13. Candidate-image exclusion remains lawful as proof support; only endpoint-resolving image-separator use invokes $\mathcal{I}_{\text{sep}}(\text{model})$ and induces $\mathcal{D}_{\text{sep}}(\text{model})$, while image-separator endpoint use also gives $\mathcal{N}_{\text{sep}}(\text{model})$.
14. The contradiction excludes $\text{Sep}_{\text{img}}^{\text{occ}}$, hence excludes Sep_{bare} as the official endpoint-resolving negative branch.
15. Bare complementarity yields Pos_{raw} , i.e. CnfSATInPolyTime .
16. Cook–Levin/Karp gives $P = NP$.

Anti-circularity audit

Potential circle or misreading	Audit response
AASC is merely philosophical.	The manuscript uses AASC as a mathematical constraint formalism. A same-mode objection must identify a failed definition, theorem, dependency claim, or bridge inference.
Kernel governance is optional.	Kernel roles are forced by target determinacy, step evaluability, act-time finality, and same-regime fidelity. Denying the kernel denies non-degenerate endpoint use or supplies a faithful weaker regime.
Endpoint target assumes $P = NP$.	The official target fixes the carrier and endpoint slots. It does not assert Pos_{raw} .
Official endpoint evaluation smuggles in the positive result.	No. $\text{OSEval}(R, \text{model})$ says only that the raw bivalent split is being evaluated as the official endpoint case split. It supplies endpoint-resolution use to the negative branch only if the negative branch is reached; it does not assert the positive branch.
$\mathcal{C}_{\text{SAT}}(R, m)$ smuggles positivity.	The context non-smuggling theorem states that $\mathcal{C}_{\text{SAT}}(R, m)$ fixes only carrier and branch slots.
Bounded-CNF means a restricted SAT fragment.	Definition 2.1 states that it means arbitrary finite encoded CNF-SAT, not fixed-width or promise SAT.
The encoded model omits candidates.	Model adequacy requires coverage, soundness, failure adequacy, and image exactness. A valid objection must identify a concrete failure of one of these clauses.
Ordinary negative theoremhood is banned.	Theorems 7.2 and 7.4 explicitly allow ordinary negative theoremhood.
Sep_{bare} is retyped by fiat.	No one-step retyping is used: Sep_{bare} passes through standard lower-bound normal form and candidate endpoint-image exclusion before endpoint-governance classification.
$\text{Sep}_{\text{img}}^{\text{occ}}$ is an arbitrary extra layer.	$\text{Sep}_{\text{img}}^{\text{occ}}$ is the endpoint-occupation form of candidate-image exclusion under official endpoint-resolution use; support-level lower-bound content remains $\text{CandImageExcl}(\text{model})$.
Candidate-image exclusion is an algorithmic separator.	The theorem-level discriminator is not a witness selector or algorithmic separator; it classifies endpoint status for already-fixed candidate roles.
Candidate-image exclusion is automatically forbidden.	No. Candidate-image exclusion may be proof support, a lower-bound lemma, or ordinary theorem content. It invokes the independence bridge only under same-carrier endpoint-resolving use. The separate unpacking lemma says only that $\text{Sep}_{\text{img}}^{\text{occ}}$ contains $\text{CandImageExcl}(\text{model})$, not conversely.
$\text{Sep}_{\text{img}}^{\text{occ}}$ collapses support use into endpoint use.	No. Support-level content is $\text{CandImageExcl}(\text{model})$. The symbol $\text{Sep}_{\text{img}}^{\text{occ}}$ is reserved for endpoint-image branch occupation under official endpoint-resolution use on the fixed carrier.

The proof bans lower bounds globally.	The non-explosiveness theorem permits lower bounds, barriers, obstructions, and proof support. Only same-carrier endpoint-resolving separator governance is excluded.
K5 bans multiple candidate procedures.	K5 bans plural endpoint-governing interiors, not multiple candidates, encodings, proof routes, or support structures.
K13 bans open problem status.	K13 classifies open, unknown, deferred, and support statuses as epistemic or report statuses, not endpoint truth branches.
Target-slot fixation smuggles in K13.	Target-slot fixation identifies what is evaluated. K13 separately proves that hidden-fifth, deferred, and bookkeeping statuses cannot occupy endpoint truth.
Minimal report evaluability smuggles in K11.	Minimal report evaluability classifies a report by role. K11 separately imposes no-overreporting: endpoint force may not exceed carrier and bridge support.
Cook–Levin/Karp is used as branch-exclusion machinery.	Cook–Levin/Karp appears only after <code>CnfSATInPolyTime</code> is proved, as the standard endpoint correspondence to $P = NP$.
Lean is substituted for the proof.	The manuscript is the proof. Lean records a reproducibility audit of the SAT bridge and kernel consequence surface.

E Notation Ledger

Symbol	Meaning	Layer
$\mathcal{C}_{\text{SAT}}(R, m)$	fixed SAT endpoint carrier and branch-slot context	context
Pos_{raw}	<code>CnfSATInPolyTime</code>	raw positive
Sep_{bare}	raw complement branch $\neg \text{Pos}_{\text{raw}}$	bare negative
Sep_{img}	Lean-side formal SAT image-separator branch <code>CnfSATImageSeparatorBranch($R, model$)</code> ; in the manuscript proof it is separated into support-level <code>CandImageExcl($model$)</code> and endpoint occupation $\text{Sep}_{\text{img}}^{\text{occ}}$	Lean bridge
$\text{Sep}_{\text{img}}^{\text{occ}}$	separator endpoint-image occupation: candidate-image exclusion used as the official endpoint-resolving negative branch on the fixed carrier; it contains <code>CandImageExcl($model$)</code> but is not entailed by support-level exclusion alone	endpoint occupation
EndpointUse	branch B offered as theorem-bearing endpoint use	endpoint act
$\text{OfficialEndpointResolution}$	ordinary theoremhood of $\neg \text{Pos}_{\text{raw}}$ offered as the official negative endpoint resolution	endpoint act
OSEval	official evaluation of the raw Pos_{raw} / Sep_{bare} bivalent split on the fixed unrestricted CNF-SAT endpoint carrier; not a truth assumption	endpoint evaluation
EndpointDisc	endpoint-status discriminator attached to branch B	consequence layer
IndependentDisc	independent same-domain governance work by C	forbidden case
$\mathcal{P}_{\text{dir}}$	direct constructive occupation of the positive CNF-SAT endpoint	SAT asymmetry
$\mathcal{D}_{\text{sep}}$	independent same-domain endpoint-status discriminator induced by separator endpoint-image occupation	SAT asymmetry

$\mathcal{I}_{\text{sep}}$	branch-local SAT independence bridge invoked by endpoint-resolving $\text{Sep}_{\text{img}}^{\text{occ}}$ on the context-bound CNF model	SAT asymmetry
$\mathcal{N}_{\text{sep}}$	kernel-forced no-independent-separator-classifier closure triggered by separator endpoint-image use	consequence layer
$\text{Model}_{\text{CNF}}$	context-bound encoded CNF-SAT model for the fixed SAT endpoint carrier	model adequacy
Adeq_{CNF}	model-adequacy packet: coverage, soundness, failure adequacy, and candidate-image exactness	model adequacy
$\text{Coverage}_{\text{CNF}}$	every deterministic polynomial-time CNF-SAT candidate has an encoded polynomial-time code	model adequacy
$\text{Sound}_{\text{CNF}}$	every encoded polynomial-time code decodes to a polynomial-time CNF-SAT candidate	model adequacy
$\text{FailAdeq}_{\text{CNF}}$	equal decoded procedures have the same SAT-failure status	model adequacy
$\text{ImageExact}_{\text{CNF}}$	CnfSATInPolyTime is exactly existence of an encoded non-failing polynomial-time candidate	model adequacy
$\text{StdModel}_{\text{CNF}}$	standard uniform encoded model whose codes are machine/bound pairs (e, k) over arbitrary finite CNF inputs	model adequacy
FailsSAT	wrong answer, unlawful output, nontermination within the declared bound, or bound failure on some lawful finite CNF input	model adequacy
StdLB	standard SAT lower-bound normal form: every encoded polynomial-time candidate fails SAT on the fixed CNF carrier	SAT-native bridge
CandImageExcl	candidate endpoint-image exclusion: no encoded polynomial-time candidate occupies the positive SAT endpoint image	SAT-native bridge
ThmLevelDisc	theorem-level candidate-status discriminator; not an algorithmic witness selector	SAT-native bridge
CandImageUse	candidate-image exclusion is lawful support unless used as endpoint-resolving separator governance	closing hinge
CandImageGov	endpoint-status governance by candidate-image exclusion	closing hinge
CandImageUseKind	four-way use classification for candidate-image exclusion	closing hinge
OfficialNegUse	official negative endpoint use on the context-bound finite CNF-SAT model	SAT negative occupation
NegOccExhaust	four-way exhaustion of official negative occupation alternatives	SAT negative occupation
GovUse	governed endpoint use on the finite CNF-SAT endpoint carrier	endpoint-status bivalence
EndpointStatus	two-status Lean surface: positive or separator	endpoint-status bivalence
A^+	kernel-forced consequence package for endpoint use	consequence layer
A_{sep}	role-specific separator status factor, downstream only	classification
A_{pos}	positive report status factor, downstream only	classification
PosClay	$\text{Pos}_{\text{raw}} \wedge A_{\text{pos}}$	positive Clay mathematical endpoint

K_{SAT}	local SAT kernel packet forced by non-degenerate fixed-carrier endpoint use	local kernel
$C_{0,\text{SAT}}$	core SAT endpoint adequacy packet: reference, standing, admissibility, irreversibility, target determinacy, report evaluability, act-time finality, and same-regime fidelity	weakening resistance
W_{sep}	alleged strict weaker same-carrier regime permitting separator governance	weakening resistance
OrdNeg	ordinary negation is granted; endpoint-bearing negation is	SAT branch
RoleClass	role-classified	audit
SepNot	separator branch is not a lawful coequal endpoint role	SAT branch
Coequal	inside the positive endpoint-image carrier	audit
NegComplexity	negative complexity results remain lawful unless used as	non-
Lawful	forbidden endpoint governance	explosiveness
RouteLocus _{SAT}	route-locus audit for endpoint-resolving negative SAT routes	route exhaustion

References

- [1] S. Cook. *The P versus NP Problem*. Clay Mathematics Institute Millennium Problem description.
- [2] A. J. Maley. *Non-Degenerate Construction and the Kernel of Admissibility*. 2026.
- [3] A. J. Maley. *Claim Standing and Legitimacy*. 2026.
- [4] A. J. Maley. *Anchor, Tensor, and Skin: A Fixed-Domain Decomposition Theorem for Admissible Description*. 2026.
- [5] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pp. 151–158, 1971.
- [6] L. A. Levin. Universal search problems. *Problemy Peredachi Informatsii*, 9(3):115–116, 1973. English translation in *Problems of Information Transmission*, 9(3):265–266, 1973.
- [7] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, editors, *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.
- [8] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [9] A. J. Maley. *AASC P vs NP SAT Endpoint Lean Audit*, version 1.0.0. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.20593794>.
- [10] A. J. Maley. *AASC-PvsNP-SAT-Endpoint-Lean-Audit*. GitHub repository, 2026. <https://github.com/somamaley-ux/AASC-PvsNP-SAT-Endpoint-Lean-Audit>.